

Custom Solver Generation for Quadratic Objective Second-Order Cone Programs

Govind M. Chari^{*}, Behçet Açıkmeşe[†]

University of Washington, Seattle, WA 98195, USA

March 16, 2025

Abstract

Second-order cone programs (SOCPs) with quadratic objective functions are common in optimal control and other fields. Most SOCP solvers which use interior-point methods are designed for linear objectives and convert quadratic objectives into linear ones via slack variables and extra constraints, despite the computational advantages of handling quadratic objectives directly. In applications like model-predictive control and online trajectory optimization, these SOCPs have known sparsity structures and require rapid solutions. When solving these problems, most solvers use sparse linear algebra routines, which introduce computational overhead and hinder performance. In contrast, custom linear algebra routines can exploit the known sparsity structure of problem data and be significantly faster. This work makes two key contributions: (1) the development of QOCO, an open-source C-based solver for quadratic objective SOCPs, and (2) the introduction of QOCOGEN, an open-source custom solver generator for quadratic objective SOCPs, which generates a solver written in C that leverages custom linear algebra. Both implement a primal-dual interior-point method with Mehrotra’s predictor-corrector. Our benchmarks show that QOCO is faster and more robust than many commonly used solvers, and solvers generated by QOCOGEN are significantly faster than QOCO and are free of dynamic memory allocation making them an attractive option for real-time optimization on resource-constrained embedded systems.

1 Introduction

We consider the quadratic objective second-order cone program (SOCP)

$$\begin{aligned} & \underset{x}{\text{minimize}} && \frac{1}{2}x^\top Px + c^\top x \\ & \text{subject to} && Gx \preceq_{\mathcal{K}} h \\ & && Ax = b, \end{aligned} \tag{1}$$

with optimization variable $x \in \mathbb{R}^n$. The cost is defined by the positive semidefinite matrix $P = P^\top \succeq 0$ and vector $c \in \mathbb{R}^n$. The constraints are defined by matrices $A \in \mathbb{R}^{p \times n}$ and $G \in \mathbb{R}^{m \times n}$ and vectors $b \in \mathbb{R}^p$ and $h \in \mathbb{R}^m$. The generalized inequality $\preceq_{\mathcal{K}}$ denotes membership in a closed, proper cone $\mathcal{K}$, i.e. $h - Gx \in \mathcal{K}$. We restrict $\mathcal{K}$ to be the Cartesian product

$$\mathcal{K} = \mathcal{C}_1 \times \mathcal{C}_2 \times \cdots \times \mathcal{C}_K,$$

where $\mathcal{C}_i$ is either the non-negative orthant or a second-order cone. Each cone $\mathcal{C}_i$ corresponds to a subset of constraints, inducing a partition on G and h , i.e., $h_i - G_i x \in \mathcal{C}_i$ for $i = 1, \dots, K$. Throughout this work, we assume that Problem 1 is **feasible** and has a **bounded** optimal objective.

^{*}Ph.D. Student, William E. Boeing Department of Aeronautics & Astronautics; gchari@uw.edu

[†]Professor, William E. Boeing Department of Aeronautics & Astronautics; behcet@uw.edu

Problem 1 appears in various applications, including model predictive control (MPC) [1], network flow optimization [2], portfolio optimization [3, 4], robust optimization [5, 6], and filter design [7], among others. Additionally, the solution of SOCPs is used as a subroutine in algorithms for nonconvex optimization such as sequential convex programming (SCP) [8, 9, 10, 11].

Problem 1 can be reformulated as a SOCP with a linear objective by introducing a slack variable t ,

$$\begin{aligned} & \underset{x,t}{\text{minimize}} && t + c^\top x \\ & \text{subject to} && \left\| \begin{bmatrix} t - \frac{1}{2} \\ P^{1/2}x \end{bmatrix} \right\|_2 \leq t + \frac{1}{2} \\ & && Gx \preceq_{\mathcal{K}} h \\ & && Ax = b. \end{aligned}$$

If P is large (has many rows and columns), then computing $P^{1/2}$ can be prohibitively expensive, making this reformulation impractical. Solving this reformulation can also be slow if the matrix square root $P^{1/2}$ has significantly more nonzero elements than P . Thus it is advantageous for a SOCP solver to natively handle quadratic objective functions without resorting to the linear objective reformulation.

Some SOCP solvers that can solve Problem 1 directly include CLARABEL [12], GUROBI [13], COSMO [14], and SCS [15]. However, COSMO and SCS implement operator-splitting methods. These methods are preferred for large-scale problems due to their low per-iteration cost but can struggle with problem scaling, conditioning, and achieving high-accuracy solutions [16]. For modestly sized SOCPs, an interior-point method (IPM) is preferred, due to its robustness to scaling and conditioning of the problem, and ability to converge to high accuracy solutions [17, Chapter 1]. One of the few open-source IPMs that can directly solve Problem 1 is CLARABEL. In addition to solving SOCPs, CLARABEL can also handle semidefinite programs, as well as problems involving exponential cones, power cones, and generalized power cones. Additionally, it supports infeasibility detection. However, CLARABEL is written in Rust. Although Rust is memory-safe due to its ownership system and can be used in embedded systems, its ecosystem is far less mature than that of C, and for legacy systems, such as aerospace software, C is still preferred.

Many engineering applications including linear MPC, nonlinear MPC [18], portfolio backtesting, sequential quadratic programming (SQP) [19, 20], and SCP, require solving SOCPs with a fixed, known sparsity structures in P, A, G and a constant cone $\mathcal{K}$. These problems often arise in real-time settings, where computational efficiency is critical. For example, sequential convex programming (SCP) iteratively solves a sequence of SOCPs with quadratic objectives and identical sparsity patterns to find stationary points of nonconvex problems. SCP has been widely applied in aerospace trajectory optimization, where real-time performance is essential. Notable applications include powered-descent guidance [21, 22, 23], in-space rendezvous [24, 25], and hypersonic entry guidance [26, 27]. SCP is also the solution method used by NASA’s SPLICE program for lunar landing guidance [28, 29, 30, 31].

Most general-purpose SOCP solvers, such as CLARABEL [12], COSMO [14], ECOS [32], GUROBI [13], MOSEK [33], and SCS rely on sparse linear algebra routines. Although sparse linear algebra allows the aforementioned solvers to solve SOCPs regardless of their sparsity structures, it can be quite slow due to the extra overhead of determining where the nonzero elements are before performing floating point operations, which leads to more CPU instructions being issued, more memory accesses, and more cache misses. However, when the sparsity structure of the problem matrices is known beforehand, it is possible to generate a custom solver that implements specialized linear algebra routines tailored to the sparsity structure of the problem. These custom solvers hardcode the exact floating point operations necessary, eliminating the overhead incurred by sparse linear algebra routines, resulting in significantly faster solves.

A *custom solver generator* is a tool that takes the sparsity structure of an optimization problem as an input and outputs a solver (typically in C) optimized for that specific sparsity structure without relying on sparse linear algebra. Two notable custom solver generators are CVXGEN [34] and BSOCP

[35, 36]. CVXGEN, which is academically licensed but not open-source, solves quadratic programs (QPs) rather than SOCPs (i.e. $\mathcal{K}$ in Problem 1 is the non-negative orthant) and BSOCP, which is neither academically licensed nor open source, solves linear objective SOCPs (i.e. $P = 0$ in Problem 1). Both have demonstrated substantial speed improvements over general-purpose solvers and have had a significant impact on aerospace trajectory optimization. For instance, after the development of lossless convexification [37, 38], BSOCP was flight-tested on Masten’s Xombie vehicle to validate G-FOLD, a powered-descent guidance algorithm based on lossless convexification [39]. Later, CVXGEN was used by SpaceX for landing their Falcon 9 boosters [40].

1.1 Contribution

This work makes two key contributions: (1) the development of QOCO, an open-source C-based solver for quadratic objective SOCPs, and (2) the introduction of QOCOGEN, an open-source custom solver generator for quadratic objective SOCPs which generates a custom solver written in C.

We demonstrate that QOCO outperforms most commercial and open-source solvers in terms of both speed and robustness, and QOCO_{custom} (the name of solvers generated by QOCOGEN) is significantly faster than QOCO. Both solvers are easy to use since QOCO and QOCOGEN can be called from CVXPY [41, 42] and CVXPYgen [43] respectively, allowing users to formulate optimization problems in a natural way following from math rather than manually converting the problem to the solver-required standard form. Additionally, QOCO can be called from C/C++, Matlab, and Python, and QOCOGEN can be called from Python.

Both implement a primal-dual interior point method with Mehrotra’s predictor-corrector [44]. However, they both use a variety of numerical enhancements to quickly and robustly solve the linear system that arises in the step direction computation. Specifically, we use an LDL^T factorization along with the Approximate Minimum Degree (AMD) [45, 46] heuristic to permute the coefficient matrix, minimizing fill-in for the factor L . We also apply static and dynamic regularization to the coefficient matrix to ensure that the matrix is invertible and the factorization succeeds. After solving the linear system, we apply iterative refinement to ensure that the original, unregularized system was solved to high accuracy, further enhancing numerical stability and robustness.

QOCO and QOCOGEN are available on GitHub at <https://github.com/qoco-org>.

1.2 Outline

Section 2 introduces the primal-dual interior point method implemented in QOCO and QOCOGEN. Section 3 explains the techniques used to ensure a stable factorization of the KKT matrix, which is essential to compute the step direction. Section 4 discusses the advantages of custom linear algebra over sparse linear algebra, and how QOCOGEN generates custom solvers which exploit the known sparsity structure of problem data. Finally, Section 5 presents extensive numerical results comparing QOCO and QOCO_{custom} to existing solvers.

1.3 Notation

(a, b) refers to the concatenation of $a \in \mathbb{R}^a$ and $b \in \mathbb{R}^b$ into a vector in $\mathbb{R}^{a+b}$

The l -dimensional non-negative orthant is given by $\mathbb{R}_+^l = \{u \in \mathbb{R}^l \mid u_i \geq 0\}$

For a vector u in the l -dimensional non-negative orthant, u_i refers to the i^{th} element of the vector

The q -dimensional second-order cone is given by $\mathcal{Q}^q = \{(u_0, u_1) \in \mathbb{R} \times \mathbb{R}^{q-1} \mid \|u_1\|_2 \leq u_0\}$

For a vector u in the q -dimensional second-order cone, we partition it as $u = (u_0, u_1)$, i.e. $u_0 \in \mathbb{R}$, $u_1 \in \mathbb{R}^{q-1}$ and $\|u_1\|_2 \leq u_0$.

When $x \in \mathcal{C}_1 \times \mathcal{C}_2 \times \cdots \times \mathcal{C}_K$, x_i refers to the i^{th} subvector which is in cone $\mathcal{C}_i$

2 Primal-dual interior point method

In this section, we describe the primal-dual interior point algorithm we implement in QOCO and QOCOGEN, as outlined in Algorithm 1. This algorithm is equivalent to the `coneqp` algorithm outlined in [47] and follows a derivation similar to those presented in [48, Chapter 6], [49, 50].

If strong duality holds for Problem 1, the optimal primal-dual solution (x^*, s^*, y^*, z^*) satisfies the Karush-Kuhn-Tucker (KKT) conditions [51, Chapter 5], which can be written as

$$Px + c + A^\top y + G^\top z = 0 \quad (2a)$$

$$Ax = b \quad (2b)$$

$$Gx + s = h \quad (2c)$$

$$s_i^\top z_i = 0 \text{ for } i = 1, \dots, K \quad (2d)$$

$$(s, z) \in \mathcal{K} \times \mathcal{K}. \quad (2e)$$

The primal-dual interior point method applies a modified Newton's method to Equations (2a) - (2d), and a line search to satisfy Equation (2e). The modifications to Newton's method correct for linearization errors in the Newton step and bias the search directions towards the interior of $\mathcal{K}$. This prevents the iterates from prematurely approaching the boundary of the cone $\mathcal{K}$, enabling longer steps without violating Equation (2e). It also ensures iterates do not converge to spurious solutions that satisfy Equations (2a) - (2d), but not Equation (2e) [50].

2.1 Central path derivation

To understand how the algorithm biases the search directions towards the interior of $\mathcal{K}$, we first introduce the concept of the *central path*.

Problem 1 can be equivalently rewritten as

$$\begin{aligned} & \underset{x}{\text{minimize}} && \frac{1}{2}x^\top Px + c^\top x + \mathcal{I}_{\mathcal{K}}(h - Gx) \\ & \text{subject to} && Ax = b, \end{aligned}$$

where $\mathcal{I}_{\mathcal{K}}$, the indicator function of the cone $\mathcal{K}$, is defined as

$$\mathcal{I}_{\mathcal{K}}(u) = \begin{cases} 0, & \text{for } u \in \mathcal{K} \\ \infty, & \text{otherwise.} \end{cases}$$

We then replace the indicator function, $\mathcal{I}_{\mathcal{K}}(u)$, with a smooth barrier function, $\phi_{\mathcal{K}}(u)$, which approaches ∞ as its argument approaches the boundary of the cone.

We use the barrier function

$$\phi_{\mathcal{K}}(u) = \sum_{i=1}^K \phi_i(u_i),$$

where ϕ_i is the barrier function for $\mathcal{C}_i$. The barrier function for the non-negative orthant and the second-order cone are

$$\phi_i(u) = \begin{cases} -\sum_{j=1}^l \log u_j, & \text{for } \mathcal{C}_i = \mathbb{R}_+^l \\ -(1/2) \log(u_0^2 - u_1^\top u_1), & \text{for } \mathcal{C}_i = \mathcal{Q}^q, \end{cases}$$

where $\log$ is the natural logarithm.

Their gradients are

$$\nabla\phi_i(u) = \begin{cases} (-1/u_1, \dots, -1/u_l), & \text{for } \mathcal{C}_i = \mathbb{R}_+^l \\ -(u^\top Ju)^{-1}Ju, & \text{for } \mathcal{C}_i = \mathcal{Q}^q, \end{cases} \quad (3)$$

where

$$J = \begin{bmatrix} 1 & 0 \\ 0 & -I_{q-1} \end{bmatrix}.$$

After replacing the indicator function with the barrier function, we obtain

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & \frac{1}{2}x^\top Px + c^\top x + \tau \sum_{i=1}^K \phi_i(h_i - G_i x) \\ \text{subject to} \quad & Ax = b, \end{aligned} \quad (4)$$

where $\tau > 0$ is a scalar. Denoting the optimal solution of Problem 4 as $x^*(\tau)$, it can be shown that as $\tau \rightarrow 0$, $x^*(\tau) \rightarrow x^*$.

The KKT condition for Problem 4 are

$$\begin{aligned} Px + c + A^\top y - \tau \sum_{i=1}^K G_i^\top \nabla\phi_i(h_i - G_i x) &= 0 \\ Ax &= b \\ h - Gx &\in \mathcal{K}. \end{aligned}$$

If we define $s = h - Gx$ and $z_i = -\tau \nabla\phi_i(h_i - G_i x)$ for $i = 1, \dots, K$, we can rewrite the KKT conditions as

$$Px + c + A^\top y + G^\top z = 0 \quad (5a)$$

$$Ax = b \quad (5b)$$

$$Gx + s = h \quad (5c)$$

$$z_i = -\tau \nabla\phi_i(s_i) \text{ for } i = 1, \dots, K \quad (5d)$$

$$(s, z) \in \mathcal{K} \times \mathcal{K}, \quad (5e)$$

where the condition $z \in \mathcal{K}$ arises because if $u \in \mathcal{K}$, then $-\nabla\phi(u) \in \mathcal{K}$ [47].

It is desirable to write Equation (5d), in a form where s_i and z_i appear symmetrically. To this end, we define the *Jordan product* [52, 49, 47], a commutative and linear operation, for the non-negative orthant and second-order cone as

$$u \circ v = \begin{cases} (u_1 v_1, \dots, u_l v_l), & \text{for } \mathcal{C}_i = \mathbb{R}_+^l \\ (u^\top v, u_0 v_1 + v_0 u_1), & \text{for } \mathcal{C}_i = \mathcal{Q}^q, \end{cases}$$

and the Jordan product for $\mathcal{K}$ as

$$u \circ v = (u_1 \circ v_1, u_2 \circ v_2, \dots, u_K \circ v_K), \quad (6)$$

where $u_i, v_i \in \mathcal{C}_i$.

The identity element $\mathbf{e}_i$ for cone $\mathcal{C}_i$ is defined as

$$\mathbf{e}_i = \begin{cases} (1, 1, \dots, 1), & \text{for } \mathcal{C}_i = \mathbb{R}_+^l \\ (1, 0, \dots, 0), & \text{for } \mathcal{C}_i = \mathcal{Q}^q, \end{cases}$$

and the identity element for $\mathcal{K}$ is

$$\mathbf{e} = (\mathbf{e}_1, \dots, \mathbf{e}_K). \quad (7)$$

Taking the Jordan product with s_i on both sides of Equation (5d) and substituting Equation (3), we obtain

$$Px + c + A^\top y + G^\top z = 0 \quad (8a)$$

$$Ax = b \quad (8b)$$

$$Gx + s = h \quad (8c)$$

$$s_i \circ z_i = \tau \mathbf{e}_i \text{ for } i = 1, \dots, K \quad (8d)$$

$$(s, z) \in \mathcal{K} \times \mathcal{K}. \quad (8e)$$

We then rewrite Equation (8d) by stacking s_i and z_i and using Equations (6) and (7) to obtain

$$Px + c + A^\top y + G^\top z = 0 \quad (9a)$$

$$Ax = b \quad (9b)$$

$$Gx + s = h \quad (9c)$$

$$s \circ z = \tau \mathbf{e} \quad (9d)$$

$$(s, z) \in \mathcal{K} \times \mathcal{K}. \quad (9e)$$

The *central path* is defined as the trajectory of points parameterized by $\tau > 0$, satisfying Equations (9a) - (9e). We denote the central path with the tuple $(x^*(\tau), s^*(\tau), y^*(\tau), z^*(\tau))$. From the above analysis, we see that the central path equations given by Equations (9a) - (9e) are equivalent to the optimality conditions for the barrier formulation of Problem 1 given by Problem 4. It can be shown that as $\tau \rightarrow 0$, $(x^*(\tau), s^*(\tau), y^*(\tau), z^*(\tau)) \rightarrow (x^*, s^*, y^*, z^*)$.

The primal-dual interior-point method applies Newton's method to move towards the central path rather than directly towards points satisfying Equation (2d). Since the central path lies within the cone, maintaining proximity to it allows the algorithm to take longer steps without violating Equation (2e) [50].

2.2 Nesterov-Todd scaling

If Newton's method were applied directly to Equations (5a) - (5d), the coefficient matrix of the resulting linear system would lack symmetry (see Appendix A). As a result, it would be necessary to store the entire matrix, rather than only the upper or lower triangular portion. Additionally, solving this system would require matrix factorizations for non-symmetric matrices, which are generally more computationally expensive than those for symmetric matrices.

To derive a symmetric linear system, we introduce the following change of variables

$$\tilde{s} = W^{-\top} s, \quad \tilde{z} = W z,$$

where we choose W such that the above transformation preserves cone membership and leaves the central path equations unchanged

$$s \in \mathcal{K} \iff \tilde{s} \in \mathcal{K}, \quad z \in \mathcal{K} \iff \tilde{z} \in \mathcal{K}, \quad s \circ z = \tau \mathbf{e} \iff \tilde{s} \circ \tilde{z} = \tau \mathbf{e}. \quad (10)$$

Using this transformation, the central path equations can be equivalently expressed as

$$Px + c + A^\top y + G^\top z = 0 \quad (11a)$$

$$Ax = b \quad (11b)$$

$$Gx + s = h \quad (11c)$$

$$(W^{-\top} s) \circ (Wz) = \tau \mathbf{e} \quad (11d)$$

$$(s, z) \in \mathcal{K} \times \mathcal{K}. \quad (11e)$$

There are many matrices, W , which satisfy Equation (10). In particular, we use the *Nesterov-Todd scaling* matrix [53, 54]. This scaling matrix is determined based on the current iterates s_k and z_k and the unique point w that satisfies

$$\nabla^2 \phi_{\mathcal{K}}(w) s_k = z_k, \quad (12)$$

where $\nabla^2 \phi_{\mathcal{K}}(w)$ is the Hessian of the barrier function evaluated at w . Since the barrier function is strictly convex, $\nabla^2 \phi_{\mathcal{K}}(w)$ is positive definite.

From this, we compute W_k as

$$\nabla^2 \phi_{\mathcal{K}}(w)^{-1} = W_k^\top W_k. \quad (13)$$

A key property that follows from Equations (12) and (13) is

$$W_k^{-\top} s_k = W_k z_k. \quad (14)$$

This property allows us to define

$$\lambda_k = W_k^{-\top} s_k = W_k z_k. \quad (15)$$

For more details on how to compute the scaling point w and the scaling matrix W_k , see [47].

2.3 Computing search directions

Given the current iterate (x_k, s_k, y_k, z_k) we define the *duality measure* as

$$\mu_k = s_k^\top z_k / m, \quad (16)$$

which represents the average violation of the complementary slackness condition given by Equation (2d). To compute a search direction for updating the current iterate, we apply Mehrotra's predictor-corrector method [44]. This scheme computes a Newton step with three key objectives: reducing the duality measure, maintaining proximity to the central path, and correcting for the linearization error introduced during this step.

Applying Newton's method to Equation (11) first requires a linearization of the central path equations around the current iterate (x_k, s_k, y_k, z_k) . This is done by substituting (x, s, y, z) with $(x_k + \Delta x, s_k +$

$\Delta s, y_k + \Delta y, z_k + \Delta z$) and ignoring the higher order term $(W^{-\top} \Delta s) \circ (W \Delta z)$. This results in a linear system of the form

$$P \Delta x + A^\top \Delta y + G^\top \Delta z = -r_x \quad (17a)$$

$$A \Delta x = -r_y \quad (17b)$$

$$G \Delta x + \Delta s = -r_z \quad (17c)$$

$$(W_k^{-\top} s_k) \circ (W_k \Delta z) + (W_k^{-\top} \Delta s) \circ (W_k z_k) = -r_s. \quad (17d)$$

Here, (r_x, r_y, r_z, r_s) are residual vectors that will be defined later.

Using Equation (15), we can rewrite the system as

$$P \Delta x + A^\top \Delta y + G^\top \Delta z = -r_x \quad (18a)$$

$$A \Delta x = -r_y \quad (18b)$$

$$G \Delta x + \Delta s = -r_z \quad (18c)$$

$$\lambda_k \circ (W_k \Delta z + W_k^{-\top} \Delta s) = -r_s. \quad (18d)$$

To obtain a symmetric system, we can eliminate Δs and rewrite the equations in matrix form as

$$\begin{bmatrix} P & A^\top & G^\top \\ A & 0 & 0 \\ G & 0 & -W_k^\top W_k \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta z \end{bmatrix} = \begin{bmatrix} -r_x \\ -r_y \\ -r_z + W_k^\top (\lambda_k \setminus r_s) \end{bmatrix} \quad (19a)$$

$$\Delta s = -r_z - G \Delta x, \quad (19b)$$

where the operator $\setminus$ represents the inverse of the Jordan product $\circ$, meaning that $x \setminus (x \circ y) = y$.

To account for the higher order term we ignored when forming Equation (17), we apply Mehrotra's predictor-corrector [44]. This method consists of a predictor step, where a search direction is computed by taking a Newton step on Equation (11) with $\tau = 0$, followed by a combined step. The combined step incorporates centering and correction terms that will help keep the next iterate close to the central path while compensating for the linearization error incurred by ignoring the $(W^{-\top} \Delta s) \circ (W \Delta z)$ term when linearizing Equation (11). Note that the predictor step is used to estimate this linearization error. Figure 1 illustrates Mehrotra's predictor-corrector.

To compute the predictor step, also called the *affine scaling direction*, $(\Delta x_a, \Delta s_a, \Delta y_a, \Delta z_a)$, we solve Equation (19) with the residual vectors

$$r_x = P x_k + c + A^\top y_k + G^\top z_k \quad (20a)$$

$$r_y = A x_k - b \quad (20b)$$

$$r_z = G x_k + s_k - h \quad (20c)$$

$$r_s = \lambda_k \circ \lambda_k. \quad (20d)$$

Since taking a full step along the affine scaling direction can violate the constraint $(s, z) \in \mathcal{K} \times \mathcal{K}$, we compute the centering parameter $\sigma \in [0, 1]$ as

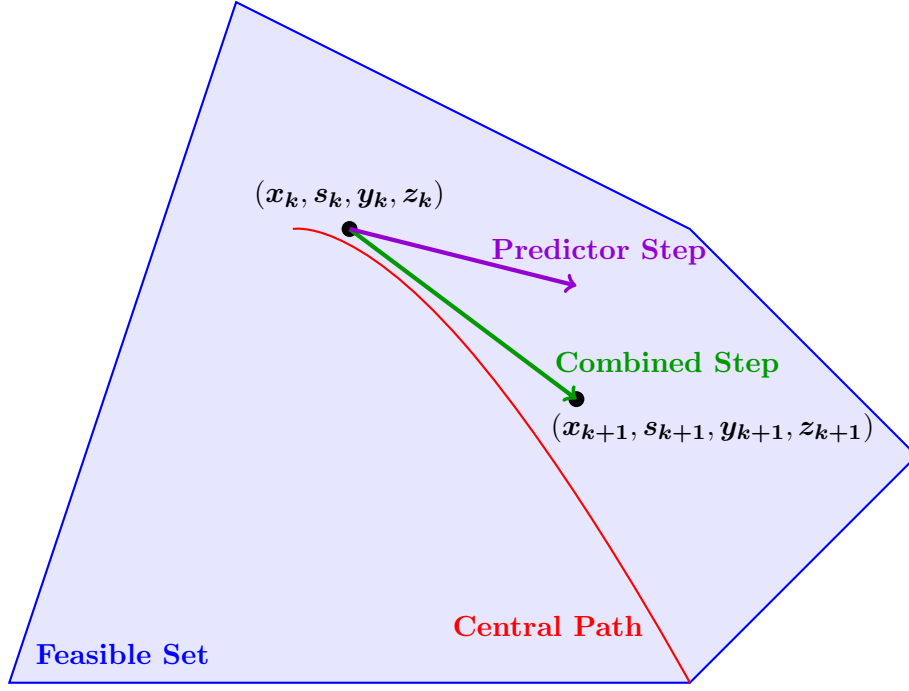


Figure 1: Mehrotra's predictor-corrector.

$$\begin{aligned}\alpha &= \sup\{\alpha \in [0, 1] \mid (s_k, z_k) + \alpha(\Delta s_a, \Delta z_a) \in \mathcal{K} \times \mathcal{K}\} \\ \rho &= \frac{(s_k + \alpha \Delta s_a)^\top (z_k + \alpha \Delta z_a)}{s_k^\top z_k} \\ \sigma &= \max\{0, \min\{1, \rho\}^3\}.\end{aligned}$$

The centering parameter balances the need to reduce the duality measure while maintaining proximity to the central path, allowing for a longer step in the next iteration [50].

Finally, we compute the combined direction, which includes both predictor and corrector information. This direction, $(\Delta x, \Delta s, \Delta y, \Delta z)$ is obtained by solving Equation (19) with the residual vectors

$$\begin{aligned}r_x &= Px_k + c + A^\top y_k + G^\top z_k \\ r_y &= Ax_k - b \\ r_z &= Gx_k + s_k - h \\ r_s &= \lambda_k \circ \lambda_k + (W_k^{-\top} \Delta s_a) \circ (W_k \Delta z_a) - \sigma \mu_k e\end{aligned}$$

This direction moves the next iterate closer to $(x^*(\sigma \mu_k), s^*(\sigma \mu_k), y^*(\sigma \mu_k), z^*(\sigma \mu_k))$, a point on the central path where the duality measure is reduced by a factor of σ . Note that the residual vector r_s has two additional terms when compared to Equation (20d): $(W_k^{-\top} \Delta s_a) \circ (W_k \Delta z_a)$ and $-\sigma \mu_k e$. The former is to correct for linearization error in Newton's method, and the latter is to maintain proximity to the central path and arises when applying Newton's method to Equation (9d) with $\tau = \sigma \mu_k$.

We then compute the next iterate by performing a line search to ensure $(s_{k+1}, z_{k+1}) \in \mathcal{K} \times \mathcal{K}$. Specifically, we update the iterate as

$$(x_{k+1}, s_{k+1}, y_{k+1}, z_{k+1}) = (x_k, s_k, y_k, z_k) + \alpha(\Delta x, \Delta s, \Delta y, \Delta z)$$

where

$$\alpha = \sup \left\{ \alpha \in [0, 1] \mid (s_k, z_k) + \frac{\alpha}{0.99}(\Delta s, \Delta z) \in \mathcal{K} \times \mathcal{K} \right\}$$

The factor of 0.99 ensures that (s_{k+1}, z_{k+1}) remains strictly in the interior of $\mathcal{K} \times \mathcal{K}$.

2.4 Initialization

To initialize the primal-dual interior point method, we follow the approach outlined in [47]. The initialization procedure consists of two steps. First, we find (x, s, y, z) that satisfy Equations (9a) - (9c). If s and z do not satisfy Equation (9e), a correction is applied to ensure they are in $\mathcal{K}$.

We begin by solving the optimization problem

$$\begin{aligned} & \underset{x}{\text{minimize}} && \frac{1}{2}x^\top Px + c^\top x + \|Gx - h\|_2^2 \\ & \text{subject to} && Ax = b. \end{aligned}$$

Since this is an equality-constrained quadratic program, the KKT conditions correspond to the linear system

$$\begin{bmatrix} P & A^\top & G^\top \\ A & 0 & 0 \\ G & 0 & -I \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} -c \\ b \\ h \end{bmatrix}. \quad (23)$$

We then set $s = -z$. This results in the tuple (x, s, y, z) satisfying Equations (9a) - (9c), but s and z may not necessarily satisfy Equation (9e).

We then initialize x_0 and y_0 as $x_0 = x$, $y_0 = y$, and initialize s_0 as

$$s_0 = \begin{cases} s, & \text{if } s \in \mathcal{K} \\ s + (1 + \alpha_s)\mathbf{e} & \text{otherwise,} \end{cases}$$

where $\alpha_s = \inf\{\alpha \mid s + \alpha\mathbf{e} \in \mathcal{K}\}$ and $\mathbf{e}$ is defined in Equation (7). We initialize z_0 as

$$z_0 = \begin{cases} z, & \text{if } z \in \mathcal{K} \\ z + (1 + \alpha_z)\mathbf{e}, & \text{otherwise} \end{cases}$$

where $\alpha_z = \inf\{\alpha \mid z + \alpha\mathbf{e} \in \mathcal{K}\}$. Note that α_s and α_z are the minimum scalings of $\mathbf{e}$ that need to be added to s and z respectively to move them to the boundary of $\mathcal{K}$, but we scale $\mathbf{e}$ by $(1 + \alpha_s)$ and $(1 + \alpha_z)$ to ensure that s_0 and z_0 are strictly in the interior of $\mathcal{K}$.

Algorithm 1 Primal-Dual Interior Point Method

Ensure: Optimal solution (x^*, s^*, y^*, z^*)

```
1: Initialization:
2:   Solve KKT system (23) for  $x, y, z$ 
3:    $x_0 \leftarrow x, y_0 \leftarrow y$ 
4:    $s \leftarrow -z$ 
5:   if  $s \in \mathcal{K}$  then
6:      $s_0 \leftarrow s$ 
7:   else
8:      $\alpha_s \leftarrow \inf\{\alpha \mid s + \alpha e \in \mathcal{K}\}$ 
9:      $s_0 \leftarrow s + (1 + \alpha_s)e$ 
10:  end if
11:  if  $z \in \mathcal{K}$  then
12:     $z_0 \leftarrow z$ 
13:  else
14:     $\alpha_z \leftarrow \inf\{\alpha \mid z + \alpha e \in \mathcal{K}\}$ 
15:     $s_0 \leftarrow z + (1 + \alpha_z)e$ 
16:  end if
17:  repeat
18:    Compute duality measure:
19:     $\mu_k \leftarrow s_k^\top z_k / m$ 
20:    Compute Nesterov-Todd scaling:
21:    Construct  $W_k$ 
22:    Calculate  $\lambda_k = W_k^{-\top} s_k = W_k z_k$ 
23:    Compute affine direction  $(\Delta x_a, \Delta s_a, \Delta y_a, \Delta z_a)$ :
24:    Solve KKT system (19) with residuals (20)
25:    Calculate centering parameter:
26:     $\alpha \leftarrow \sup\{\alpha \in [0, 1] \mid (s_k, z_k) + \alpha(\Delta s_a, \Delta z_a) \in \mathcal{K}\}$ 
27:     $\rho \leftarrow \frac{(s_k + \alpha \Delta s_a)^\top (z_k + \alpha \Delta z_a)}{s_k^\top z_k}$ 
28:     $\sigma \leftarrow \max\{0, \min\{1, \rho\}^3\}$ 
29:    Compute combined direction  $(\Delta x, \Delta s, \Delta y, \Delta z)$ :
30:    Solve (19) with residuals (22)
31:    Compute step-size:
32:     $\alpha \leftarrow \sup\{\alpha \in [0, 1] \mid (s_k, z_k) + \frac{\alpha}{0.99}(\Delta s, \Delta z) \in \mathcal{K}\}$ 
33:    Update iterates:
34:     $x_{k+1} \leftarrow x_k + \alpha \Delta x$ 
35:     $s_{k+1} \leftarrow s_k + \alpha \Delta s$ 
36:     $y_{k+1} \leftarrow y_k + \alpha \Delta y$ 
37:     $z_{k+1} \leftarrow z_k + \alpha \Delta z$ 
38:  until Stopping criteria (29) satisfied.
```

3 Algorithm Implementation

This section describes the implementation of the primal-dual interior point method in QOCO and QOCOGEN. We focus on three key aspects: the matrix factorization used to solve the system in (19), numerical techniques for ensuring a robust factorization and solution of (19), and the stopping criteria. These aspects apply to both QOCO and QOCOGEN, but while both use the same underlying matrix factorization, their implementations are different. These differences will be discussed in Section 4.1.

3.1 Linear System Solve

The coefficient matrix in (19), which we will refer to as the Karush-Kuhn-Tucker (KKT) matrix is given as

$$K = \begin{bmatrix} P & A^\top & G^\top \\ A & 0 & 0 \\ G & 0 & -W_k^\top W_k \end{bmatrix}. \quad (24)$$

Algorithm 1 requires solving the linear system $K\xi = r$ for two different residual vectors r . Since K remains constant for both solves, we factorize it once and then apply two backsolves.

The KKT matrix (24) is symmetric but indefinite, meaning it has both positive and negative eigenvalues. A common approach for factoring such matrices is the Bunch-Parlett factorization [55], which factors K as

$$\Pi K \Pi^\top = LDL^\top, \quad (25)$$

where Π is a permutation matrix, L is a lower triangular matrix, and D is a block diagonal matrix with 1×1 and 2×2 blocks. However, Π must be selected during runtime with knowledge of the data in K , as the factorization may not exist for all permutations. Additionally, K may not be invertible if, for example, A is not full row rank.

Static Regularization To ensure a numerically stable factorization that exists for all permutations Π (allowing us to select Π based solely on the sparsity pattern of K), we apply *static regularization* [56, 12, 34] to the KKT matrix

$$\hat{K} = \left[\begin{array}{c|cc} P & A^\top & G^\top \\ \hline A & 0 & 0 \\ G & 0 & -W_k^\top W_k \end{array} \right] + \left[\begin{array}{c|cc} \epsilon_s I & 0 & 0 \\ \hline 0 & -\epsilon_s I & 0 \\ 0 & 0 & -\epsilon_s I \end{array} \right]. \quad (26)$$

In QOCO and QOCOGEN, we use $\epsilon_s = 10^{-8}$.

Now $\hat{K}$ is a *quasidefinite* matrix, which is a special case of a symmetric indefinite matrix where the $(1, 1)$ block is positive definite and the $(2, 2)$ block is negative definite [57].

For such matrices, the $LDL^\top$ factorization

$$\Pi \hat{K} \Pi^\top = LDL^\top, \quad (27)$$

exists for any permutation Π , and D will be a diagonal matrix with known signs for its diagonal elements [57].

A judicious choice of Π can minimize the *fill-in* for the factor L . Fill-in is the introduction of non-zero elements in positions of a matrix where they previously did not exist. Excessive fill-in can lead to several negative consequences, including increased storage requirements (since more non-zero elements must be stored in memory) and increased computational cost (due to more floating-point operations). Finding an optimal Π to minimize fill-in is NP-complete [58], but heuristic methods such as the Approximate Minimum Degree (AMD) ordering can effectively compute near-optimal permutations [45]. In QOCO and QOCOGEN, we use Tim Davis' implementation of the AMD ordering [46] to determine Π .

After computing Π , the $LDL^\top$ factorization consists of two phases: a symbolic and numeric factorization. The symbolic factorization determines the elimination tree, a data structure which encodes dependencies between elements in the matrix during the factorization process, and the sparsity pattern of the factor L . It only requires the sparsity pattern of the regularized KKT matrix $\hat{K}$, which remains unchanged throughout Algorithm 1. Therefore, the symbolic factorization needs to be computed only once. The numeric factorization calculates the numerical values of L and D , which depend on the values of the nonzero elements of $\hat{K}$. Since these values change at each iteration of Algorithm 1, the numeric factorization must be computed at every step.

To compute the $LDL^\top$ factorization, we use a modified version of QDLDL [56] with *dynamic regularization* for QOCO, and a custom $LDL^\top$ routine for QOCOGEN which we will discuss in Section 4.1.

Dynamic Regularization Although the $LDL^\top$ factorization must theoretically exist for $\hat{K}$, due to the nature of floating-point arithmetic it is possible for diagonal elements D_{ii} to be rounded to zero or be very close to zero leading to divide-by-zero errors and the factorization failing. To alleviate this and factor $\hat{K}$ in a stable manner, we apply *dynamic regularization* as given by Algorithm 2, which bounds the magnitude of D_{ii} away from zero, by some $\epsilon_d > 0$, and corrects for its sign if necessary [12, 34, 32]. In QOCO and QOCOGEN, we use $\epsilon_d = 10^{-8}$.

Algorithm 2 Dynamic Regularization

```

1: Given:  $\epsilon_d > 0$ 
2: if  $D_{ii}$  should be positive then
3:   if  $D_{ii} < 0$  then
4:      $D_{ii} = \epsilon_d$ 
5:   else
6:      $D_{ii} = D_{ii} + \epsilon_d$ 
7:   end if
8: else if  $D_{ii}$  should be negative then
9:   if  $D_{ii} > 0$  then
10:     $D_{ii} = -\epsilon_d$ 
11:   else
12:     $D_{ii} = D_{ii} - \epsilon_d$ 
13:   end if
14: end if

```

Iterative Refinement After static and dynamic regularization we end up solving the linear system $\hat{K}\xi = r$ rather than system $K\xi = r$. To get a solution to the latter system, we apply *iterative refinement* [59, Section 2.5.1]. This process corrects the solution by iteratively reducing the residual error, $\|r - K\xi\|$, ensuring higher numerical accuracy without requiring an additional factorization. It involves iteratively solving the system

$$\hat{K}\Delta\xi^k = r - K\xi^k, \quad (28)$$

and updating $\xi^{k+1} = \xi^k + \Delta\xi^k$, where ξ^0 solves the first linear system (i.e. $\hat{K}\xi^0 = r$). Note that since we have already computed the factorization of $\hat{K}$, one iteration of Equation (28) only requires backsolves and no matrix factorization.

3.2 Stopping Criterion

Algorithm 1 terminates when three conditions are met: primal feasibility (29a), stationarity (29b), and complementary slackness (29c). We use both absolute and relative tolerances to ensure robustness across different problem scales. Our stopping criteria is met if the following conditions hold:

$$\left\| \begin{bmatrix} Ax_k - b \\ Gx_k + s_k - h \end{bmatrix} \right\|_\infty \leq \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \max \{ \|Ax_k\|_\infty, \|b\|_\infty, \|Gx_k\|_\infty, \|h\|_\infty, \|s_k\|_\infty \} \quad (29a)$$

$$\|Px_k + A^\top y_k + G^\top z_k + c\|_\infty \leq \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \max \{ \|Px_k\|_\infty, \|A^\top y_k\|_\infty, \|G^\top z_k\|_\infty, \|c\|_\infty \} \quad (29b)$$

$$|s_k^\top z_k| \leq \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \max \{ 1, |p_k|, |d_k| \}, \quad (29c)$$

where the primal and dual objectives are

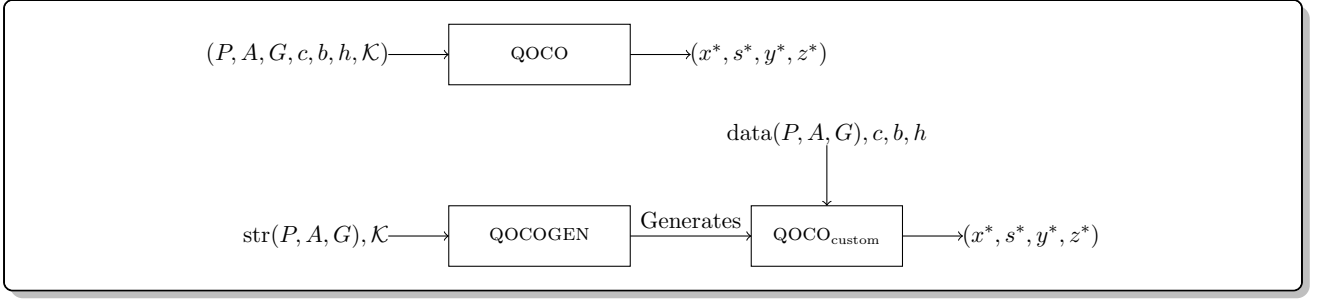


Figure 2: Usage of QOCO, QOCOGEN, and QOCO_{custom}.

$$p_k := \frac{1}{2}x_k^\top Px_k + c^\top x_k, \quad d_k := -\frac{1}{2}x_k^\top Px_k - b^\top y_k - h^\top z_k.$$

4 Sparsity Exploiting Custom Solver

This section discusses QOCOGEN, which generates an implementation of Algorithm 1 called QOCO_{custom}. Unlike generic solvers which use sparse linear algebra, QOCO_{custom} uses a custom $LDL^\top$ factorization and other tailored linear algebra routines, which exploit the known sparsity structure of problem data, to solve Problem 1 significantly faster.

While QOCO can solve instances of Problem 1 with any sparsity pattern for P, A , and G and cone $\mathcal{K}$, the use of custom linear algebra in QOCO_{custom} restricts it to instances with the same sparsity structure for P, A , and G and optimizes over the same cone $\mathcal{K}$. This makes QOCO_{custom} useful for applications that repeatedly solve optimization problems with identical sparsity structures, such as sequential convex programming [8].

Figure 2 illustrates the usage of QOCO, QOCOGEN, and QOCO_{custom}. In the figure, the matrices (P, A, G) contain two pieces of information: their sparsity patterns $\text{str}(P, A, G)$ (the location of nonzero elements) and the values of the nonzero elements $\text{data}(P, A, G)$. We see that QOCO takes in (P, A, G) , which contains both the sparsity patterns and nonzero elements for the matrices, and the rest of the problem data and returns the optimal solution (x^*, s^*, y^*, z^*) . In contrast, QOCOGEN only takes in the sparsity patterns of (P, A, G) and cone $\mathcal{K}$, and then generates QOCO_{custom}. This instance of QOCO_{custom} can then solve optimization problems where the sparsity patterns of (P, A, G) do not change, but the nonzero values for (P, A, G) and the vector data (c, b, h) can change.

Although QOCO avoids dynamic memory allocation during the solution process, it still relies on `setup` and `cleanup` functions for dynamic memory allocation and deallocation before and after solving the problem. In contrast, QOCO_{custom} exclusively uses static memory allocation, making it well-suited for resource-constrained embedded systems, where dynamic memory allocation can lead to non-deterministic behavior and memory fragmentation.

4.1 Custom Linear Algebra

A key difference between QOCO_{custom} and QOCO is that QOCO_{custom} uses custom linear algebra routines rather than sparse linear algebra. The main motivation for custom routines is to speed up the $LDL^\top$ factorization of the regularized KKT matrix, which is the most computationally expensive step of Algorithm 1. Typically, the KKT matrix is factored using a sparse linear algebra routine that stores the matrix in either *compressed sparse column* (CSC) or *compressed sparse row* (CSR) format. However, these sparse routines involve more than just the floating-point operations required for factorization. They also incur additional overhead to locate nonzero elements and their positions in the matrix. For example, in a CSC matrix, accessing data first requires indexing into the column pointer and row index arrays. This results in more CPU instructions, increased memory accesses, and a higher likelihood of cache misses. We provide empirical evidence supporting these claims in Appendix B.

```

1  # Adata stores the nonzeros of A in column-major format, i.e.
2  # Adata[0] = a00
3  # Adata[1] = a20
4  # Adata[2] = a02
5  def custom_matvec(y, Adata, x):
6      y[0] = Adata[0]*x[0] + Adata[2]*x[2]
7      y[1] = 0
8      y[2] = Adata[1]*x[0]

```

Listing 1: Custom matrix-vector multiplication.

When the sparsity structure of the KKT matrix is known *a priori* it is possible to write a custom $LDL^\top$ factorization routine which only contains code to perform the necessary floating-point operations and data accesses, eliminating the additional overhead that typical sparse linear algebra routines have. This makes the custom $LDL^\top$ factorization significantly faster than a sparse $LDL^\top$ factorization. We also note that since the AMD ordering we employ to compute the permutation matrix Π only depends on the sparsity pattern of the KKT matrix, we can determine Π at the time of code generation, so the generated code does not include the code to compute the AMD ordering.

A concrete example of a custom linear algebra routine for matrix-vector multiplication is given by Listing 1, where the sparsity pattern of A is fixed and defined in Equation (30). Note that the custom routine only includes the necessary floating-point operations. Although Listing 1 depicts Python code, the custom linear algebra routines in $\text{QOCO}_{\text{custom}}$ are in C.

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix} = \underbrace{\begin{bmatrix} a_{00} & 0 & a_{02} \\ 0 & 0 & 0 \\ a_{20} & 0 & 0 \end{bmatrix}}_A \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} \quad (30)$$

A drawback of these custom routines is that the amount of code generated increases with the problem size. As a result, the time to generate and compile $\text{QOCO}_{\text{custom}}$ can become burdensome as the problem sizes get very large.

4.2 Code Generation

QOCOGEN is written in Python and example code for generating $\text{QOCO}_{\text{custom}}$ for a problem family defined by Equation (31) is given by Listing 2. The generated solver $\text{QOCO}_{\text{custom}}$ is customized to the sparsity pattern of the matrices P, A, G which are passed to the function `qocogen.generate_solver()` as sparse `scipy` matrices.

$$\begin{aligned} & \underset{x}{\text{minimize}} && x_1^2 + x_2^2 + x_3^2 + x_4 \\ & \text{subject to} && x_1 + x_2 = 1 \\ & && x_2 + x_3 = 1 \\ & && x_1 \geq 0 \\ & && \sqrt{x_3^2 + x_4^2} \leq x_2 \end{aligned} \quad (31)$$

The file tree for $\text{QOCO}_{\text{custom}}$ is given by Figure 3. The file `qoco_custom.c` contains the `qoco_custom_solve()` function which implements Algorithm 1, `ldl.c` contains the custom $LDL^\top$ factorization of the regularized KKT matrix, `CMakeLists.txt` is the CMake configuration file which automates the build process [60], `runtest.c` gives an example of calling $\text{QOCO}_{\text{custom}}$ with default data and times the resulting solve, and the remaining files contain various helper functions needed to implement Algorithm 1.

Listing 3 provides an example of calling $\text{QOCO}_{\text{custom}}$ from C, updating the problem data, and solving it again.

```

1  import qocogen
2  import numpy as np
3  from scipy import sparse
4
5  # Define problem data
6  P = sparse.diags([2, 2, 2, 0], 0).tocsc()
7
8  c = np.array([0, 0, 0, 1])
9  G = -sparse.identity(4).tocsc()
10 h = np.zeros(4)
11 A = sparse.csc_matrix([[1, 1, 0, 0], [0, 1, 1, 0]])
12 b = np.array([1, 1])
13
14 l = 1
15 n = 4
16 m = 4
17 p = 2
18 nsoc = 1
19 q = np.array([3])
20
21 # Generate custom solver.
22 qocogen.generate_solver(n, m, p, P, c, A, b, G, h, l, nsoc, q)

```

Listing 2: Generating a custom solver with QOCOGEN.

```

1  #include "qoco_custom.h"
2
3  int main() {
4      // Instantiate workspace.
5      Workspace work;
6
7      // Set default settings, but settings can be modified with
8      // work.settings.setting_name = ...
9      set_default_settings(&work);
10
11     // Loads the default data for the problem
12     // (i.e. the data passed to the qocogen.generate() call).
13     load_data(&work);
14
15     // Solve original problem.
16     qoco_custom_solve(&work);
17     printf("\nobj: %f", work.sol.obj);
18
19     // Can modify non-zero elements of P,A,G and c, b, h.
20     update_P(&work, Pnew);
21     update_A(&work, Anew);
22     update_G(&work, Gnew);
23     update_c(&work, cnew);
24     update_b(&work, bnew);
25     update_h(&work, hnew);
26
27     // Solve updated problem.
28     qoco_custom_solve(&work);
29     printf("\nobj: %f", work.sol.obj);
30 }

```

Listing 3: Calling `qococustom` from C/C++.



Figure 3: QOCO_{custom} file structure.

5 Numerical Results

We benchmarked QOCO and QOCO_{custom} against several conic interior-point solvers: the open-source solvers ECOS [32] and CLARABEL [12], the commercial solvers GUROBI [13] and MOSEK [33], and against the academically licensed custom solver generator CVXGEN which generates a primal-dual interior point method for quadratic programs. We did not test against the custom solver generator BSOCP since it is not academically licensed. For all solvers, we used their default settings but set the tolerances $\epsilon_{abs} = \epsilon_{rel} = 10^{-7}$, and disabled the presolve features of GUROBI and MOSEK to ensure all solvers are given the same problem formulation.

We evaluated our solvers on benchmark problems spanning multiple fields and a range of problem sizes. In our experiments, we define problem size as the total number of nonzero elements in A , G , and in the upper half of P . For all benchmark problems, we tested QOCO, but only tested QOCO_{custom} for the smaller problems, because code generation and compile times become prohibitively long for larger problem instances. We also evaluated QOCO on the difficult Maros–Mészáros problems, a repository of 138 challenging quadratic programs (QPs) [61]. To evaluate the solvers’ performance, we used performance profiles [62] and the shifted geometric mean, both of which are frequently used to assess solver performance [12, 32, 56, 14, 63].

All numerical results were generated on a desktop computer with an AMD Ryzen 9 7950X3D processor running at 4.2 GHz and with 128 GB of RAM. Our benchmarks are written in Python and we used CVXPY [41, 42] to interface with the solvers. Our benchmarks are publicly available [64].

We provide detailed numeric results in Appendix D, which includes iteration counts and solve times for each solver on all problems we ran. Because Gurobi does not report iteration counts via the CVXPY interface for SOCPs, these values are omitted from our results.

Shifted Geometric Mean We use the normalized shifted geometric mean to assign a scalar value to the performance of a solver on a test set of problems. The shifted geometric mean of N runtimes for solver s is computed as

$$g_s = \left[\prod_{p=1}^N (t_{s,p} + k) \right]^{1/N} - k,$$

where $t_{s,p}$ is the runtime of solver s on problem p and $k = 1$ is the shift. If the solver does not find an optimal solution, then the runtime is set to the maximum allowable runtime (10 seconds for benchmark problems and 1200 seconds for the Maros–Mészáros problems).

We then define the normalized shifted geometric mean for solver s as

$$r_s = g_s / \min_s g_s,$$

so the solver with the lowest shifted geometric mean will have a normalized shifted geometric mean of 1.00.

Performance Profiles We also plot the relative and absolute performance profiles to compare solvers. We first define the relative performance ratio of solver s on problem p as

$$u_{s,p} = t_{s,p} / \min_s t_{s,p}.$$

The relative performance profile then plots $f_s^r(\tau)$ where

$$f_s^r(\tau) = \frac{1}{N} \sum_{p=1}^N \mathcal{I}_{\leq \tau}(u_{s,p}),$$

and $\mathcal{I}_{\leq \tau}(z) = 1$ if $z \leq \tau$ and $\mathcal{I}_{\leq \tau}(z) = 0$ otherwise. If the relative performance profile passes through the point (x, y) for solver s , this means that solver s solves a fraction y of the problems within a factor of x of the fastest solver.

The absolute performance profile plots $f_s^a(\tau)$ where

$$f_s^a(\tau) = \frac{1}{N} \sum_{p=1}^N \mathcal{I}_{\leq \tau}(t_{s,p}).$$

If the absolute performance profile passes through the point (x, y) for solver s , this means that solver s solves a fraction y of the problems within x time. For both relative and absolute profiles, the profile of the highest-performing solver will lie above the profiles of the other solvers.

5.1 Benchmark Problems

We solved optimization problems from five problem classes: robust Kalman filtering, group lasso regression [65], the losslessly convexified powered-descent guidance problem [37], Markowitz portfolio optimization [3], and the oscillating masses control problem [66]. The first three of these are SOCPs and the last two are QPs. For each of the five classes, we considered 10 different problem sizes, generating 20 unique problem instances per size, for a total of 1000 distinct optimization problems. Details on the problems we solved can be found in Appendix C. For these problems, we plotted runtime as a function of problem size in addition to reporting performance profiles and shifted geometric means.

For each solver, we solved each optimization problem 100 times and recorded the minimum runtime, which includes both setup and solve time. Since the computer runs various background processes that can artificially inflate execution time, we conducted 100 runs and reported the minimum execution time, thereby providing a more accurate measure of the solver’s performance.

Although the oscillating masses and portfolio optimization problems are QPs and could be solved with CVXGEN, we limited testing to the two smaller oscillating masses problems. For larger instances, CVXGEN was unable to generate code due to the size of the problems. For the portfolio optimization problems, we opted not to test CVXGEN due to the sparsity pattern of the factor matrix F in Problem 34. To specify the sparsity pattern of F in CVXGEN, we must manually provide the locations of nonzero elements into the web interface, which would have been prohibitively expensive, since F has hundreds of nonzero elements. In contrast, to generate code with QOCOGEN, we pass in problem data as sparse `scipy` matrices rather than manually specifying sparsity patterns. We could have specified F as a

dense matrix and only populated the nonzero elements for CVXGEN, this would severely hinder its performance and result in an unfair comparison.

When computing normalized shifted geometric mean, relative performance profiles, and absolute performance profiles, all solvers must be tested on the same set of problems. Since we test QOCO_{custom} on only half of the problems we provide two versions of the aforementioned metrics. The first version, given by Figure 5, includes CLARABEL, ECOS, GUROBI, MOSEK, and QOCO evaluated on all problems. The second version, given by Figure 6, also includes QOCO_{custom}, but the solvers are evaluated on the half of the benchmark problems which QOCO_{custom} is tested on.

Figures 4, 5, and 6 show that QOCO is the fastest generic solver on our benchmark problems. Additionally, QOCO_{custom} is the fastest solver by a significant margin and is as performant as CVXGEN on the 40 smallest oscillating mass problems. This latter result is unsurprising, as QOCO_{custom} uses an identical algorithm to CVXGEN when solving QPs and both utilize custom linear algebra routines.

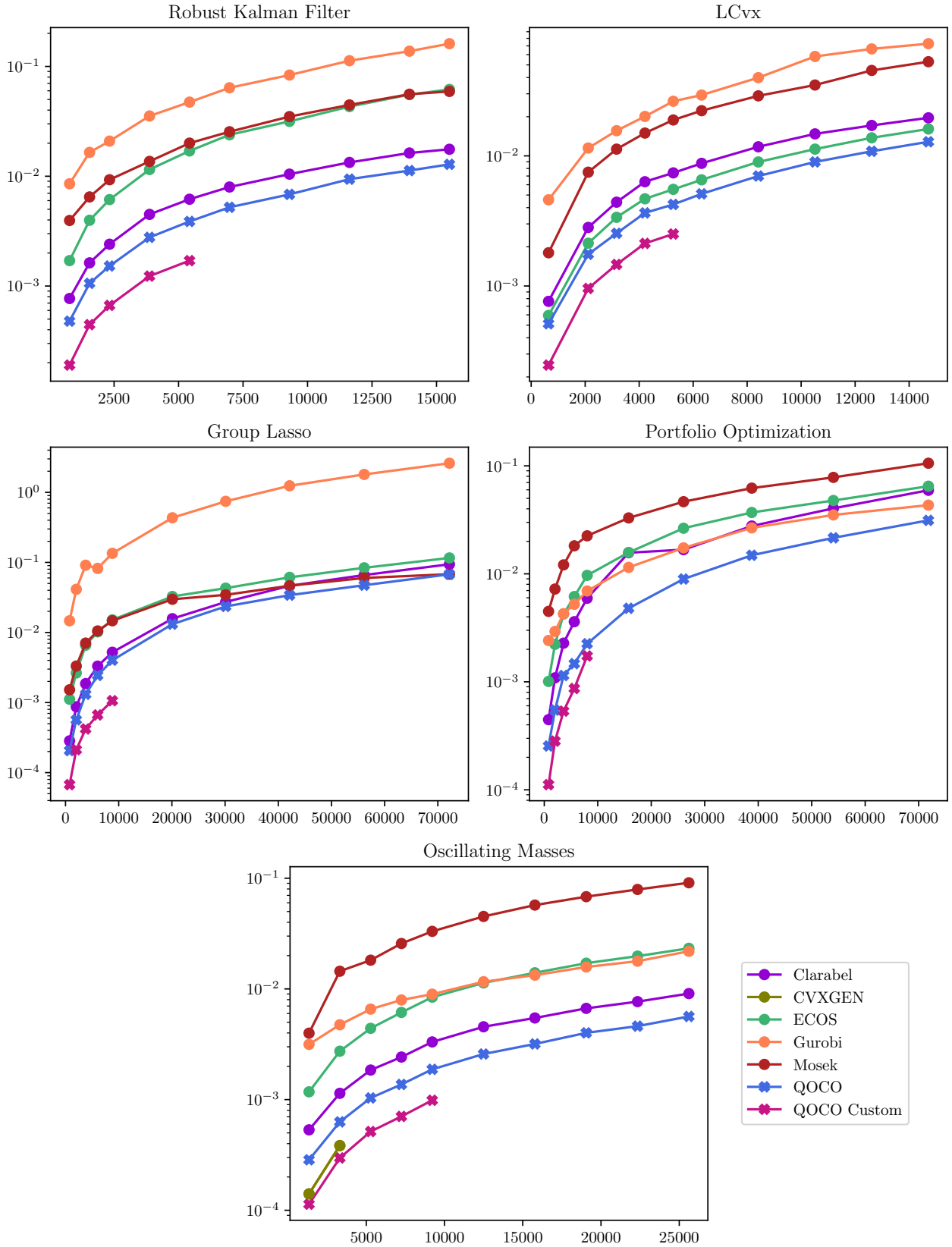
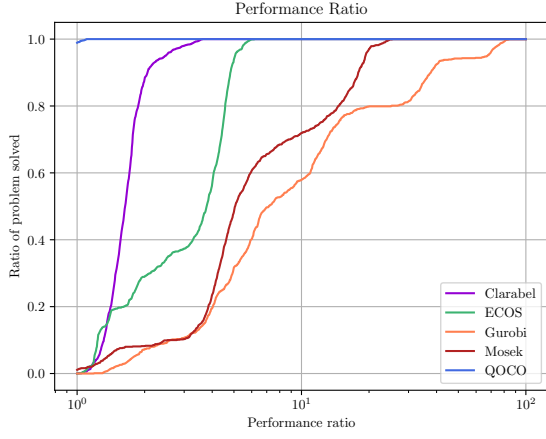
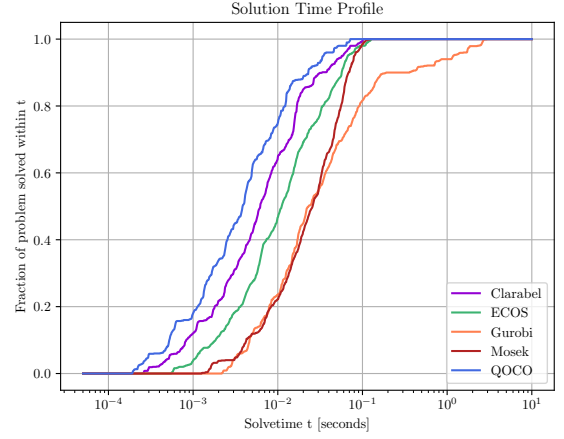


Figure 4: Solvetime in seconds vs problem size for benchmark problems



(a) Relative performance profile

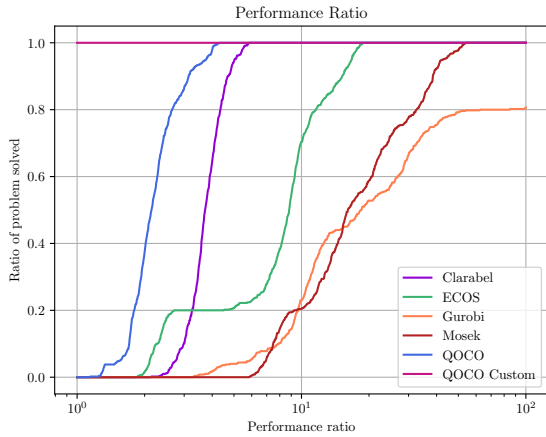


(b) Absolute performance profile

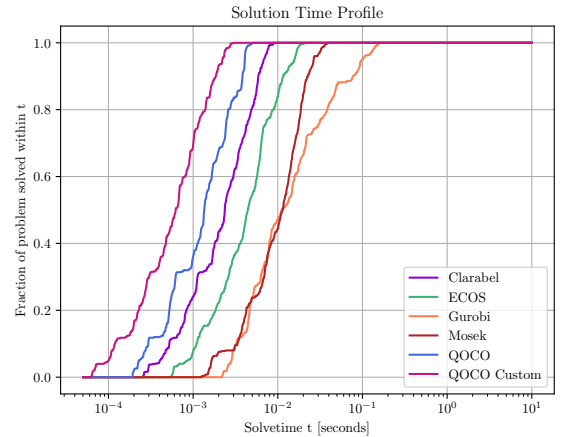
	QOCO	Clarabel	ECOS	Gurobi	Mosek
Shifted GM	1.0	1.5	2.4	14.4	3.8
Failure Rate (%)	0.0	0.0	0.0	0.0	0.0

(c) Shifted geometric means and failure rates

Figure 5: Performance profiles for all benchmark problems



(a) Relative performance profile



(b) Absolute performance profile

	QOCO Custom	QOCO	Clarabel	ECOS	Gurobi	Mosek
Shifted GM	1.0	2.1	3.5	6.9	30.4	15.1
Failure Rate (%)	0.0	0.0	0.0	0.0	0.0	0.0

(c) Shifted geometric means and failure rates

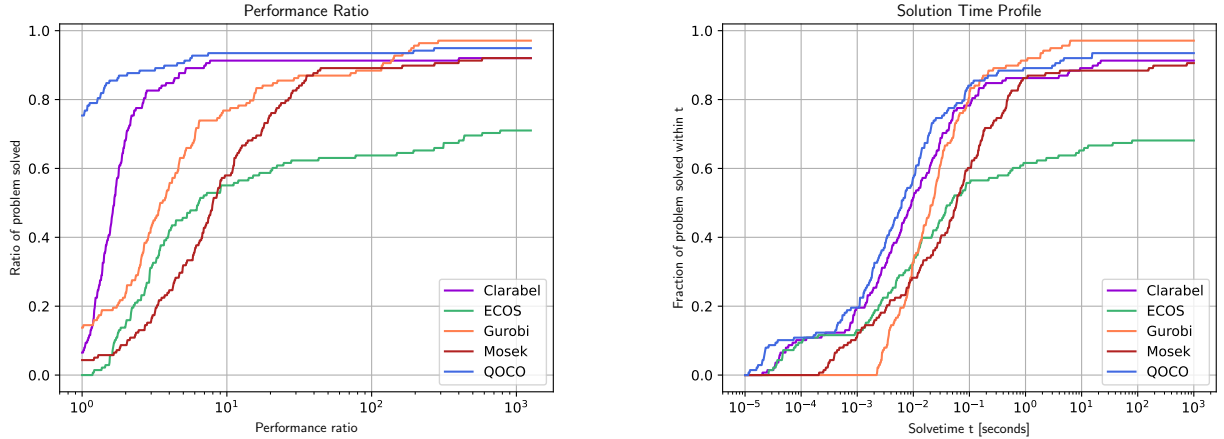
Figure 6: Performance profiles for benchmark problems with custom solver

5.2 Maros–Mészáros problems

The Maros–Mészáros problems are a set of 138 QPs that include a wide range of problem sizes (from 3 to around 300,000 variables) and contain very difficult problems that have ill-conditioning and poor scaling [61]. These properties make the Maros–Mészáros problems a good test set to assess the robustness of solvers. We test QOCO but not QOCO_{custom} on these problems since many problems are far too large to generate code for. Additionally, we ran each problem once rather than the 100 runs we did for the benchmark problems since many of the Maros–Mészáros problems are extremely large, and running

them 100 times per solver would be extremely cumbersome.

Figure 7 shows that the proprietary solver Gurobi successfully solved the largest fraction of the problems followed by QOCO, CLARABEL, MOSEK, and ECOS. We can also see that for the majority of the problems, QOCO was the fastest solver, whereas MOSEK and ECOS tended to be the slowest. The latter result is unsurprising as MOSEK and ECOS can only handle linear objectives and the reformulation of the quadratic objective into a second-order cone likely introduced significant fill-in, slowing these solvers down. We also observed that for many of the largest problems, Gurobi was the fastest solver, due to multithreading in its matrix factorization.



(a) Relative performance profile

(b) Absolute performance profile

	QOCO	Clarabel	ECOS	Gurobi	Mosek
Shifted GM	2.0	3.0	27.4	1.0	3.8
Failure Rate (%)	6.5	8.7	31.9	2.9	9.4

(c) Shifted geometric means and failure rates

Figure 7: Performance profiles for Maros–Mészáros problems

6 Conclusion

We have presented QOCO, a C-based solver, and QOCOGEN, a custom solver generator for quadratic objective second-order cone programs (SOCPs). Both implement primal-dual interior-point methods, with QOCOGEN generating custom solvers written in C, called $\text{QOCO}_{\text{custom}}$, that exploit the sparsity pattern of problems to gain a computational advantage. We demonstrate that QOCO is robust and faster than most of its competitors and $\text{QOCO}_{\text{custom}}$ is significantly faster than QOCO, making it a useful tool for real-time applications, such as model-predictive control, trajectory optimization, and other domains where computational efficiency is critical.

Since both QOCO and QOCOGEN are open-source, they are accessible to users in both academia and industry. QOCO and QOCOGEN are integrated with CVXPY and CVXPYgen respectively, allowing users to formulate optimization problems in a natural way following from math rather than manually converting the problem to the solver-required standard form. This integration enhances usability of our solvers.

Documentation and installation instructions for QOCO and QOCOGEN are available at <https://qoco-org.github.io/qoco/index.html>.

Acknowledgments This research was supported by ONR grant N00014-20-1-2288, AFOSR grant FA9550-20-1-0053, and Blue Origin, LLC. Government sponsorship is acknowledged. The authors would like to thank Danylo Malyuta for his review of this paper.

References

- [1] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.
- [2] L. R. Ford and D. R. Fulkerson, “A suggested computation for maximal multi-commodity network flows,” *Management Science*, vol. 5, no. 1, pp. 97–101, 1958.
- [3] H. Markowitz, “Portfolio selection,” *J. Finance*, vol. 7, p. 77, Mar. 1952.
- [4] M. S. Lobo, M. Fazel, and S. Boyd, “Portfolio optimization with linear and fixed transaction costs,” *Annals of Operations Research*, vol. 152, pp. 341–365, 2007.
- [5] A. Ben-Tal and A. Nemirovski, “Robust convex optimization,” *Mathematics of Operations Research*, vol. 23, no. 4, pp. 769–805, 1998.
- [6] A. Ben-Tal and A. Nemirovski, “Robust solutions of uncertain linear programs,” *Operations Research Letters*, vol. 25, no. 1, pp. 1–13, 1999.
- [7] M. S. Lobo, L. Vandenberghe, S. Boyd, and H. Lebret, “Applications of second-order cone programming,” *Linear algebra and its applications*, vol. 284, no. 1-3, pp. 193–228, 1998.
- [8] D. Malyuta, T. P. Reynolds, M. Szmuk, T. Lew, R. Bonalli, M. Pavone, and B. Açikmeşe, “Convex optimization for trajectory generation: A tutorial on generating dynamically feasible trajectories reliably and efficiently,” *IEEE Control Systems Magazine*, vol. 42, no. 5, pp. 40–113, 2022.
- [9] D. Drusvyatskiy and A. S. Lewis, “Error bounds, quadratic growth, and linear convergence of proximal methods,” *Mathematics of Operations Research*, vol. 43, no. 3, pp. 919–948, 2018.
- [10] T. Lipp and S. Boyd, “Variations and extension of the convex–concave procedure,” *Optimization and Engineering*, vol. 17, pp. 263–287, 2016.
- [11] A. L. Yuille and A. Rangarajan, “The concave-convex procedure (cccp),” *Advances in neural information processing systems*, vol. 14, 2001.
- [12] P. J. Goulart and Y. Chen, “Clarabel: An interior-point solver for conic programs with quadratic objectives,” 2024.
- [13] Gurobi Optimization, LLC, “Gurobi Optimizer Reference Manual,” 2023.
- [14] M. Garstka, M. Cannon, and P. Goulart, “Cosmo: A conic operator splitting method for convex conic problems,” *Journal of Optimization Theory and Applications*, vol. 190, p. 779–810, Aug. 2021.
- [15] B. O’Donoghue, “Operator splitting for a homogeneous embedding of the linear complementarity problem,” *SIAM Journal on Optimization*, vol. 31, pp. 1999–2023, 08 2021.
- [16] A. Themelis and P. Patrinos, “Supermann: a superlinearly convergent algorithm for finding fixed points of nonexpansive operators,” *IEEE Transactions on Automatic Control*, vol. 64, no. 12, pp. 4875–4890, 2019.
- [17] E. K. Ryu and W. Yin, *Large-Scale Convex Optimization: Algorithms and Analyses via Monotone Operators*. Cambridge University Press, Nov. 2022.
- [18] J. Rawlings, E. Meadows, and K. Muske, “Nonlinear model predictive control: A tutorial and survey,” *IFAC Proceedings Volumes*, vol. 27, no. 2, pp. 185–197, 1994. IFAC Symposium on Advanced Control of Chemical Processes, Kyoto, Japan, 25-27 May 1994.
- [19] J. Nocedal and S. Wright, *Numerical optimization*, pp. 1–664. Springer Series in Operations Research and Financial Engineering, Springer Nature, 2006.
- [20] P. T. Boggs and J. W. Tolle, “Sequential quadratic programming,” *Acta numerica*, vol. 4, pp. 1–51, 1995.

- [21] M. Szmuk, T. P. Reynolds, and B. Açıkmeşe, “Successive convexification for real-time six-degree-of-freedom powered descent guidance with state-triggered constraints,” *Journal of Guidance, Control, and Dynamics*, vol. 43, p. 1399–1413, Aug. 2020.
- [22] A. G. Kamath, P. Elango, Y. Yu, S. Mceowen, G. M. Chari, J. M. Carson III, and B. Açıkmeşe, “Real-time sequential conic optimization for multi-phase rocket landing guidance,” *IFAC-PapersOnLine*, vol. 56, no. 2, pp. 3118–3125, 2023. 22nd IFAC World Congress.
- [23] G. M. Chari, A. G. Kamath, P. Elango, and B. Acikmese, “Fast monte carlo analysis for 6-dof powered-descent guidance via gpu-accelerated sequential convex programming,” in *AIAA SCITECH 2024 Forum*, American Institute of Aeronautics and Astronautics, Jan. 2024.
- [24] A. Berning, E. Burnett, and S. Bieniawski, “Chance-constrained, drift-safe guidance for spacecraft rendezvous,” in *AAS Rocky Mountain Guidance, Navigation and Control Conference*, AAS, 2023.
- [25] G. M. Chari and B. Açıkmeşe, “Spacecraft rendezvous guidance via factorization-free sequential convex programming using a first-order method,” *arXiv preprint arXiv:2402.04561*, 2024.
- [26] S. Mceowen, A. G. Kamath, P. Elango, T. Kim, S. C. Buckner, and B. Acikmese, “High-accuracy 3-dof hypersonic reentry guidance via sequential convex programming,” in *AIAA scitech 2023 forum*, p. 0300, 2023.
- [27] S. Mceowen, D. J. Calderone, A. Tiwary, J. S. Zhou, T. Kim, P. Elango, and B. Acikmese, “Auto-tuned primal-dual successive convexification for hypersonic reentry guidance,” in *AIAA SCITECH 2025 Forum*, p. 1317, 2025.
- [28] J. M. Carson, M. M. Munk, R. R. Sostaric, J. N. Estes, F. Amzajerddian, J. B. Blair, D. K. Rutishauser, C. I. Restrepo, A. M. Dwyer-Cianciolo, G. Chen, and T. Tse, *The SPLICE Project: Continuing NASA Development of GN&C Technologies for Safe and Precise Landing*.
- [29] T. P. Reynolds, M. Szmuk, D. Malyuta, M. Mesbahi, B. Açıkmeşe, and J. M. Carson, “Dual quaternion-based powered descent guidance with state-triggered constraints,” *Journal of Guidance, Control, and Dynamics*, vol. 43, p. 1584–1599, Sept. 2020.
- [30] A. G. Kamath, P. Elango, T. Kim, S. Mceowen, Y. Yu, J. M. Carson, M. Mesbahi, and B. Acikmese, *Customized Real-Time First-Order Methods for Onboard Dual Quaternion-based 6-DoF Powered-Descent Guidance*.
- [31] G. Mendeck and W. May, “Space technology mission directorate-game changing development program-fy23 splice annual review presentation,” in *NASA Game Changing Development Annual Program Review*, 2023.
- [32] A. Domahidi, E. Chu, and S. Boyd, “ECOS: An SOCP solver for embedded systems,” in *European Control Conference (ECC)*, pp. 3071–3076, 2013.
- [33] M. ApS, *MOSEK Optimization Suite 10.2.6*, 2024.
- [34] J. Mattingley and S. Boyd, “Cvxgen: A code generator for embedded convex optimization,” *Optimization and Engineering*, vol. 13, pp. 1–27, 2012.
- [35] D. Dueri, J. Zhang, and B. Açıkmeşe, “Automated custom code generation for embedded, real-time second order cone programming,” *IFAC Proceedings Volumes*, vol. 47, no. 3, pp. 1605–1612, 2014. 19th IFAC World Congress.
- [36] D. Dueri, B. Açıkmeşe, D. P. Scharf, and M. W. Harris, “Customized real-time interior-point methods for onboard powered-descent guidance,” *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 197–212, 2017.
- [37] B. Acikmese and S. R. Ploen, “Convex programming approach to powered descent guidance for mars landing,” *Journal of Guidance Control and Dynamics*, vol. 30, pp. 1353–1366, Sept. 2007.

- [38] B. Acikmese, J. M. Carson, and L. Blackmore, “Lossless convexification of nonconvex control bound and pointing constraints of the soft landing optimal control problem,” *IEEE Transactions on Control Systems Technology*, vol. 21, no. 6, pp. 2104–2113, 2013.
- [39] D. P. Scharf, B. Açıkmeşe, D. Dueri, J. Benito, and J. Casoliva, “Implementation and experimental demonstration of onboard powered-descent guidance,” *Journal of Guidance, Control, and Dynamics*, vol. 40, p. 213–229, Feb. 2017.
- [40] L. Blackmore, “Autonomous precision landing of space rockets,” in *Frontiers of Engineering: Reports on Leading-Edge Engineering from the 2016 Symposium*, vol. 46, pp. 15–20, 2016.
- [41] S. Diamond and S. Boyd, “CVXPY: A Python-embedded modeling language for convex optimization,” *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.
- [42] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, “A rewriting system for convex optimization problems,” *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.
- [43] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, “Embedded code generation with cvxpy,” *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.
- [44] S. Mehrotra, “On the implementation of a primal-dual interior point method,” *SIAM Journal on Optimization*, vol. 2, no. 4, pp. 575–601, 1992.
- [45] P. R. Amestoy, T. A. Davis, and I. S. Duff, “An approximate minimum degree ordering algorithm,” *SIAM Journal on Matrix Analysis and Applications*, vol. 17, p. 886–905, Oct. 1996.
- [46] T. A. Davis, “Algorithm 849: A concise sparse cholesky factorization package,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 31, no. 4, pp. 587–591, 2005.
- [47] L. Vandenberghe, “The cvxopt linear and quadratic cone program solvers,” *Online: <http://cvxopt.org/documentation/coneprog.pdf>*, 2010.
- [48] A. Ben-Tal and A. Nemirovski, *Lectures on Modern Convex Optimization: Analysis, Algorithms, and Engineering Applications*. Society for Industrial and Applied Mathematics, Jan. 2001.
- [49] F. Alizadeh and D. Goldfarb, “Second-order cone programming,” *Mathematical programming*, vol. 95, no. 1, pp. 3–51, 2003.
- [50] S. J. Wright, *Primal-Dual Interior-Point Methods*. Society for Industrial and Applied Mathematics, Jan. 1997.
- [51] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, Mar. 2004.
- [52] J. Faraut and A. Korányi, *Analysis on symmetric cones*. Oxford university press, 1994.
- [53] Y. E. Nesterov and M. J. Todd, “Self-scaled barriers and interior-point methods for convex programming,” *Mathematics of Operations research*, vol. 22, no. 1, pp. 1–42, 1997.
- [54] Y. E. Nesterov and M. J. Todd, “Primal-dual interior-point methods for self-scaled cones,” *SIAM Journal on optimization*, vol. 8, no. 2, pp. 324–364, 1998.
- [55] J. R. Bunch and B. N. Parlett, “Direct methods for solving symmetric indefinite systems of linear equations,” *SIAM Journal on Numerical Analysis*, vol. 8, no. 4, pp. 639–655, 1971.
- [56] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: an operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.
- [57] R. J. Vanderbei, “Symmetric quasidefinite matrices,” *SIAM Journal on Optimization*, vol. 5, no. 1, pp. 100–113, 1995.
- [58] M. Yannakakis, “Computing the minimum fill-in is np-complete,” *SIAM Journal on Algebraic Discrete Methods*, vol. 2, p. 77–79, Mar. 1981.

- [59] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes 3rd Edition: The Art of Scientific Computing*. USA: Cambridge University Press, 3 ed., 2007.
- [60] Kitware, Inc., *CMake: Cross-Platform Build System*, 2024.
- [61] I. Maros and C. Mészáros, “A repository of convex quadratic programming problems,” *Optimization Methods and Software*, vol. 11, p. 671–681, Jan. 1999.
- [62] E. D. Dolan and J. J. Moré, “Benchmarking optimization software with performance profiles,” *Mathematical Programming*, vol. 91, p. 201–213, Jan. 2002.
- [63] R. Schwan, Y. Jiang, D. Kuhn, and C. N. Jones, “PIQP: A proximal interior-point quadratic programming solver,” in *2023 62nd IEEE Conference on Decision and Control (CDC)*, pp. 1088–1093, 2023.
- [64] G. M. Chari and B. Açikmeşe, “QOCO Benchmarks.” <https://github.com/qoco-org/qoco-benchmarks>, 2025.
- [65] M. Yuan and Y. Lin, “Model selection and estimation in regression with grouped variables,” *Journal of the Royal Statistical Society Series B*, vol. 68, pp. 49–67, 02 2006.
- [66] Y. Wang and S. Boyd, “Fast model predictive control using online optimization,” *IEEE Transactions on Control Systems Technology*, vol. 18, no. 2, pp. 267–278, 2010.
- [67] CVXPY, “Robust kalman filtering for vehicle tracking.” https://www.cvxpy.org/examples/applications/robust_kalman.html. Accessed: 2024-11-16.

A Nonsymmetric Newton System

In this section we will show that Newton steps applied to Equations (5a), (5b), (5c), (5d) result in a nonsymmetric linear system which is less efficient to store and factor than the symmetric system presented in Equation (19).

We must first introduce the arrow matrix, a matrix that will allow us to rewrite the Jordan product, defined in Equation (6), as a matrix-vector multiplication. The arrow matrix can be written for vectors in the non-negative orthant and second-order cone as

$$\text{Arw}(x) = \begin{cases} \text{diag}(x) & \text{if } x \in \mathbb{R}_+^l \\ \begin{bmatrix} x_0 & x_1^\top \\ x_1 & x_0 I \end{bmatrix} & \text{if } x \in \mathcal{Q}^q. \end{cases}$$

For a vector in cone $\mathcal{K}$ where

$$\mathcal{K} = \mathcal{C}_1 \times \mathcal{C}_2 \times \cdots \times \mathcal{C}_K,$$

and $\mathcal{C}_i$ is the non-negative orthant or a second-order cone, we can write the arrow matrix as

$$\text{Arw}(x) = \text{blkdiag}(\text{Arw}(x_1), \dots, \text{Arw}(x_K)) \quad \text{where } x_i \in \mathcal{C}_i,$$

Note that if x is in the interior of $\mathcal{K}$, then $\text{Arw}(x)$ is positive definite and thus is invertible.

We can then write the Jordan product for two vectors $u, v \in \mathcal{K}$ as

$$x \circ y = \text{Arw}(x)y,$$

We now linearize Equations (9a) - (9e) about the current iterate, (x_k, s_k, y_k, z_k) , write the Jordan products using the arrow matrix and get the linear system

$$\begin{bmatrix} P & 0 & A^\top & G^\top \\ 0 & \text{Arw}(z_k) & 0 & \text{Arw}(s_k) \\ A & 0 & 0 & 0 \\ G & I & 0 & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta s \\ \Delta y \\ \Delta z \end{bmatrix} = \begin{bmatrix} -r_x \\ -r_s \\ -r_y \\ -r_z \end{bmatrix}$$

We can attempt to symmetrize the coefficient matrix by multiplying the second equation by $\text{Arw}(s_k)^{-1}$ to get

$$\begin{bmatrix} P & 0 & A^\top & G^\top \\ 0 & \text{Arw}(s_k)^{-1} \text{Arw}(z_k) & 0 & I \\ A & 0 & 0 & 0 \\ G & I & 0 & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta s \\ \Delta y \\ \Delta z \end{bmatrix} = \begin{bmatrix} -r_x \\ -\text{Arw}(s_k)^{-1} r_s \\ -r_y \\ -r_z \end{bmatrix}. \quad (32)$$

However, the coefficient matrix above will only be symmetric if $\text{Arw}(s_k)^{-1} \text{Arw}(z_k)$ is symmetric. This can only be ensured if $\mathcal{K}$ only consisted of the non-negative orthant and no second-order cones. In other words, the system in (32) would only be symmetric if Problem 1 was a quadratic program rather than a second-order cone program. Even if we were to eliminate Δs from Equation (32), we would still not have a symmetric linear system.

B Custom LDL Factorization Performance

This section provides empirical evidence supporting our claims in Sections 1 and 4.1. Specifically, we show that sparse linear algebra incurs extra overhead from identifying and locating nonzero elements, leading to more CPU instructions, memory accesses and cache misses. In contrast, a custom implementation that hardcodes the exact memory accesses and floating point operations reduces these inefficiencies. We specifically compare QDLDL (used in QOCO) against our custom $LDL^\top$ factorization (used in QOCO_{custom}), since the most expensive operation in a generic implementation of Algorithm 1 is the $LDL^\top$ factorization. To compare the two matrix factorization implementations, we use Linux `perf`, a profiling tool that provides CPU performance metrics, which will allow us to quantify the computational overhead of sparse versus custom factorizations.

As a representative example of a matrix we would factor, we consider the KKT matrix associated with the Linear Quadratic Regulator (LQR). The LQR problem is

$$\begin{aligned} & \underset{x,u}{\text{minimize}} && \frac{1}{2} \left(\sum_{k=1}^T x_k^\top Q_k x_k + \sum_{k=1}^{T-1} u_k^\top R_k u_k \right) \\ & \text{subject to} && x_{k+1} = A_k x_k + B_k u_k \quad \forall k \in [1, T-1] \\ & && x_1 = x_{\text{init}}, \end{aligned}$$

and its associated KKT matrix is

$$K = \begin{bmatrix} P & H^\top \\ H & -\epsilon I \end{bmatrix},$$

where

$$P = \text{blkdiag}(Q_1, Q_2, \dots, Q_T, R_1, R_2, \dots, R_{T-1})$$

$$H = \begin{bmatrix} A_1 & -I & & & & B_1 & & \\ & A_2 & -I & & & & B_2 & \\ & & & \ddots & & & & \ddots \\ & & & & A_{N-1} & -I & & B_{N-1} \\ I & & & & & & & \end{bmatrix}.$$

Notice that we apply static regularization to ensure that the (2,2) block of K is negative definite, making K quasidefinite. The sparsity pattern of K is depicted by Figure 8.

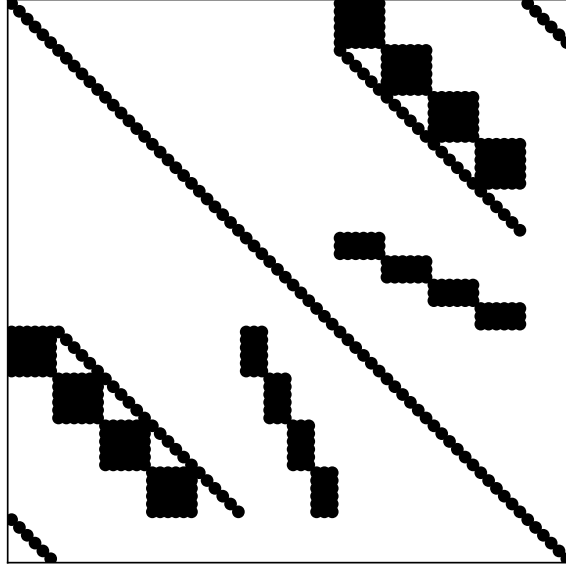


Figure 8: Sparsity pattern of LQR KKT matrix for $T=5$

For simplicity, we assume time-invariant system and cost matrices, i.e., $Q_k = R_k = I$ and fixed matrices $A \in \mathbb{R}^{6 \times 6}$, $B \in \mathbb{R}^{6 \times 3}$. We choose the elements of A and B randomly and consider time horizons T : $\{5, 15, 50, 75, 100\}$. The actual elements of these matrices do not matter much, as our goal with these experiments is to understand the CPU behavior which (to first order) is unaffected by the value of elements in the matrices. To get accurate profiling results, we run each matrix factorization 1 million times to get the average runtime, number of instructions, number of L1 data cache loads, number of L1 cache misses, number of branches, and number of branch mispredictions.

From Table 1, we can see that the custom $LDL^\top$ factorization is about four times faster than the sparse factorization because it does much less work: issuing fewer instructions, performing fewer memory accesses, and incurring fewer L1 cache misses. Although QDLDL has a far more branches than our custom $LDL^\top$ implementation, the branches are quite predictable, and pipeline stalls are avoided.

By hardcoding the exact memory accesses and floating point operations, our custom $LDL^\top$ factorization eliminates much of the overhead associated with sparse matrix operations. This substantial reduction in memory accesses and instructions leads to a significantly faster factorization.

Table 1: Perf results for qdldl and custom $LDL^\top$ factorization

Size	<u>Runtime (us)</u>		<u>Instructions</u>		<u>L1 cache loads</u>		<u>L1 cache misses</u>		<u>Branches</u>		<u>Branch misses</u>	
	qdldl	custom	qdldl	custom	qdldl	custom	qdldl	custom	qdldl	custom	qdldl	custom
5	2.103	0.435	36650	4858	14049	1851	0	0	5419	5	1	0
15	7.507	1.777	132796	18053	50550	6442	572	27	19124	8	4	0
50	27.077	6.117	469289	62046	179709	21468	2539	1204	67081	17	11	1
75	38.848	8.485	709731	93912	279033	32211	3735	1828	101347	24	14	2
100	49.577	12.894	950101	125747	365981	43559	5567	2459	135597	35	18	5

C Problem Classes

In this section, we describe the five problem classes considered in our numerical experiments, along with various problem sizes and instances within each problem class. We use the notation $\mathcal{U}(a, b)$ to denote a uniform distribution on the interval $[a, b]$, $\mathcal{N}(\mu, \sigma)$ for a Gaussian distribution with mean μ and standard deviation σ , $0_{m \times n}$ for the matrix of all zeros in $\mathbb{R}^{m \times n}$, and I_n for the identity matrix in $\mathbb{R}^{n \times n}$.

C.1 Robust Kalman Filter

We consider the robust Kalman filtering problem for vehicle tracking, as outlined in [67]. This problem is a quadratic objective second-order cone program (SOCP) and takes the form

$$\begin{aligned} & \underset{x_k, w_k, v_k}{\text{minimize}} && \sum_{k=0}^{N-1} (\|w_k\|_2^2 + \tau \phi_\rho(v_k)) \\ & \text{subject to} && x_{k+1} = Ax_k + Bw_k \quad \forall k \in [0, N-1] \\ & && y_k = Cx_k + v_k \quad \forall k \in [0, N-1], \end{aligned} \tag{33}$$

where ϕ_ρ is the Huber loss function

$$\phi_\rho(z) = \begin{cases} \|z\|_2^2 & \|z\|_2 \leq \rho \\ 2\rho\|z\|_2 - \rho^2 & \|z\|_2 > \rho. \end{cases}$$

In this problem, $x_k \in \mathbb{R}^4$ represents the state of the vehicle at timestep k , $w_k \in \mathbb{R}^2$ is an unknown force vector acting on the vehicle at timestep k , $v_k \in \mathbb{R}^2$ is the measurement noise vector at timestep k , and $y_k \in \mathbb{R}^2$ represents a noisy measurement of the vehicle's state at timestep k . The matrices A and B are known dynamics matrices for the system, and C is the known observation matrix. To reconstruct the vehicle's state trajectory, we solve Problem 33.

We use the system matrices

$$\begin{aligned} A &= \begin{bmatrix} 1 & 0 & (1 - \frac{\gamma}{2}\Delta t)\Delta t & 0 \\ 0 & 1 & 0 & (1 - \frac{\gamma}{2}\Delta t)\Delta t \\ 0 & 0 & 1 - \gamma\Delta t & 0 \\ 0 & 0 & 0 & 1 - \gamma\Delta t \end{bmatrix} \\ B &= \begin{bmatrix} \frac{1}{2}\Delta t^2 & 0 \\ 0 & \frac{1}{2}\Delta t^2 \\ \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \\ C &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \end{aligned}$$

which correspond to a two-dimensional double integrator model with linear velocity drag and unit mass, where the input is a force and we observe the position of the vehicle. We use $\gamma = 0.05$ for the velocity drag parameter, Δt for the discretization time, and $\rho = 2$ and $\tau = 2$ for the Huber loss function.

Problem Sizes To generate different problem sizes, we vary the number of timesteps N . We consider the following values for N : $\{25, 50, 75, 125, 175, 225, 300, 375, 450, 500\}$. For each of these problem sizes, we set $\Delta t = T/(N-1)$, where $T = 50$ is the time horizon.

Problem Instances For each problem size, we generate 20 random problem instances, corresponding to 20 different observation trajectories y_t . To do this, we first randomly generate the force and noise vectors w_t and v_t , respectively, for $t \in [0, N - 1]$. Then, we forward-simulate an initial state of $x_0 = [0 \ 0 \ 0 \ 0]$ through the dynamics defined in the constraints of Problem 33. Each element of w_t and v_t is drawn from a standard Gaussian distribution $\mathcal{N}(0, 1)$. Additionally, for 20 % of the elements of v_t , we introduce outlier noise by drawing elements from $\mathcal{N}(0, 20)$, to simulate noise with a larger magnitude.

C.2 Lossless Convexification

The losslessly convexified powered-descent guidance problem is an optimal control problem that aims to compute a soft landing trajectory for a rocket while respecting relevant constraints. One important constraint is the lower thrust bound on the engine, which is nonconvex. However, it has been shown that this constraint admits a *lossless* convex relaxation [37, 38]. This means that the solution to the relaxed convex problem guarantees a solution to the original nonconvex problem. The losslessly convexified problem can be written as an SOCP

$$\begin{aligned}
& \underset{x, z, u, \sigma}{\text{minimize}} && -z_T \\
& \text{subject to} && x_{k+1} = Ax_k + Bu_k + g \quad \forall k \in [0, T - 1] \\
& && z_{k+1} = z_k - \alpha \sigma_k \Delta t \quad \forall k \in [0, T - 1] \\
& && \|u_k\|_2 \leq \sigma_k \quad \forall k \in [0, T - 1] \\
& && \log(m_{\text{wet}} - \alpha \rho_2 k \Delta t) \leq z_k \leq \log(m_{\text{wet}} - \alpha \rho_1 k \Delta t) \quad \forall k \in [0, T - 1] \\
& && \mu_{1,k} \left[1 - [z_k - z_{0,k}] + \frac{[z_k - z_{0,k}]^2}{2} \right] \leq \sigma_k \leq \mu_{2,k} [1 - (z_k - z_{0,k})] \quad \forall k \in [0, T - 1] \\
& && e_3^\top u_k \geq \sigma_k \cos(\theta_{\max}) \quad \forall k \in [0, T - 1] \\
& && x_0 = x_{\text{init}}, \quad z_0 = \log(m_{\text{wet}}), \quad z_T \geq \log(m_{\text{dry}}),
\end{aligned}$$

where $x_k \in \mathbb{R}^6$ is the position and velocity of the rocket at timestep k , $u_k \in \mathbb{R}^3$ is the thrust per unit mass at timestep k , $z_k \in \mathbb{R}$ is the natural logarithm of the rocket's mass at timestep k , and $\sigma_k \in \mathbb{R}$ is a slack variable at timestep k . Further we define $z_{0,k} := \log(m_{\text{wet}} - \alpha \rho_2 k \Delta t)$, $\mu_{1,k} := \rho_1 e^{-z_0}$, and $\mu_{2,k} := \rho_2 e^{-z_0}$. The parameter $\rho_1 \in \mathbb{R}$ is the minimum thrust of the engine, $\rho_2 \in \mathbb{R}$ is the maximum thrust of the engine, $\alpha \in \mathbb{R}$ is a mass depletion parameter that describes how efficient the engine is, $m_{\text{wet}} \in \mathbb{R}$ is the sum of the dry mass of the rocket and the initial fuel mass, $\theta_{\max} \in \mathbb{R}$ is the maximum angle from vertical that the thrust vector can make, $x_{\text{init}} \in \mathbb{R}^6$ is the initial state of the rocket, $\Delta t \in \mathbb{R}$ is the discretization time, and e_3 is the third canonical basis vector in $\mathbb{R}^3$. The dynamics matrices A and B , and the vector g , are defined as

$$\begin{aligned}
A &= \begin{bmatrix} I_3 & \Delta t(I_3) \\ 0_{3 \times 3} & I_3 \end{bmatrix} \\
B &= \begin{bmatrix} \frac{1}{2} \Delta t^2(I_3) \\ \Delta t(I_3) \end{bmatrix} \\
g &= [0 \ 0 \ -0.5g_0 \Delta t^2 \ 0 \ 0 \ -g_0 \Delta t].
\end{aligned}$$

For this problem class we use the parameters

$$g_0 = 9.807, \quad \rho_1 = 100, \quad \rho_2 = 500, \quad m_{\text{dry}} = 25, \quad m_{\text{wet}} = 35, \quad \theta_{\max} = \pi/4, \quad \alpha = 0.001.$$

Problem Sizes To generate different problem sizes, we vary the number of timesteps T . The discretization time is computed as $\Delta t = t_f / (T - 1)$, where $t_f = 20$. We consider the following values for T : {15, 50, 75, 100, 125, 150, 200, 250, 300, 350}.

Problem Instances For each problem size (i.e., for each number of timesteps), we generate 20 random problem instances, which involves generating a random initial state x_{init} . We generate x_{init} as

$$x_{\text{init}} \sim \begin{bmatrix} \mathcal{U}(-10, 10) \\ \mathcal{U}(-10, 10) \\ \mathcal{U}(200, 400) \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

C.3 Group Lasso

The group lasso problem is a quadratic objective SOCP. It is a variation of the least-squares regression problem, where the regression variables are partitioned into disjoint groups, and a regularization term is added to encourage group sparsity. Let $x = [x^{(1)}, x^{(2)}, \dots, x^{(N)}]$ represent the partitioning of the regression variables, where each $x^{(i)}$ is a group of variables. We can write the problem as

$$\underset{x}{\text{minimize}} \quad \|Ax - b\|_2^2 + \lambda \sum_{i=1}^N \|x^{(i)}\|_2.$$

This can be written as an SOCP by introducing a slack variable $t \in \mathbb{R}^N$, and an auxiliary variable $y \in \mathbb{R}^m$ to avoid computing $A^\top A$ and maintain sparsity in the Hessian of the objective function. The problem is now

$$\begin{aligned} \underset{x, t}{\text{minimize}} \quad & \|y\|_2^2 + \lambda \sum_{i=1}^N t_i \\ \text{subject to} \quad & \|x^{(i)}\|_2 \leq t_i \quad \forall i \in [1, N] \\ & y = Ax - b, \end{aligned}$$

where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$.

Problem Sizes To generate different problem sizes, we vary the number of groups N . We consider the following values for N : $\{1, 2, 3, 4, 5, 8, 10, 12, 14, 16\}$.

Problem Instances For each problem size (i.e., for each number of groups) we generate 20 random problem instances, corresponding to the generation of the matrix A and the vector b . We set A to have $m = 250N$ rows and $n = 10N$ columns, where x is partitioned into N groups, each containing ten regression variables. The matrix A is sparse, with 10% of its elements nonzero, and the nonzero elements are drawn from the uniform distribution $\mathcal{U}(0, 1)$.

To generate b , we first generate $\hat{x} \in \mathbb{R}^n$ and partition it into N groups of ten variables. Half of the groups are set to zero, and the remaining elements are drawn from the standard Gaussian distribution $\mathcal{N}(0, 1)$. Next, we generate an error vector $e \in \mathbb{R}^m$ with elements drawn from $\mathcal{N}(0, 1/n)$. Finally, we compute $b = A\hat{x} + e$. Note that each problem instance will have a different sparsity pattern for A , which requires us to generate a new custom solver for each instance.

C.4 Portfolio Optimization

We consider the Markowitz portfolio optimization problem with a no-short-selling constraint [3, 51]. This problem is the QP

$$\begin{aligned}
& \underset{x}{\text{minimize}} && \gamma x^\top \Sigma x - \mu^\top x \\
& \text{subject to} && \sum_{i=1}^n x_i = 1 \\
& && x_i \geq 0 \quad \forall i \in [1, n],
\end{aligned}$$

where $x \in \mathbb{R}^n$ is the allocation vector, with x_i representing the percentage of resources invested in asset i , $\mu \in \mathbb{R}^n$ is the vector of expected returns, and $\Sigma \in \mathbb{S}_+^n$ is the covariance matrix of asset returns.

Typically, the return covariance matrix Σ is approximated as the sum of a low-rank matrix and a diagonal matrix. Thus, we write $\Sigma = FF^\top + D$, where $F \in \mathbb{R}^{n \times k}$ is a factor matrix of rank k . We then add the auxiliary variable y and the constraint $y = F^\top x$ to avoid computing $FF^\top$ and maintain sparsity in the Hessian of the objective function. The final problem can be rewritten as

$$\begin{aligned}
& \underset{x, y}{\text{minimize}} && x^\top Dx + y^\top y - \gamma^{-1} \mu^\top x \\
& \text{subject to} && y = F^\top x \\
& && \sum_{i=1}^n x_i = 1 \\
& && x_i \geq 0 \quad \forall i \in [1, n].
\end{aligned} \tag{34}$$

Problem Sizes To generate different problem sizes, we vary the number of factors, k . We consider the following values for k : $\{2, 4, 6, 8, 10, 15, 20, 25, 30, 35\}$.

Problem Instances For each problem size (i.e., each number of factors), we generate 20 random problem instances. This corresponds to generating the matrices D , F , and the vector μ . We set the number of assets to $n = 100k$, and for all instances, we take $\gamma = 1$. The diagonal elements of D are drawn from $\mathcal{N}(0, 1)$, and the elements of μ are drawn from $\mathcal{N}(0, \sqrt{k})$. The matrix F is a sparse matrix, with 50% of its elements being nonzero, and its nonzero elements are drawn from $\mathcal{N}(0, 1)$. Note that each problem instance will have a different sparsity pattern for F , which requires us to generate a new custom solver for each instance.

C.5 Oscillating Masses

The oscillating masses problem is an optimal control problem involving a system of N masses in one dimension, where each mass is connected to its neighboring mass by a spring, and the outermost two masses are attached to a fixed wall. The goal is to apply forces to each mass to bring the system to equilibrium [66]. This problem can be formulated as a QP

$$\begin{aligned}
& \underset{x, u}{\text{minimize}} && \frac{1}{2} \left(\sum_{k=0}^T x_k^\top Q x_k + \sum_{k=0}^{T-1} u_k^\top R u_k \right) \\
& \text{subject to} && x_{k+1} = A x_k + B u_k \quad \forall k \in [0, T-1] \\
& && x_0 = x_{\text{init}} \\
& && \|x_k\|_\infty \leq x_{\text{max}} \quad \forall k \in [0, T] \\
& && \|u_k\|_\infty \leq u_{\text{max}} \quad \forall k \in [0, T-1].
\end{aligned}$$

where $x_k \in \mathbb{R}^{2N}$ is the position and velocity of the masses at timestep k , $u_k \in \mathbb{R}^N$ is force applied to each mass at timestep k , and $Q \in \mathbb{S}_+^{2N}$ and $R \in \mathbb{S}_+^N$ penalize deviation from equilibrium and control effort respectively.

The dynamics matrices A and B are derived from an exact discretization of the continuous-time linear dynamics, using a zero-order hold on control input

$$\begin{aligned} A &= e^{A_c \Delta t} \\ B &= A_c^{-1}(A - I_{2N})B_c, \end{aligned}$$

where

$$\begin{aligned} A_c &= \begin{bmatrix} 0_{N \times N} & I_N \\ L_N & 0_{N \times N} \end{bmatrix} \\ B_c &= \begin{bmatrix} 0_{N \times N} \\ I_N \end{bmatrix}. \end{aligned}$$

Here, $L_N \in \mathbb{R}^{N \times N}$ is a symmetric tridiagonal matrix with -2 on the main diagonal and 1 on the subdiagonal and superdiagonal.

For this problem class, we use the parameters

$$N = 4, \Delta t = 0.25, x_{\max} = 2, u_{\max} = 5.$$

Problem Sizes To generate different problem sizes, we vary the number of timesteps T . We consider the following values for T : $\{8, 20, 32, 44, 56, 76, 96, 116, 136, 156\}$.

Problem Instances For each problem size (i.e., each number of timesteps), we generate 20 random problem instances. This involves generating a random initial state x_{init} , as well as the matrices Q and R . We draw each element of x_{init} from $\mathcal{N}(0, 1)$, then clip the elements such that they lie within the interval $[-0.9x_{\max}, 0.9x_{\max}]$. This ensures that the initial state satisfies the state constraints, making the resulting problem feasible. The matrices Q and R are generated as diagonal matrices, with each diagonal element drawn from the uniform distribution $\mathcal{U}(0, 10)$.

D Detailed Numeric Results

Table 2: Iterations and solver runtimes for Maros–Mészáros problems

Problem	Size	Iterations					Solver Runtime (s)				
		QOCO	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	CLARABEL	ECOS	GUROBI	MOSEK
LASER	9216	10	10	-	21	24	0.00258	0.00414	-	0.02421	0.03150
QPCBOE1	4743	21	15	-	21	33	0.00613	0.00632	-	0.01534	0.03697
HUESTIS	39968	7	8	-	14	-	0.01261	0.02198	-	0.03369	-
QBANDM	3007	21	20	32	29	22	0.00345	0.00502	0.01198	0.01520	0.02451
QSC205	775	15	18	17	30	15	0.00111	0.00192	0.00183	0.00950	0.00823
LISWET11	40002	-	-	-	40	31	-	-	-	0.15115	0.26106
LISWET8	40002	-	-	-	417	70	-	-	-	1.83018	0.90131
AUG2DC	60200	0	0	-	2	15	0.01392	0.02377	-	0.02266	0.46025
QSEBA	6548	30	29	-	27	30	0.00717	0.01452	-	0.02000	0.05188
AUG2D	59800	0	0	-	2	11	0.01088	0.02133	-	0.01970	0.30137
QGROW15	7365	18	20	17	20	28	0.00616	0.01114	0.01225	0.02356	0.06942
CONT-200	319196	13	10	30	12	28	3.41491	3.91965	8.99180	0.75313	2.27627
LISWET1	40002	-	-	-	27	52	-	-	-	0.09510	0.55497
ZECEVIC2	9	6	7	8	8	7	0.00002	0.00005	0.00004	0.00275	0.00102
QFFFFF80	8997	95	25	45	48	22	0.08216	0.03295	0.49989	0.06666	0.10850
AUG2DCQP	80400	11	12	-	17	-	0.08341	0.14402	-	0.09956	-
QPILOTNO	16258	39	31	43	56	27	0.09448	0.11113	0.21933	0.12265	0.15589
CVXQP2.M	6733	13	9	30	15	17	0.03255	0.02544	11.03589	0.06452	1.07815
POWELL20	30000	38	-	-	60	-	0.05451	-	-	0.21773	-
STADAT2	15997	17	44	-	18	21	0.00939	0.04388	-	0.02200	0.04019
QPCBOE12	1623	40	19	-	26	48	0.00298	0.00274	-	0.00741	0.01278
PRIMAL3	22292	11	8	18	17	8	0.01240	0.01680	0.03449	0.01265	0.04159
QSHIP08S	21178	14	15	26	24	18	0.02043	0.02884	1.58040	0.04072	0.17699
DUALC2	1645	18	10	13	11	16	0.00054	0.00061	0.00091	0.00860	0.00116
LISWET4	40002	10	22	-	13	19	0.01575	0.05955	-	0.05271	0.13555
QPCSTAIR	4872	25	22	-	23	39	0.00897	0.01026	-	0.02300	0.05212
PRIMALC2	2076	24	15	30	11	16	0.00130	0.00142	0.00353	0.00385	0.00828
LISWET6	40002	9	17	-	13	20	0.01906	0.05229	-	0.05445	0.14808
QSHIP04L	8506	14	15	26	21	28	0.00769	0.00918	0.02240	0.02517	0.06894
QE226	3824	18	24	22	25	19	0.00466	0.00709	0.02897	0.01216	0.01492
PRIMAL4	17520	9	9	16	16	11	0.01266	0.01348	0.02799	0.00947	0.04455
QSCSD8	13844	11	10	15	15	12	0.00735	0.01020	0.05332	0.02534	0.06593
QSCORPIO	1824	15	11	13	24	15	0.00128	0.00169	0.00258	0.00776	0.03019
CONT-201	290393	12	10	-	14	-	4.17722	5.71965	-	0.82300	-
GOULDQP2	3142	9	12	20	13	10	0.00188	0.00313	0.58865	0.00962	0.05237
LISWET10	40002	-	-	-	125	20	-	-	-	0.44295	0.14301
HS268	40	5	10	15	12	15	0.00003	0.00007	0.00009	0.00356	0.00057
QBORE3D	1834	18	28	24	26	20	0.00202	0.00442	0.00487	0.00950	0.01198
QSHIP12L	83825	18	15	34	23	39	0.21193	0.19337	79.39230	0.10314	4.08685
QSHIP04S	5866	14	14	24	19	21	0.00373	0.00592	0.01427	0.01438	0.04390
QSHARE1B	1415	28	28	46	30	31	0.00200	0.00343	0.00512	0.00929	0.02247
QGFRDXPN	3889	24	20	-	43	31	0.00589	0.00682	-	0.03540	0.08180
CONT-050	19796	10	9	24	10	18	0.03964	0.04369	0.10372	0.02808	0.08417
MOSARQP1	8467	9	10	24	15	11	0.00520	0.00843	0.03596	0.01658	0.02250
BOYD2	517049	68	-	-	78	-	2.99925	-	-	6.10176	-
QSCAGR25	2182	17	19	34	24	-	0.00160	0.00345	0.00844	0.01023	-

Table 2: Iterations and solver runtimes for Maros–Mészáros problems

Problem	Size	Iterations					Solver Runtime (s)				
		QOCO	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	CLARABEL	ECOS	GUROBI	MOSEK
HUES-MOD	39968	13	12	-	13	23	0.02296	0.02834	-	0.02918	0.36701
YAO	8007	-	-	-	25	36	-	-	-	0.01411	0.05955
CONT-101	72693	10	10	-	13	25	0.30755	0.44878	-	0.09657	0.47646
HS53	24	5	6	9	8	7	0.00003	0.00004	0.00005	0.00273	0.00030
CVXQP1_M	7482	13	10	-	16	21	0.04223	0.03895	-	0.09931	0.43868
STCQP2	48135	7	8	25	12	14	0.08184	0.10365	0.53351	0.21152	0.16697
QGROW22	10930	22	25	20	25	31	0.00986	0.01891	0.02126	0.03386	0.10940
HS35	11	5	7	10	9	9	0.00002	0.00005	0.00004	0.00366	0.00077
STCQP1	48135	7	7	28	-	24	0.03555	0.04303	0.32154	-	0.24021
STADAT1	15997	20	15	-	185	-	0.01086	0.01243	-	0.17253	-
PRIMAL1	6140	12	9	17	18	8	0.00325	0.00402	0.00986	0.00567	0.00878
DUAL3	6441	9	11	13	11	7	0.00313	0.00378	0.00485	0.01875	0.00723
HS21	8	9	8	8	8	10	0.00002	0.00003	0.00003	0.00255	0.00027
QBEACONF	3664	12	-	29	18	22	0.00182	-	0.00717	0.00796	0.01373
QISRAEL	3109	26	25	31	23	23	0.00366	0.00576	0.01015	0.01732	0.00747
LISWET2	40002	9	17	-	15	20	0.01800	0.04961	-	0.05521	0.14195
DTOC3	49994	4	6	-	8	26	0.01081	0.02869	-	0.02627	0.38133
GOULDQP3	3840	6	7	25	9	17	0.00109	0.00237	3.86470	0.01007	0.17719
PRIMAL2	8691	11	8	16	17	8	0.00431	0.00527	0.01403	0.00586	0.01458
TAME	7	4	5	5	6	4	0.00002	0.00004	0.00003	0.00368	0.00031
PRIMALC5	2860	20	13	14	11	14	0.00141	0.00149	0.00241	0.00348	0.00938
AUG3DC	10419	0	0	14	2	10	0.00227	0.00270	0.03786	0.00950	0.04388
AUG3DCQP	14292	9	10	29	16	22	0.01343	0.01937	0.08650	0.02952	0.16977
QSCFXM2	7228	32	30	36	56	-	0.01022	0.01737	0.04520	0.05266	-
VALUES	4428	12	13	-	-	-	0.00149	0.00242	-	-	-
QAFIRO	121	15	13	14	15	13	0.00017	0.00023	0.00020	0.00347	0.00100
QETAMACR	7761	26	19	25	28	27	0.05831	0.04680	0.91543	0.04582	0.10767
QSCSD6	7070	14	13	16	17	13	0.00407	0.00613	0.02588	0.01711	0.05499
QSTAIR	5423	23	33	32	41	40	0.01108	0.01966	0.03227	0.03192	0.05891
PRIMALC1	2514	25	16	27	10	20	0.00237	0.00153	0.00347	0.00359	0.01178
QSHELL	40488	28	36	-	33	27	0.10774	0.12657	-	0.07681	0.18787
CVXQP2_L	67483	22	9	-	16	31	15.34297	9.37783	-	2.75816	750.19697
QSCFXM1	3779	26	25	35	49	31	0.00448	0.00762	0.02215	0.02710	0.08131
DUAL1	3813	10	12	14	12	12	0.00182	0.00219	0.00283	0.01026	0.00219
UBH1	66033	7	17	-	16	8	0.08430	0.08050	-	0.05857	0.15917
DUALC5	2276	10	10	18	14	12	0.00043	0.00084	0.00126	0.01239	0.00120
LISWET9	40002	-	-	-	-	62	-	-	-	-	0.75226
Q25FV47	71470	27	25	34	37	53	0.18642	0.18053	2.34299	0.16627	0.52865
PRIMALC8	5182	20	12	14	10	22	0.00247	0.00322	0.00401	0.00440	0.02708
QPCBLEND	657	16	16	17	20	17	0.00060	0.00095	0.00097	0.00530	0.00192
LOTSCHD	72	7	8	18	11	15	0.00004	0.00008	0.00011	0.00273	0.00067
QSCAGR7	585	15	15	29	23	32	0.00044	0.00084	0.00174	0.00461	0.00756
QSCRS8	4472	23	31	33	35	30	0.00722	0.01372	0.01325	0.02854	0.06120
CVXQP1_S	734	8	8	12	13	12	0.00054	0.00085	0.01330	0.00830	0.00374
QADLITTL	567	11	13	17	16	14	0.00041	0.00076	0.00140	0.00485	0.00326
CVXQP2_S	660	8	9	12	14	11	0.00045	0.00073	0.01244	0.00789	0.00310

Table 2: Iterations and solver runtimes for Maros–Mészáros problems

Problem	Size	Iterations					Solver Runtime (s)				
		QOCO	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	CLARABEL	ECOS	GUROBI	MOSEK
QRECIPE	988	18	16	19	20	15	0.00110	0.00153	0.00195	0.00703	0.00469
QSCSD1	3893	11	10	12	16	12	0.00267	0.00363	0.00896	0.01347	0.01324
GENHS28	43	0	0	9	1	5	0.00002	0.00004	0.00008	0.00576	0.00039
QSCTAP1	2325	15	10	10	17	13	0.00176	0.00185	0.00272	0.00821	0.02044
QBRANDY	2462	19	18	28	40	28	0.00330	0.00420	0.00662	0.01947	0.02632
HS52	14	0	0	7	2	8	0.00002	0.00003	0.00005	0.00333	0.00039
HS51	14	0	0	7	2	7	0.00001	0.00002	0.00004	0.00242	0.00025
CVXQP3.S	808	9	10	13	14	14	0.00050	0.00089	0.01110	0.00681	0.00338
CVXQP3.L	82481	-	10	-	14	42	-	21.86824	-	6.19038	187.95385
STADAT3	31997	17	54	28	27	22	0.02096	0.10802	0.08724	0.06329	0.07766
S268	40	5	10	15	12	15	0.00002	0.00006	0.00007	0.00266	0.00044
QSTANDAT	5030	24	20	11	47	22	0.00536	0.00792	0.04844	0.03088	0.05748
EXDATA	1137750	16	21	19	14	13	4.82029	6.01972	36.06845	1.83909	0.83910
QGROW7	3550	19	21	16	20	27	0.00310	0.00556	0.00557	0.01416	0.03009
DUAL4	3024	8	11	13	10	13	0.00134	0.00224	0.00236	0.00856	0.00247
QFORPLAN	5591	28	21	-	56	-	0.00617	0.00799	-	0.02855	-
QSCTAP3	12401	16	11	11	19	12	0.01999	0.01361	0.07613	0.03330	0.08330
QSHARE2B	828	19	15	24	21	20	0.00065	0.00092	0.00141	0.00396	0.00258
HS35MOD	12	9	11	11	15	13	0.00002	0.00003	0.00005	0.00227	0.00032
DUAL2	4796	8	10	13	11	7	0.00168	0.00263	0.00429	0.00878	0.00594
MOSARQP2	4775	8	9	27	16	13	0.00445	0.00614	0.02766	0.01492	0.01722
QPTTEST	10	5	7	9	10	10	0.00001	0.00003	0.00003	0.00225	0.00021
AUG3D	9219	0	0	13	2	6	0.00191	0.00252	0.03937	0.00711	0.04820
QSIERRA	11557	22	26	38	24	21	0.01371	0.02585	0.05065	0.03655	0.12325
LISWET5	40002	8	9	-	13	39	0.01421	0.03237	-	0.05394	0.40437
LISWET3	40002	9	16	-	13	20	0.01809	0.05045	-	0.05186	0.14603
LISWET12	40002	-	-	-	-	60	-	-	-	-	0.72638
CVXQP3.M	8231	23	11	-	18	21	0.08985	0.05016	-	0.14954	0.46277
HS118	108	11	11	9	13	10	0.00007	0.00012	0.00011	0.00229	0.00048
QSHIP12S	28344	18	15	31	24	26	0.02525	0.03026	7.90644	0.04829	0.79477
DUALC1	1998	23	11	29	15	14	0.00076	0.00074	0.00162	0.01125	0.00095
CVXQP1.L	74982	-	10	-	15	37	-	19.33309	-	4.44083	157.56906
LISWET7	40002	11	-	-	58	44	0.01843	-	-	0.26488	0.49234
CONT-100	79596	11	9	22	11	19	0.32731	0.38091	0.80000	0.08902	0.27225
QSCFXM3	10369	31	32	34	58	31	0.02114	0.02997	0.07188	0.08477	0.09320
DPKLO1	1652	0	0	13	2	4	0.00018	0.00027	0.00204	0.00283	0.00136
KSIP	18871	14	14	21	17	11	0.00666	0.00982	0.01018	0.01295	0.00380
DUALC8	4076	14	9	27	14	22	0.00111	0.00136	0.00493	0.02466	0.00201
QSHIP08L	52050	14	15	28	22	20	0.11742	0.14559	13.08404	0.08743	0.45644
AUG3DQP	13092	11	11	31	17	13	0.01592	0.02040	0.08446	0.03143	0.07889
AUG2DQP	80000	11	13	-	17	-	0.07453	0.14683	-	0.10080	-
HS76	20	6	6	10	10	7	0.00002	0.00004	0.00005	0.00300	0.00038
QSCTAP2	9371	14	10	10	18	11	0.00958	0.00923	0.03774	0.02440	0.07304
BOYD1	745507	37	-	-	30	-	0.90575	-	-	1.08528	-
QCAPRI	3147	40	31	-	33	39	0.00557	0.00889	-	0.02008	0.06086
CONT-300	653093	12	10	-	14	-	15.35224	15.73776	-	2.06562	-

Table 3: Iterations and solver runtimes for robust Kalman filter problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
ROBUST_KALMAN_FILTER_N_25_I_0	775	12	15	-	12	9	9	0.00091	0.00167	0.00801	0.00420	0.00051	0.00020
ROBUST_KALMAN_FILTER_N_25_I_1	775	10	15	-	11	9	9	0.00081	0.00170	0.00895	0.00415	0.00051	0.00020
ROBUST_KALMAN_FILTER_N_25_I_2	775	11	18	-	12	10	10	0.00087	0.00197	0.00998	0.00416	0.00056	0.00021
ROBUST_KALMAN_FILTER_N_25_I_3	775	10	17	-	12	9	9	0.00082	0.00170	0.00776	0.00398	0.00044	0.00020
ROBUST_KALMAN_FILTER_N_25_I_4	775	11	15	-	12	9	9	0.00087	0.00166	0.00933	0.00478	0.00051	0.00019
ROBUST_KALMAN_FILTER_N_25_I_5	775	10	14	-	10	8	8	0.00080	0.00160	0.00887	0.00400	0.00047	0.00018
ROBUST_KALMAN_FILTER_N_25_I_6	775	11	15	-	12	9	9	0.00086	0.00169	0.01004	0.00413	0.00052	0.00020
ROBUST_KALMAN_FILTER_N_25_I_7	775	10	17	-	11	9	9	0.00080	0.00193	0.00975	0.00389	0.00051	0.00020
ROBUST_KALMAN_FILTER_N_25_I_8	775	10	15	-	11	9	9	0.00083	0.00163	0.00885	0.00390	0.00052	0.00020
ROBUST_KALMAN_FILTER_N_25_I_9	775	9	18	-	11	9	9	0.00075	0.00176	0.00724	0.00356	0.00046	0.00018
ROBUST_KALMAN_FILTER_N_25_I_10	775	11	16	-	13	10	10	0.00075	0.00153	0.00809	0.00419	0.00048	0.00020
ROBUST_KALMAN_FILTER_N_25_I_11	775	12	19	-	12	10	10	0.00080	0.00177	0.00831	0.00378	0.00049	0.00020
ROBUST_KALMAN_FILTER_N_25_I_12	775	10	17	-	12	9	9	0.00068	0.00159	0.00728	0.00372	0.00044	0.00018
ROBUST_KALMAN_FILTER_N_25_I_13	775	8	18	-	13	8	8	0.00059	0.00175	0.00799	0.00413	0.00041	0.00016
ROBUST_KALMAN_FILTER_N_25_I_14	775	10	17	-	12	9	9	0.00068	0.00162	0.00760	0.00381	0.00045	0.00019
ROBUST_KALMAN_FILTER_N_25_I_15	775	10	16	-	11	8	8	0.00070	0.00157	0.00805	0.00351	0.00042	0.00016
ROBUST_KALMAN_FILTER_N_25_I_16	775	10	17	-	12	9	9	0.00070	0.00161	0.00762	0.00386	0.00044	0.00018
ROBUST_KALMAN_FILTER_N_25_I_17	775	9	17	-	11	9	9	0.00064	0.00180	0.01127	0.00367	0.00045	0.00018
ROBUST_KALMAN_FILTER_N_25_I_18	775	11	16	-	12	9	9	0.00080	0.00156	0.00795	0.00396	0.00048	0.00018
ROBUST_KALMAN_FILTER_N_25_I_19	775	11	19	-	11	10	10	0.00075	0.00196	0.00793	0.00370	0.00051	0.00021
ROBUST_KALMAN_FILTER_N_50_I_0	1550	13	18	-	13	12	12	0.00167	0.00415	0.01567	0.00701	0.00127	0.00053
ROBUST_KALMAN_FILTER_N_50_I_1	1550	14	19	-	14	12	12	0.00209	0.00410	0.01851	0.00789	0.00126	0.00054
ROBUST_KALMAN_FILTER_N_50_I_2	1550	11	20	-	12	10	10	0.00167	0.00441	0.01895	0.00712	0.00108	0.00044
ROBUST_KALMAN_FILTER_N_50_I_3	1550	11	19	-	12	10	10	0.00169	0.00426	0.02149	0.00714	0.00108	0.00040
ROBUST_KALMAN_FILTER_N_50_I_4	1550	11	20	-	11	10	10	0.00166	0.00426	0.01637	0.00651	0.00108	0.00046
ROBUST_KALMAN_FILTER_N_50_I_5	1550	11	19	-	11	10	10	0.00168	0.00386	0.01841	0.00680	0.00101	0.00045
ROBUST_KALMAN_FILTER_N_50_I_6	1550	12	19	-	12	11	11	0.00157	0.00363	0.01517	0.00638	0.00103	0.00044
ROBUST_KALMAN_FILTER_N_50_I_7	1550	10	17	-	10	9	9	0.00135	0.00338	0.01494	0.00546	0.00089	0.00036
ROBUST_KALMAN_FILTER_N_50_I_8	1550	11	20	-	10	10	10	0.00145	0.00374	0.01444	0.00548	0.00096	0.00041
ROBUST_KALMAN_FILTER_N_50_I_9	1550	12	24	-	12	10	10	0.00152	0.00454	0.01599	0.00647	0.00097	0.00041
ROBUST_KALMAN_FILTER_N_50_I_10	1550	11	21	-	11	9	9	0.00145	0.00406	0.01574	0.00576	0.00093	0.00037
ROBUST_KALMAN_FILTER_N_50_I_11	1550	11	19	-	12	10	10	0.00146	0.00374	0.01475	0.00612	0.00099	0.00040
ROBUST_KALMAN_FILTER_N_50_I_12	1550	12	18	-	11	11	11	0.00155	0.00402	0.01724	0.00593	0.00117	0.00050
ROBUST_KALMAN_FILTER_N_50_I_13	1550	13	21	-	14	12	12	0.00199	0.00425	0.01969	0.00778	0.00127	0.00048
ROBUST_KALMAN_FILTER_N_50_I_14	1550	13	19	-	13	12	12	0.00167	0.00421	0.01539	0.00650	0.00109	0.00053
ROBUST_KALMAN_FILTER_N_50_I_15	1550	11	19	-	12	10	10	0.00172	0.00364	0.01478	0.00624	0.00096	0.00039
ROBUST_KALMAN_FILTER_N_50_I_16	1550	12	21	-	11	10	10	0.00155	0.00401	0.01620	0.00600	0.00094	0.00044
ROBUST_KALMAN_FILTER_N_50_I_17	1550	13	21	-	12	11	11	0.00165	0.00394	0.01572	0.00610	0.00102	0.00043
ROBUST_KALMAN_FILTER_N_50_I_18	1550	11	18	-	11	10	10	0.00144	0.00345	0.01480	0.00610	0.00102	0.00040
ROBUST_KALMAN_FILTER_N_50_I_19	1550	13	19	-	13	12	12	0.00170	0.00380	0.01551	0.00675	0.00116	0.00053
ROBUST_KALMAN_FILTER_N_75_I_0	2325	13	18	-	12	11	11	0.00250	0.00548	0.02104	0.00899	0.00153	0.00067
ROBUST_KALMAN_FILTER_N_75_I_1	2325	12	19	-	11	10	10	0.00229	0.00552	0.02102	0.00838	0.00147	0.00062
ROBUST_KALMAN_FILTER_N_75_I_2	2325	11	21	-	11	10	10	0.00221	0.00629	0.02039	0.00836	0.00145	0.00062
ROBUST_KALMAN_FILTER_N_75_I_3	2325	13	21	-	14	11	11	0.00247	0.00600	0.02210	0.01016	0.00161	0.00067
ROBUST_KALMAN_FILTER_N_75_I_4	2325	13	22	-	14	12	12	0.00251	0.00646	0.02187	0.01024	0.00163	0.00073
ROBUST_KALMAN_FILTER_N_75_I_5	2325	12	23	-	13	11	11	0.00232	0.00661	0.02040	0.00925	0.00154	0.00068

Table 3: Iterations and solver runtimes for robust Kalman filter problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
ROBUST_KALMAN_FILTER_N_75_I_6	2325	12	21	-	12	10	10	0.00230	0.00634	0.02142	0.00912	0.00143	0.00063
ROBUST_KALMAN_FILTER_N_75_I_7	2325	12	24	-	12	10	10	0.00230	0.00727	0.02007	0.00882	0.00146	0.00069
ROBUST_KALMAN_FILTER_N_75_I_8	2325	12	21	-	12	10	10	0.00229	0.00603	0.01992	0.00915	0.00142	0.00062
ROBUST_KALMAN_FILTER_N_75_I_9	2325	12	20	-	11	10	10	0.00232	0.00575	0.02182	0.00839	0.00143	0.00063
ROBUST_KALMAN_FILTER_N_75_I_10	2325	14	23	-	14	12	12	0.00264	0.00648	0.02124	0.01016	0.00168	0.00072
ROBUST_KALMAN_FILTER_N_75_I_11	2325	13	21	-	12	11	11	0.00246	0.00622	0.01754	0.00891	0.00154	0.00067
ROBUST_KALMAN_FILTER_N_75_I_12	2325	12	22	-	12	11	11	0.00234	0.00624	0.02128	0.00904	0.00154	0.00069
ROBUST_KALMAN_FILTER_N_75_I_13	2325	13	21	-	12	11	11	0.00245	0.00611	0.02185	0.00887	0.00154	0.00068
ROBUST_KALMAN_FILTER_N_75_I_14	2325	13	19	-	12	11	11	0.00244	0.00558	0.02053	0.00882	0.00160	0.00065
ROBUST_KALMAN_FILTER_N_75_I_15	2325	14	22	-	15	12	12	0.00267	0.00634	0.02140	0.01082	0.00165	0.00075
ROBUST_KALMAN_FILTER_N_75_I_16	2325	13	22	-	12	11	11	0.00246	0.00630	0.01927	0.00899	0.00154	0.00069
ROBUST_KALMAN_FILTER_N_75_I_17	2325	12	21	-	12	10	10	0.00231	0.00611	0.02065	0.00906	0.00142	0.00062
ROBUST_KALMAN_FILTER_N_75_I_18	2325	13	20	-	14	11	11	0.00250	0.00578	0.02298	0.01035	0.00154	0.00069
ROBUST_KALMAN_FILTER_N_75_I_19	2325	12	20	-	13	10	10	0.00229	0.00568	0.02107	0.00992	0.00144	0.00062
ROBUST_KALMAN_FILTER_N_125_I_0	3875	13	22	-	14	11	11	0.00418	0.01056	0.03251	0.01433	0.00257	0.00117
ROBUST_KALMAN_FILTER_N_125_I_1	3875	14	23	-	13	12	12	0.00447	0.01152	0.03139	0.01300	0.00282	0.00140
ROBUST_KALMAN_FILTER_N_125_I_2	3875	13	20	-	12	11	11	0.00420	0.01122	0.03640	0.01222	0.00300	0.00117
ROBUST_KALMAN_FILTER_N_125_I_3	3875	13	21	-	13	10	10	0.00488	0.01034	0.03417	0.01317	0.00240	0.00120
ROBUST_KALMAN_FILTER_N_125_I_4	3875	13	20	-	12	10	10	0.00416	0.00991	0.03691	0.01212	0.00247	0.00106
ROBUST_KALMAN_FILTER_N_125_I_5	3875	13	26	-	14	12	12	0.00418	0.01285	0.03163	0.01427	0.00275	0.00125
ROBUST_KALMAN_FILTER_N_125_I_6	3875	14	21	-	13	12	12	0.00443	0.01017	0.03108	0.01283	0.00276	0.00126
ROBUST_KALMAN_FILTER_N_125_I_7	3875	13	21	-	14	12	12	0.00416	0.01198	0.03990	0.01367	0.00319	0.00141
ROBUST_KALMAN_FILTER_N_125_I_8	3875	11	23	-	12	10	10	0.00421	0.01136	0.03093	0.01260	0.00252	0.00119
ROBUST_KALMAN_FILTER_N_125_I_9	3875	12	23	-	12	10	10	0.00454	0.01325	0.04026	0.01384	0.00275	0.00120
ROBUST_KALMAN_FILTER_N_125_I_10	3875	15	24	-	15	12	12	0.00554	0.01366	0.04077	0.01713	0.00317	0.00142
ROBUST_KALMAN_FILTER_N_125_I_11	3875	13	24	-	13	11	11	0.00496	0.01335	0.04015	0.01551	0.00300	0.00130
ROBUST_KALMAN_FILTER_N_125_I_12	3875	13	23	-	15	11	11	0.00493	0.01106	0.03885	0.01553	0.00258	0.00115
ROBUST_KALMAN_FILTER_N_125_I_13	3875	11	21	-	11	10	10	0.00365	0.01220	0.03033	0.01155	0.00277	0.00105
ROBUST_KALMAN_FILTER_N_125_I_14	3875	14	23	-	13	11	11	0.00519	0.01256	0.03178	0.01334	0.00298	0.00129
ROBUST_KALMAN_FILTER_N_125_I_15	3875	13	21	-	13	11	11	0.00485	0.01181	0.03807	0.01472	0.00300	0.00116
ROBUST_KALMAN_FILTER_N_125_I_16	3875	12	21	-	13	11	11	0.00459	0.01058	0.03455	0.01461	0.00260	0.00129
ROBUST_KALMAN_FILTER_N_125_I_17	3875	13	23	-	13	12	12	0.00419	0.01117	0.03539	0.01349	0.00280	0.00124
ROBUST_KALMAN_FILTER_N_125_I_18	3875	13	21	-	12	11	11	0.00416	0.01051	0.03393	0.01216	0.00258	0.00119
ROBUST_KALMAN_FILTER_N_125_I_19	3875	14	21	-	13	12	12	0.00440	0.01028	0.03846	0.01306	0.00281	0.00122
ROBUST_KALMAN_FILTER_N_175_I_0	5425	13	23	-	14	11	11	0.00576	0.01609	0.04946	0.02003	0.00372	0.00182
ROBUST_KALMAN_FILTER_N_175_I_1	5425	13	24	-	14	11	11	0.00586	0.01695	0.04905	0.01895	0.00369	0.00167
ROBUST_KALMAN_FILTER_N_175_I_2	5425	13	24	-	14	11	11	0.00581	0.01677	0.04803	0.01918	0.00385	0.00162
ROBUST_KALMAN_FILTER_N_175_I_3	5425	13	22	-	13	11	11	0.00583	0.01531	0.04494	0.01763	0.00383	0.00159
ROBUST_KALMAN_FILTER_N_175_I_4	5425	13	22	-	14	11	11	0.00675	0.01538	0.04717	0.01978	0.00374	0.00166
ROBUST_KALMAN_FILTER_N_175_I_5	5425	14	26	-	14	11	11	0.00622	0.01812	0.04802	0.01980	0.00373	0.00162
ROBUST_KALMAN_FILTER_N_175_I_6	5425	15	22	-	14	12	12	0.00657	0.01523	0.04868	0.01976	0.00405	0.00197
ROBUST_KALMAN_FILTER_N_175_I_7	5425	14	25	-	14	11	11	0.00651	0.01728	0.04697	0.01966	0.00377	0.00163
ROBUST_KALMAN_FILTER_N_175_I_8	5425	14	24	-	15	11	11	0.00620	0.01666	0.04773	0.02065	0.00381	0.00183
ROBUST_KALMAN_FILTER_N_175_I_9	5425	14	24	-	15	12	12	0.00727	0.01647	0.04878	0.02087	0.00401	0.00198
ROBUST_KALMAN_FILTER_N_175_I_10	5425	13	24	-	14	11	11	0.00584	0.01975	0.04389	0.01975	0.00431	0.00169
ROBUST_KALMAN_FILTER_N_175_I_11	5425	13	24	-	15	12	12	0.00679	0.01684	0.04417	0.02087	0.00403	0.00175

Table 3: Iterations and solver runtimes for robust Kalman filter problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
ROBUST_KALMAN_FILTER_N_175_I_12	5425	14	23	-	16	12	12	0.00621	0.01603	0.04882	0.02320	0.00398	0.00170
ROBUST_KALMAN_FILTER_N_175_I_13	5425	12	26	-	14	10	10	0.00543	0.02103	0.04248	0.01993	0.00403	0.00168
ROBUST_KALMAN_FILTER_N_175_I_14	5425	13	22	-	13	10	10	0.00674	0.01584	0.05128	0.01833	0.00350	0.00145
ROBUST_KALMAN_FILTER_N_175_I_15	5425	14	26	-	15	13	13	0.00618	0.01838	0.04897	0.02098	0.00427	0.00187
ROBUST_KALMAN_FILTER_N_175_I_16	5425	12	22	-	13	11	11	0.00553	0.01566	0.04850	0.01898	0.00372	0.00162
ROBUST_KALMAN_FILTER_N_175_I_17	5425	14	25	-	15	11	11	0.00623	0.01752	0.04675	0.02127	0.00372	0.00160
ROBUST_KALMAN_FILTER_N_175_I_18	5425	13	25	-	14	12	12	0.00594	0.01770	0.04420	0.01985	0.00397	0.00170
ROBUST_KALMAN_FILTER_N_175_I_19	5425	13	24	-	15	11	11	0.00584	0.01683	0.04848	0.02111	0.00374	0.00161
ROBUST_KALMAN_FILTER_N_225_I_0	6975	13	24	-	14	11	-	0.00757	0.02219	0.05807	0.02588	0.00496	-
ROBUST_KALMAN_FILTER_N_225_I_1	6975	13	24	-	13	11	-	0.00748	0.02199	0.05756	0.02292	0.00512	-
ROBUST_KALMAN_FILTER_N_225_I_2	6975	13	26	-	15	12	-	0.00754	0.02367	0.06344	0.02702	0.00524	-
ROBUST_KALMAN_FILTER_N_225_I_3	6975	12	23	-	13	10	-	0.00708	0.02084	0.06611	0.02305	0.00457	-
ROBUST_KALMAN_FILTER_N_225_I_4	6975	13	25	-	14	11	-	0.00757	0.02393	0.06104	0.02447	0.00496	-
ROBUST_KALMAN_FILTER_N_225_I_5	6975	13	25	-	13	11	-	0.00874	0.02696	0.05785	0.02351	0.00492	-
ROBUST_KALMAN_FILTER_N_225_I_6	6975	13	26	-	15	11	-	0.00880	0.02462	0.06758	0.02683	0.00490	-
ROBUST_KALMAN_FILTER_N_225_I_7	6975	13	24	-	16	11	-	0.00755	0.02537	0.05830	0.02795	0.00567	-
ROBUST_KALMAN_FILTER_N_225_I_8	6975	13	24	-	13	11	-	0.00877	0.02552	0.06356	0.02308	0.00569	-
ROBUST_KALMAN_FILTER_N_225_I_9	6975	12	25	-	14	11	-	0.00816	0.02334	0.05694	0.02564	0.00492	-
ROBUST_KALMAN_FILTER_N_225_I_10	6975	14	26	-	15	12	-	0.00796	0.02748	0.06755	0.02696	0.00605	-
ROBUST_KALMAN_FILTER_N_225_I_11	6975	14	25	-	15	12	-	0.00928	0.02284	0.06145	0.02675	0.00528	-
ROBUST_KALMAN_FILTER_N_225_I_12	6975	12	25	-	12	10	-	0.00697	0.02360	0.06125	0.02157	0.00459	-
ROBUST_KALMAN_FILTER_N_225_I_13	6975	13	24	-	14	11	-	0.00759	0.02204	0.08955	0.02526	0.00563	-
ROBUST_KALMAN_FILTER_N_225_I_14	6975	13	24	-	15	11	-	0.00754	0.02213	0.05645	0.02666	0.00491	-
ROBUST_KALMAN_FILTER_N_225_I_15	6975	14	24	-	13	10	-	0.00797	0.02213	0.06271	0.02311	0.00459	-
ROBUST_KALMAN_FILTER_N_225_I_16	6975	13	24	-	13	11	-	0.00750	0.02191	0.06950	0.02277	0.00486	-
ROBUST_KALMAN_FILTER_N_225_I_17	6975	14	23	-	13	11	-	0.00794	0.02353	0.06576	0.02390	0.00566	-
ROBUST_KALMAN_FILTER_N_225_I_18	6975	13	25	-	15	11	-	0.00877	0.02643	0.06683	0.03059	0.00569	-
ROBUST_KALMAN_FILTER_N_225_I_19	6975	13	25	-	15	12	-	0.00878	0.02639	0.06711	0.03064	0.00606	-
ROBUST_KALMAN_FILTER_N_300_I_0	9300	14	23	-	15	11	-	0.01230	0.02920	0.07393	0.03522	0.00675	-
ROBUST_KALMAN_FILTER_N_300_I_1	9300	13	25	-	16	11	-	0.00996	0.03078	0.07924	0.03700	0.00673	-
ROBUST_KALMAN_FILTER_N_300_I_2	9300	14	25	-	13	10	-	0.01053	0.03125	0.08683	0.03018	0.00626	-
ROBUST_KALMAN_FILTER_N_300_I_3	9300	13	25	-	14	11	-	0.01003	0.03218	0.08651	0.03377	0.00679	-
ROBUST_KALMAN_FILTER_N_300_I_4	9300	13	23	-	13	11	-	0.01006	0.02924	0.08816	0.03026	0.00672	-
ROBUST_KALMAN_FILTER_N_300_I_5	9300	13	25	-	15	11	-	0.01000	0.03138	0.08341	0.03518	0.00674	-
ROBUST_KALMAN_FILTER_N_300_I_6	9300	13	24	-	14	11	-	0.00996	0.03027	0.08081	0.03316	0.00685	-
ROBUST_KALMAN_FILTER_N_300_I_7	9300	14	25	-	16	11	-	0.01068	0.03071	0.08734	0.03687	0.00688	-
ROBUST_KALMAN_FILTER_N_300_I_8	9300	13	25	-	15	10	-	0.00999	0.03179	0.08249	0.03536	0.00639	-
ROBUST_KALMAN_FILTER_N_300_I_9	9300	14	26	-	13	11	-	0.01063	0.03282	0.08182	0.03022	0.00677	-
ROBUST_KALMAN_FILTER_N_300_I_10	9300	13	24	-	13	11	-	0.01002	0.03008	0.08660	0.03015	0.00678	-
ROBUST_KALMAN_FILTER_N_300_I_11	9300	13	25	-	16	11	-	0.01003	0.03607	0.08646	0.03671	0.00776	-
ROBUST_KALMAN_FILTER_N_300_I_12	9300	14	26	-	15	11	-	0.01245	0.03253	0.07619	0.04085	0.00691	-
ROBUST_KALMAN_FILTER_N_300_I_13	9300	13	25	-	15	11	-	0.01001	0.03161	0.08319	0.03523	0.00678	-
ROBUST_KALMAN_FILTER_N_300_I_14	9300	14	25	-	15	11	-	0.01066	0.03163	0.07509	0.03532	0.00786	-
ROBUST_KALMAN_FILTER_N_300_I_15	9300	13	27	-	15	11	-	0.00997	0.03292	0.08569	0.03499	0.00687	-
ROBUST_KALMAN_FILTER_N_300_I_16	9300	16	26	-	16	12	-	0.01194	0.03260	0.08105	0.03728	0.00725	-
ROBUST_KALMAN_FILTER_N_300_I_17	9300	13	23	-	13	10	-	0.00987	0.02901	0.09474	0.03012	0.00634	-

Table 3: Iterations and solver runtimes for robust Kalman filter problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
ROBUST_KALMAN_FILTER_N_300_I_18	9300	13	24	-	13	10	-	0.00986	0.03074	0.08284	0.03015	0.00637	-
ROBUST_KALMAN_FILTER_N_300_I_19	9300	13	25	-	19	11	-	0.01003	0.03526	0.08672	0.05072	0.00690	-
ROBUST_KALMAN_FILTER_N_375_I_0	11625	13	26	-	13	11	-	0.01465	0.04878	0.10385	0.03771	0.01025	-
ROBUST_KALMAN_FILTER_N_375_I_1	11625	13	26	-	16	10	-	0.01451	0.04146	0.10711	0.04625	0.00820	-
ROBUST_KALMAN_FILTER_N_375_I_2	11625	12	26	-	14	11	-	0.01180	0.04283	0.10340	0.04159	0.00882	-
ROBUST_KALMAN_FILTER_N_375_I_3	11625	13	26	-	14	11	-	0.01253	0.04203	0.09593	0.03980	0.00894	-
ROBUST_KALMAN_FILTER_N_375_I_4	11625	13	24	-	16	11	-	0.01258	0.03876	0.09791	0.04590	0.00876	-
ROBUST_KALMAN_FILTER_N_375_I_5	11625	12	23	-	13	10	-	0.01174	0.03728	0.11823	0.03755	0.00820	-
ROBUST_KALMAN_FILTER_N_375_I_6	11625	14	25	-	16	12	-	0.01331	0.04077	0.13839	0.05476	0.00940	-
ROBUST_KALMAN_FILTER_N_375_I_7	11625	14	25	-	15	11	-	0.01339	0.04071	0.11648	0.04368	0.00874	-
ROBUST_KALMAN_FILTER_N_375_I_8	11625	13	26	-	16	11	-	0.01259	0.04182	0.10793	0.04589	0.01014	-
ROBUST_KALMAN_FILTER_N_375_I_9	11625	14	27	-	15	11	-	0.01346	0.04374	0.13064	0.04391	0.00891	-
ROBUST_KALMAN_FILTER_N_375_I_10	11625	14	25	-	14	11	-	0.01348	0.04713	0.11812	0.04127	0.01033	-
ROBUST_KALMAN_FILTER_N_375_I_11	11625	12	26	-	15	10	-	0.01367	0.04176	0.10039	0.04392	0.00828	-
ROBUST_KALMAN_FILTER_N_375_I_12	11625	13	24	-	16	11	-	0.01250	0.04469	0.11300	0.04598	0.01010	-
ROBUST_KALMAN_FILTER_N_375_I_13	11625	13	27	-	14	11	-	0.01467	0.04935	0.11771	0.04580	0.01018	-
ROBUST_KALMAN_FILTER_N_375_I_14	11625	14	26	-	15	11	-	0.01557	0.04223	0.13709	0.04383	0.00888	-
ROBUST_KALMAN_FILTER_N_375_I_15	11625	14	26	-	15	11	-	0.01346	0.04199	0.10943	0.04412	0.00887	-
ROBUST_KALMAN_FILTER_N_375_I_16	11625	13	27	-	16	12	-	0.01264	0.04334	0.10027	0.04589	0.01077	-
ROBUST_KALMAN_FILTER_N_375_I_17	11625	13	26	-	16	11	-	0.01258	0.04814	0.10561	0.04790	0.01013	-
ROBUST_KALMAN_FILTER_N_375_I_18	11625	13	24	-	15	11	-	0.01459	0.03878	0.11506	0.05058	0.01005	-
ROBUST_KALMAN_FILTER_N_375_I_19	11625	14	25	-	16	11	-	0.01344	0.04601	0.11903	0.04579	0.01016	-
ROBUST_KALMAN_FILTER_N_450_I_0	13950	13	26	-	16	11	-	0.01755	0.05943	0.13787	0.06379	0.01251	-
ROBUST_KALMAN_FILTER_N_450_I_1	13950	13	27	-	15	11	-	0.01761	0.06068	0.13805	0.06043	0.01265	-
ROBUST_KALMAN_FILTER_N_450_I_2	13950	13	26	-	15	11	-	0.01755	0.05289	0.15038	0.06417	0.01084	-
ROBUST_KALMAN_FILTER_N_450_I_3	13950	13	28	-	15	10	-	0.01499	0.06268	0.13392	0.05238	0.01051	-
ROBUST_KALMAN_FILTER_N_450_I_4	13950	13	31	-	15	11	-	0.01753	0.05969	0.12347	0.06045	0.01090	-
ROBUST_KALMAN_FILTER_N_450_I_5	13950	14	26	-	15	11	-	0.01596	0.05177	0.13747	0.05258	0.01252	-
ROBUST_KALMAN_FILTER_N_450_I_6	13950	14	29	-	16	11	-	0.01600	0.05609	0.13698	0.05515	0.01088	-
ROBUST_KALMAN_FILTER_N_450_I_7	13950	13	27	-	16	11	-	0.01501	0.05145	0.14600	0.05486	0.01091	-
ROBUST_KALMAN_FILTER_N_450_I_8	13950	14	28	-	16	12	-	0.01609	0.05432	0.13131	0.05507	0.01152	-
ROBUST_KALMAN_FILTER_N_450_I_9	13950	13	29	-	14	10	-	0.01497	0.05663	0.14099	0.04749	0.01039	-
ROBUST_KALMAN_FILTER_N_450_I_10	13950	14	27	-	15	10	-	0.01595	0.06025	0.14367	0.05227	0.01175	-
ROBUST_KALMAN_FILTER_N_450_I_11	13950	12	25	-	15	10	-	0.01635	0.05020	0.13812	0.06005	0.01029	-
ROBUST_KALMAN_FILTER_N_450_I_12	13950	15	26	-	16	12	-	0.01697	0.05084	0.13032	0.05577	0.01160	-
ROBUST_KALMAN_FILTER_N_450_I_13	13950	13	26	-	16	11	-	0.01504	0.05123	0.12504	0.05499	0.01091	-
ROBUST_KALMAN_FILTER_N_450_I_14	13950	13	26	-	14	10	-	0.01502	0.05176	0.13912	0.04784	0.01034	-
ROBUST_KALMAN_FILTER_N_450_I_15	13950	14	28	-	16	10	-	0.01606	0.05606	0.12310	0.05692	0.01182	-
ROBUST_KALMAN_FILTER_N_450_I_16	13950	14	29	-	16	11	-	0.01590	0.05792	0.13368	0.05491	0.01091	-
ROBUST_KALMAN_FILTER_N_450_I_17	13950	13	27	-	14	11	-	0.01747	0.05376	0.15784	0.05465	0.01091	-
ROBUST_KALMAN_FILTER_N_450_I_18	13950	14	26	-	15	13	-	0.01610	0.05996	0.13933	0.05233	0.01228	-
ROBUST_KALMAN_FILTER_N_450_I_19	13950	13	25	-	17	11	-	0.01744	0.04958	0.14799	0.06067	0.01084	-
ROBUST_KALMAN_FILTER_N_500_I_0	15500	14	25	-	14	11	-	0.01781	0.05705	0.18759	0.05262	0.01240	-
ROBUST_KALMAN_FILTER_N_500_I_1	15500	13	25	-	16	11	-	0.01664	0.05575	0.14857	0.06107	0.01249	-
ROBUST_KALMAN_FILTER_N_500_I_2	15500	13	25	-	15	10	-	0.01664	0.05711	0.15082	0.05883	0.01156	-
ROBUST_KALMAN_FILTER_N_500_I_3	15500	13	26	-	14	11	-	0.01672	0.06066	0.16043	0.05296	0.01227	-

Table 3: Iterations and solver runtimes for robust Kalman filter problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
ROBUST_KALMAN_FILTER_N_500_I_4	15500	13	29	-	16	11	-	0.01941	0.07490	0.15324	0.06134	0.01428	-
ROBUST_KALMAN_FILTER_N_500_I_5	15500	13	25	-	15	10	-	0.01652	0.06453	0.14032	0.05833	0.01341	-
ROBUST_KALMAN_FILTER_N_500_I_6	15500	14	29	-	16	11	-	0.02072	0.06508	0.17977	0.06136	0.01437	-
ROBUST_KALMAN_FILTER_N_500_I_7	15500	13	25	-	14	11	-	0.01673	0.06524	0.16417	0.05285	0.01429	-
ROBUST_KALMAN_FILTER_N_500_I_8	15500	14	27	-	16	11	-	0.02074	0.07022	0.17136	0.06185	0.01437	-
ROBUST_KALMAN_FILTER_N_500_I_9	15500	13	30	-	16	10	-	0.01941	0.06671	0.17077	0.06119	0.01201	-
ROBUST_KALMAN_FILTER_N_500_I_10	15500	14	26	-	15	11	-	0.01779	0.05815	0.14614	0.05873	0.01234	-
ROBUST_KALMAN_FILTER_N_500_I_11	15500	13	28	-	15	11	-	0.01657	0.06165	0.14662	0.05840	0.01236	-
ROBUST_KALMAN_FILTER_N_500_I_12	15500	13	26	-	16	10	-	0.01666	0.05863	0.15932	0.06092	0.01163	-
ROBUST_KALMAN_FILTER_N_500_I_13	15500	12	25	-	15	10	-	0.01554	0.05651	0.14555	0.05786	0.01232	-
ROBUST_KALMAN_FILTER_N_500_I_14	15500	13	26	-	16	11	-	0.01957	0.05692	0.16304	0.06113	0.01237	-
ROBUST_KALMAN_FILTER_N_500_I_15	15500	14	27	-	16	11	-	0.01773	0.05933	0.19075	0.06103	0.01254	-
ROBUST_KALMAN_FILTER_N_500_I_16	15500	13	26	-	16	11	-	0.01670	0.05790	0.16259	0.06289	0.01237	-
ROBUST_KALMAN_FILTER_N_500_I_17	15500	13	27	-	16	11	-	0.01671	0.06048	0.15740	0.06095	0.01250	-
ROBUST_KALMAN_FILTER_N_500_I_18	15500	13	26	-	15	11	-	0.01664	0.05763	0.16208	0.05853	0.01254	-
ROBUST_KALMAN_FILTER_N_500_I_19	15500	13	28	-	16	11	-	0.01677	0.07028	0.16353	0.06141	0.01430	-

Table 4: Iterations and solver runtimes for lossless convexification problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
LCVX_N_15_I_0	644	8	7	-	7	9	9	0.00084	0.00067	0.00505	0.00198	0.00055	0.00026
LCVX_N_15_I_1	644	8	7	-	7	9	9	0.00082	0.00065	0.00515	0.00193	0.00056	0.00024
LCVX_N_15_I_2	644	10	7	-	7	10	10	0.00097	0.00068	0.00535	0.00194	0.00060	0.00028
LCVX_N_15_I_3	644	8	7	-	8	10	10	0.00085	0.00064	0.00505	0.00240	0.00060	0.00028
LCVX_N_15_I_4	644	9	7	-	7	9	9	0.00090	0.00059	0.00443	0.00195	0.00047	0.00023
LCVX_N_15_I_5	644	10	7	-	7	9	9	0.00082	0.00059	0.00432	0.00162	0.00051	0.00023
LCVX_N_15_I_6	644	7	7	-	7	10	10	0.00065	0.00058	0.00440	0.00162	0.00051	0.00026
LCVX_N_15_I_7	644	9	7	-	7	10	10	0.00076	0.00058	0.00454	0.00161	0.00051	0.00026
LCVX_N_15_I_8	644	8	7	-	7	9	9	0.00071	0.00058	0.00430	0.00168	0.00047	0.00023
LCVX_N_15_I_9	644	8	7	-	8	10	10	0.00072	0.00056	0.00457	0.00215	0.00051	0.00025
LCVX_N_15_I_10	644	8	7	-	7	9	9	0.00071	0.00059	0.00456	0.00173	0.00048	0.00023
LCVX_N_15_I_11	644	7	7	-	7	9	9	0.00065	0.00058	0.00430	0.00163	0.00047	0.00023
LCVX_N_15_I_12	644	9	7	-	8	10	10	0.00083	0.00055	0.00444	0.00195	0.00051	0.00025
LCVX_N_15_I_13	644	7	7	-	7	9	9	0.00064	0.00058	0.00446	0.00163	0.00047	0.00023
LCVX_N_15_I_14	644	8	7	-	7	10	10	0.00071	0.00056	0.00443	0.00174	0.00053	0.00026
LCVX_N_15_I_15	644	8	7	-	7	10	10	0.00077	0.00056	0.00445	0.00163	0.00051	0.00025
LCVX_N_15_I_16	644	7	7	-	7	9	9	0.00065	0.00058	0.00448	0.00160	0.00047	0.00023
LCVX_N_15_I_17	644	8	7	-	7	10	10	0.00075	0.00059	0.00459	0.00163	0.00052	0.00025
LCVX_N_15_I_18	644	8	7	-	7	9	9	0.00071	0.00059	0.00444	0.00161	0.00048	0.00023
LCVX_N_15_I_19	644	9	7	-	8	10	10	0.00078	0.00059	0.00461	0.00197	0.00051	0.00025
LCVX_N_50_I_0	2114	9	8	-	10	10	10	0.00278	0.00205	0.01024	0.00657	0.00166	0.00093
LCVX_N_50_I_1	2114	9	7	-	11	10	10	0.00261	0.00185	0.01066	0.00742	0.00167	0.00094
LCVX_N_50_I_2	2114	9	7	-	10	10	10	0.00260	0.00212	0.01193	0.00662	0.00192	0.00101
LCVX_N_50_I_3	2114	10	9	-	13	10	10	0.00339	0.00269	0.01270	0.00951	0.00194	0.00100
LCVX_N_50_I_4	2114	8	7	-	10	10	10	0.00281	0.00209	0.01227	0.00737	0.00191	0.00103
LCVX_N_50_I_5	2114	8	7	-	11	10	10	0.00244	0.00185	0.01040	0.00765	0.00166	0.00092
LCVX_N_50_I_6	2114	9	7	-	11	10	10	0.00259	0.00182	0.01134	0.00764	0.00165	0.00093
LCVX_N_50_I_7	2114	8	8	-	10	10	10	0.00239	0.00218	0.01072	0.00672	0.00167	0.00099
LCVX_N_50_I_8	2114	10	8	-	10	10	10	0.00274	0.00213	0.01097	0.00670	0.00165	0.00090
LCVX_N_50_I_9	2114	10	8	-	10	10	10	0.00277	0.00214	0.01102	0.00672	0.00167	0.00087
LCVX_N_50_I_10	2114	10	8	-	11	10	10	0.00278	0.00212	0.01095	0.00784	0.00168	0.00089
LCVX_N_50_I_11	2114	10	8	-	11	10	10	0.00291	0.00203	0.01069	0.00713	0.00166	0.00092
LCVX_N_50_I_12	2114	9	7	-	10	10	10	0.00257	0.00183	0.01087	0.00660	0.00166	0.00092
LCVX_N_50_I_13	2114	9	8	-	11	10	10	0.00272	0.00204	0.01022	0.00728	0.00167	0.00100
LCVX_N_50_I_14	2114	10	7	-	10	10	10	0.00289	0.00210	0.01576	0.00823	0.00189	0.00102
LCVX_N_50_I_15	2114	9	8	-	11	10	10	0.00300	0.00207	0.01070	0.00796	0.00167	0.00092
LCVX_N_50_I_16	2114	9	7	-	10	10	10	0.00255	0.00176	0.01094	0.00739	0.00167	0.00091
LCVX_N_50_I_17	2114	10	9	-	11	10	10	0.00290	0.00269	0.01292	0.00770	0.00192	0.00101
LCVX_N_50_I_18	2114	9	8	-	11	10	10	0.00310	0.00238	0.01204	0.00853	0.00192	0.00102
LCVX_N_50_I_19	2114	12	9	-	12	10	10	0.00382	0.00267	0.01244	0.00827	0.00191	0.00101
LCVX_N_75_I_0	3164	10	8	-	10	10	10	0.00483	0.00367	0.01726	0.01255	0.00281	0.00157
LCVX_N_75_I_1	3164	10	9	-	12	10	10	0.00503	0.00395	0.01880	0.01296	0.00284	0.00157
LCVX_N_75_I_2	3164	10	8	-	11	10	10	0.00520	0.00350	0.01749	0.01170	0.00284	0.00160
LCVX_N_75_I_3	3164	8	8	-	10	10	10	0.00420	0.00326	0.01437	0.01028	0.00242	0.00139
LCVX_N_75_I_4	3164	10	9	-	12	10	10	0.00426	0.00398	0.01629	0.01286	0.00282	0.00159
LCVX_N_75_I_5	3164	9	8	-	10	10	10	0.00456	0.00373	0.01735	0.01230	0.00288	0.00159

Table 4: Iterations and solver runtimes for lossless convexification problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
LCVX_N_75_I_6	3164	8	8	-	10	10	10	0.00421	0.00314	0.01460	0.01023	0.00243	0.00137
LCVX_N_75_I_7	3164	11	9	-	11	10	10	0.00464	0.00343	0.01502	0.01178	0.00242	0.00141
LCVX_N_75_I_8	3164	10	8	-	11	10	10	0.00437	0.00302	0.01486	0.01073	0.00244	0.00142
LCVX_N_75_I_9	3164	11	9	-	11	10	10	0.00460	0.00341	0.01494	0.01078	0.00244	0.00139
LCVX_N_75_I_10	3164	10	8	-	10	10	10	0.00404	0.00311	0.01478	0.01152	0.00243	0.00141
LCVX_N_75_I_11	3164	8	8	-	10	10	10	0.00363	0.00314	0.01447	0.01010	0.00244	0.00141
LCVX_N_75_I_12	3164	11	9	-	11	10	10	0.00470	0.00344	0.01473	0.01054	0.00243	0.00142
LCVX_N_75_I_13	3164	9	8	-	11	10	10	0.00410	0.00300	0.01479	0.01059	0.00244	0.00158
LCVX_N_75_I_14	3164	10	9	-	11	10	10	0.00442	0.00348	0.01499	0.01232	0.00246	0.00144
LCVX_N_75_I_15	3164	10	9	-	11	10	10	0.00438	0.00346	0.01438	0.01071	0.00243	0.00140
LCVX_N_75_I_16	3164	10	8	-	11	10	10	0.00440	0.00300	0.01516	0.01072	0.00247	0.00145
LCVX_N_75_I_17	3164	11	9	-	11	10	10	0.00467	0.00345	0.01504	0.01073	0.00247	0.00142
LCVX_N_75_I_18	3164	10	8	-	11	10	10	0.00407	0.00326	0.01829	0.01182	0.00244	0.00140
LCVX_N_75_I_19	3164	9	8	-	10	10	10	0.00383	0.00301	0.01468	0.01010	0.00243	0.00145
LCVX_N_100_I_0	4214	9	8	-	10	10	10	0.00531	0.00413	0.01846	0.01415	0.00332	0.00202
LCVX_N_100_I_1	4214	10	9	-	11	10	10	0.00587	0.00528	0.02145	0.01364	0.00380	0.00219
LCVX_N_100_I_2	4214	12	9	-	12	11	11	0.00772	0.00505	0.02538	0.01594	0.00419	0.00245
LCVX_N_100_I_3	4214	10	9	-	11	10	10	0.00713	0.00530	0.02058	0.01506	0.00382	0.00222
LCVX_N_100_I_4	4214	8	8	-	10	10	10	0.00522	0.00469	0.02155	0.01427	0.00382	0.00224
LCVX_N_100_I_5	4214	11	9	-	11	10	10	0.00756	0.00530	0.02170	0.01503	0.00379	0.00221
LCVX_N_100_I_6	4214	10	9	-	11	10	10	0.00692	0.00461	0.01840	0.01704	0.00329	0.00188
LCVX_N_100_I_7	4214	11	9	-	12	11	11	0.00616	0.00435	0.01749	0.01617	0.00362	0.00216
LCVX_N_100_I_8	4214	9	8	-	10	10	10	0.00523	0.00412	0.01727	0.01262	0.00332	0.00196
LCVX_N_100_I_9	4214	9	8	-	10	10	10	0.00474	0.00404	0.01726	0.01276	0.00331	0.00191
LCVX_N_100_I_10	4214	10	8	-	11	10	10	0.00560	0.00395	0.01779	0.01347	0.00331	0.00189
LCVX_N_100_I_11	4214	10	8	-	11	10	10	0.00574	0.00388	0.01737	0.01600	0.00331	0.00190
LCVX_N_100_I_12	4214	10	9	-	12	10	10	0.00592	0.00456	0.01834	0.01565	0.00331	0.00194
LCVX_N_100_I_13	4214	10	9	-	12	11	11	0.00578	0.00507	0.01823	0.01437	0.00418	0.00242
LCVX_N_100_I_14	4214	11	9	-	12	11	11	0.00722	0.00435	0.02542	0.01744	0.00388	0.00205
LCVX_N_100_I_15	4214	10	9	-	11	10	10	0.00565	0.00536	0.02161	0.01337	0.00387	0.00223
LCVX_N_100_I_16	4214	10	8	-	10	10	10	0.00666	0.00479	0.02177	0.01598	0.00380	0.00220
LCVX_N_100_I_17	4214	12	9	-	12	10	10	0.00783	0.00533	0.02160	0.01689	0.00381	0.00219
LCVX_N_100_I_18	4214	10	8	-	10	10	10	0.00668	0.00410	0.01870	0.01297	0.00334	0.00221
LCVX_N_100_I_19	4214	11	9	-	11	10	10	0.00760	0.00529	0.02164	0.01657	0.00380	0.00222
LCVX_N_125_I_0	5264	11	9	-	12	11	11	0.00889	0.00639	0.03142	0.02272	0.00506	0.00297
LCVX_N_125_I_1	5264	11	9	-	11	10	10	0.00876	0.00562	0.03142	0.01875	0.00406	0.00235
LCVX_N_125_I_2	5264	10	9	-	12	10	10	0.00742	0.00549	0.02604	0.02137	0.00407	0.00233
LCVX_N_125_I_3	5264	11	9	-	11	10	10	0.00759	0.00566	0.02148	0.01894	0.00401	0.00237
LCVX_N_125_I_4	5264	9	8	-	10	10	10	0.00647	0.00494	0.02034	0.01619	0.00407	0.00232
LCVX_N_125_I_5	5264	11	9	-	12	10	10	0.00770	0.00540	0.02805	0.01804	0.00406	0.00233
LCVX_N_125_I_6	5264	11	9	-	11	10	10	0.00745	0.00566	0.02949	0.01920	0.00406	0.00234
LCVX_N_125_I_7	5264	11	8	-	11	10	10	0.00727	0.00513	0.02713	0.01875	0.00404	0.00275
LCVX_N_125_I_8	5264	9	8	-	10	10	10	0.00589	0.00582	0.02469	0.01816	0.00489	0.00276
LCVX_N_125_I_9	5264	10	9	-	11	10	10	0.00749	0.00572	0.02529	0.01747	0.00409	0.00272
LCVX_N_125_I_10	5264	9	8	-	11	10	10	0.00597	0.00513	0.01997	0.02011	0.00405	0.00234
LCVX_N_125_I_11	5264	8	8	-	11	10	10	0.00561	0.00499	0.02039	0.01770	0.00411	0.00237

Table 4: Iterations and solver runtimes for lossless convexification problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
LCVX_N_125_L12	5264	9	8	-	11	10	10	0.00714	0.00491	0.02246	0.01729	0.00410	0.00233
LCVX_N_125_L13	5264	11	8	-	10	10	10	0.00701	0.00595	0.03027	0.01666	0.00471	0.00275
LCVX_N_125_L14	5264	9	8	-	10	10	10	0.00709	0.00590	0.03029	0.01800	0.00471	0.00278
LCVX_N_125_L15	5264	11	9	-	12	10	10	0.00911	0.00544	0.02933	0.02052	0.00405	0.00234
LCVX_N_125_L16	5264	11	9	-	11	10	10	0.00753	0.00575	0.02619	0.01939	0.00402	0.00245
LCVX_N_125_L17	5264	11	9	-	12	11	11	0.00774	0.00546	0.02632	0.02054	0.00441	0.00256
LCVX_N_125_L18	5264	11	9	-	12	10	10	0.00783	0.00549	0.02611	0.02030	0.00404	0.00274
LCVX_N_125_L19	5264	11	9	-	11	10	10	0.00806	0.00575	0.02910	0.01742	0.00408	0.00232
LCVX_N_150_L0	6314	10	8	-	11	10	-	0.00786	0.00612	0.02926	0.02194	0.00500	-
LCVX_N_150_L1	6314	11	9	-	12	11	-	0.00957	0.00654	0.03172	0.02258	0.00538	-
LCVX_N_150_L2	6314	9	8	-	11	10	-	0.00803	0.00585	0.02330	0.02031	0.00490	-
LCVX_N_150_L3	6314	10	9	-	12	10	-	0.00922	0.00777	0.02501	0.02377	0.00494	-
LCVX_N_150_L4	6314	9	8	-	11	10	-	0.00948	0.00680	0.02824	0.02464	0.00571	-
LCVX_N_150_L5	6314	11	10	-	12	11	-	0.00982	0.00724	0.02544	0.02362	0.00539	-
LCVX_N_150_L6	6314	8	8	-	11	10	-	0.00673	0.00608	0.02354	0.02240	0.00495	-
LCVX_N_150_L7	6314	9	8	-	11	10	-	0.00718	0.00586	0.02917	0.02252	0.00494	-
LCVX_N_150_L8	6314	11	9	-	11	10	-	0.00915	0.00702	0.03529	0.02027	0.00497	-
LCVX_N_150_L9	6314	12	11	-	12	11	-	0.01013	0.00809	0.03704	0.02204	0.00533	-
LCVX_N_150_L10	6314	11	9	-	11	10	-	0.00926	0.00695	0.03020	0.01989	0.00495	-
LCVX_N_150_L11	6314	10	8	-	11	10	-	0.00868	0.00591	0.03679	0.02231	0.00496	-
LCVX_N_150_L12	6314	10	8	-	11	10	-	0.00866	0.00586	0.02538	0.02202	0.00493	-
LCVX_N_150_L13	6314	12	9	-	13	11	-	0.01014	0.00660	0.02530	0.02384	0.00540	-
LCVX_N_150_L14	6314	9	8	-	11	10	-	0.00776	0.00591	0.02464	0.02160	0.00496	-
LCVX_N_150_L15	6314	10	8	-	11	10	-	0.00869	0.00580	0.03040	0.02198	0.00492	-
LCVX_N_150_L16	6314	11	9	-	12	10	-	0.00948	0.00658	0.03178	0.02267	0.00492	-
LCVX_N_150_L17	6314	9	8	-	11	10	-	0.00766	0.00680	0.03032	0.02280	0.00571	-
LCVX_N_150_L18	6314	9	8	-	11	10	-	0.00852	0.00605	0.02975	0.02419	0.00494	-
LCVX_N_150_L19	6314	11	9	-	11	10	-	0.00920	0.00709	0.03412	0.01958	0.00491	-
LCVX_N_200_L0	8414	10	9	-	12	11	-	0.01143	0.00892	0.03671	0.02733	0.00717	-
LCVX_N_200_L1	8414	10	9	-	12	11	-	0.01170	0.00886	0.03590	0.02720	0.00720	-
LCVX_N_200_L2	8414	9	9	-	12	11	-	0.01059	0.00905	0.03362	0.02715	0.00720	-
LCVX_N_200_L3	8414	12	9	-	12	11	-	0.01327	0.00889	0.04336	0.02782	0.00710	-
LCVX_N_200_L4	8414	12	9	-	12	11	-	0.01386	0.00874	0.03456	0.03067	0.00713	-
LCVX_N_200_L5	8414	11	9	-	11	10	-	0.01195	0.00924	0.04326	0.02603	0.00664	-
LCVX_N_200_L6	8414	11	8	-	11	10	-	0.01121	0.00793	0.03630	0.02847	0.00658	-
LCVX_N_200_L7	8414	11	9	-	12	11	-	0.01286	0.00881	0.05245	0.03026	0.00714	-
LCVX_N_200_L8	8414	10	8	-	12	10	-	0.01117	0.00798	0.03576	0.03125	0.00659	-
LCVX_N_200_L9	8414	9	8	-	11	10	-	0.00971	0.00796	0.03288	0.02907	0.00663	-
LCVX_N_200_L10	8414	11	12	-	12	11	-	0.01259	0.01169	0.04709	0.02880	0.00715	-
LCVX_N_200_L11	8414	9	8	-	13	10	-	0.01061	0.00797	0.03540	0.02946	0.00659	-
LCVX_N_200_L12	8414	10	8	-	11	10	-	0.01031	0.00781	0.03495	0.02987	0.00659	-
LCVX_N_200_L13	8414	10	8	-	11	10	-	0.01042	0.00799	0.03464	0.02826	0.00668	-
LCVX_N_200_L14	8414	11	11	-	12	11	-	0.01269	0.01097	0.04928	0.03008	0.00718	-
LCVX_N_200_L15	8414	11	9	-	12	10	-	0.01204	0.00929	0.04334	0.03131	0.00658	-
LCVX_N_200_L16	8414	9	8	-	11	10	-	0.00971	0.00813	0.04864	0.02568	0.00668	-
LCVX_N_200_L17	8414	12	9	-	12	11	-	0.01312	0.00888	0.04630	0.03020	0.00717	-

Table 4: Iterations and solver runtimes for lossless convexification problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
LCVX_N_200_L18	8414	9	8	-	11	10	-	0.00966	0.00914	0.03348	0.02590	0.00764	-
LCVX_N_200_L19	8414	12	10	-	12	11	-	0.01595	0.01134	0.04035	0.03376	0.00833	-
LCVX_N_250_L0	10514	10	9	-	12	11	-	0.01613	0.01238	0.04357	0.03788	0.01035	-
LCVX_N_250_L1	10514	11	10	-	12	11	-	0.01734	0.01417	0.04455	0.03854	0.01039	-
LCVX_N_250_L2	10514	11	9	-	12	11	-	0.01491	0.01096	0.06412	0.03377	0.00894	-
LCVX_N_250_L3	10514	9	8	-	11	10	-	0.01209	0.00976	0.04371	0.03196	0.00824	-
LCVX_N_250_L4	10514	10	8	-	12	10	-	0.01297	0.00965	0.05525	0.03753	0.00829	-
LCVX_N_250_L5	10514	11	8	-	11	10	-	0.01413	0.00983	0.05159	0.03588	0.00827	-
LCVX_N_250_L6	10514	9	8	-	12	10	-	0.01215	0.00974	0.04114	0.03851	0.00829	-
LCVX_N_250_L7	10514	11	8	-	11	10	-	0.01415	0.00984	0.07292	0.03271	0.00830	-
LCVX_N_250_L8	10514	12	12	-	12	11	-	0.01583	0.01423	0.07551	0.03456	0.00900	-
LCVX_N_250_L9	10514	9	8	-	11	10	-	0.01214	0.01133	0.04417	0.03677	0.00958	-
LCVX_N_250_L10	10514	12	9	-	11	10	-	0.01802	0.01307	0.06299	0.03559	0.00956	-
LCVX_N_250_L11	10514	11	9	-	11	10	-	0.01673	0.01162	0.06861	0.03580	0.00821	-
LCVX_N_250_L12	10514	12	9	-	13	11	-	0.01621	0.01095	0.06591	0.03763	0.00890	-
LCVX_N_250_L13	10514	11	8	-	11	10	-	0.01414	0.01006	0.06107	0.03165	0.00960	-
LCVX_N_250_L14	10514	10	9	-	12	11	-	0.01417	0.01097	0.05955	0.03464	0.00902	-
LCVX_N_250_L15	10514	9	8	-	11	10	-	0.01203	0.00981	0.06903	0.03250	0.00830	-
LCVX_N_250_L16	10514	10	8	-	11	10	-	0.01306	0.00984	0.05888	0.03269	0.00824	-
LCVX_N_250_L17	10514	10	8	-	11	10	-	0.01316	0.01139	0.04364	0.03279	0.00822	-
LCVX_N_250_L18	10514	13	9	-	12	11	-	0.02033	0.01271	0.06754	0.03774	0.01034	-
LCVX_N_250_L19	10514	10	9	-	11	10	-	0.01540	0.01307	0.06816	0.03144	0.00956	-
LCVX_N_300_L0	12614	11	11	-	13	11	-	0.01840	0.01611	0.06711	0.04499	0.01068	-
LCVX_N_300_L1	12614	11	9	-	12	10	-	0.01711	0.01391	0.07451	0.04436	0.00989	-
LCVX_N_300_L2	12614	10	11	-	13	11	-	0.01689	0.01585	0.05271	0.04656	0.01069	-
LCVX_N_300_L3	12614	12	10	-	13	12	-	0.01890	0.01467	0.07362	0.04415	0.01154	-
LCVX_N_300_L4	12614	10	8	-	12	10	-	0.01585	0.01191	0.07343	0.04025	0.00994	-
LCVX_N_300_L5	12614	12	10	-	14	11	-	0.01901	0.01476	0.08295	0.04839	0.01077	-
LCVX_N_300_L6	12614	10	8	-	13	10	-	0.01555	0.01262	0.05116	0.04782	0.01023	-
LCVX_N_300_L7	12614	9	8	-	12	10	-	0.01461	0.01368	0.07382	0.04529	0.01144	-
LCVX_N_300_L8	12614	10	9	-	13	11	-	0.01857	0.01335	0.05193	0.04822	0.01078	-
LCVX_N_300_L9	12614	11	9	-	13	11	-	0.01695	0.01322	0.06888	0.04389	0.01076	-
LCVX_N_300_L10	12614	13	10	-	13	11	-	0.02362	0.01457	0.08371	0.04780	0.01241	-
LCVX_N_300_L11	12614	10	9	-	13	11	-	0.01585	0.01350	0.05985	0.04982	0.01250	-
LCVX_N_300_L12	12614	11	9	-	13	11	-	0.01714	0.01542	0.07645	0.04790	0.01251	-
LCVX_N_300_L13	12614	9	9	-	13	11	-	0.01711	0.01361	0.04904	0.04500	0.01081	-
LCVX_N_300_L14	12614	10	9	-	12	10	-	0.01572	0.01389	0.08113	0.04145	0.00987	-
LCVX_N_300_L15	12614	11	8	-	12	10	-	0.01689	0.01197	0.06695	0.04455	0.00989	-
LCVX_N_300_L16	12614	9	8	-	12	10	-	0.01464	0.01368	0.05260	0.04061	0.01152	-
LCVX_N_300_L17	12614	10	8	-	12	10	-	0.01862	0.01196	0.08553	0.04934	0.00985	-
LCVX_N_300_L18	12614	9	8	-	12	10	-	0.01446	0.01191	0.04953	0.04100	0.00994	-
LCVX_N_300_L19	12614	10	10	-	13	11	-	0.01679	0.01469	0.05266	0.04568	0.01084	-
LCVX_N_350_L0	14714	9	9	-	13	11	-	0.02004	0.01545	0.07680	0.05598	0.01249	-
LCVX_N_350_L1	14714	11	10	-	13	11	-	0.02061	0.01728	0.06032	0.05012	0.01260	-
LCVX_N_350_L2	14714	11	9	-	12	11	-	0.01994	0.01560	0.06840	0.05095	0.01260	-
LCVX_N_350_L3	14714	10	8	-	12	10	-	0.01839	0.01380	0.07929	0.05342	0.01161	-

Table 4: Iterations and solver runtimes for lossless convexification problems

Problem	Size	<u>Iterations</u>						<u>Solver Runtime (s)</u>					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
LCVX_N_350_I_4	14714	9	8	-	12	10	-	0.01700	0.01389	0.05833	0.04700	0.01158	-
LCVX_N_350_I_5	14714	11	9	-	12	11	-	0.01983	0.01550	0.10310	0.05331	0.01261	-
LCVX_N_350_I_6	14714	10	10	-	12	11	-	0.01853	0.01738	0.06233	0.04658	0.01265	-
LCVX_N_350_I_7	14714	11	9	-	12	11	-	0.01990	0.01556	0.10463	0.05172	0.01256	-
LCVX_N_350_I_8	14714	10	8	-	12	10	-	0.01841	0.01379	0.05820	0.04862	0.01168	-
LCVX_N_350_I_9	14714	11	9	-	12	11	-	0.01967	0.01550	0.07686	0.04768	0.01260	-
LCVX_N_350_I_10	14714	9	8	-	12	11	-	0.01696	0.01403	0.06066	0.05296	0.01266	-
LCVX_N_350_I_11	14714	12	10	-	14	12	-	0.02212	0.01970	0.08369	0.05846	0.01574	-
LCVX_N_350_I_12	14714	12	10	-	14	12	-	0.02615	0.01714	0.07824	0.05940	0.01350	-
LCVX_N_350_I_13	14714	11	10	-	14	11	-	0.02099	0.01721	0.08116	0.05812	0.01254	-
LCVX_N_350_I_14	14714	10	11	-	13	11	-	0.01949	0.01833	0.07396	0.05615	0.01257	-
LCVX_N_350_I_15	14714	11	9	-	14	11	-	0.01968	0.01787	0.07639	0.06019	0.01456	-
LCVX_N_350_I_16	14714	11	9	-	12	11	-	0.02357	0.01550	0.07644	0.05193	0.01264	-
LCVX_N_350_I_17	14714	8	8	-	12	10	-	0.01551	0.01374	0.05710	0.05051	0.01343	-
LCVX_N_350_I_18	14714	9	11	-	13	11	-	0.01701	0.01860	0.06107	0.05173	0.01263	-
LCVX_N_350_I_19	14714	10	8	-	12	10	-	0.01837	0.01601	0.05973	0.05333	0.01342	-

Table 5: Iterations and solver runtimes for group lasso problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
GROUP_LASSO_N_1.L.0	761	4	11	-	5	6	6	0.00025	0.00109	0.01260	0.00126	0.00019	0.00006
GROUP_LASSO_N_1.L.1	761	4	11	-	7	6	6	0.00026	0.00108	0.01325	0.00153	0.00020	0.00006
GROUP_LASSO_N_1.L.2	761	4	10	-	7	6	6	0.00027	0.00097	0.01365	0.00152	0.00019	0.00006
GROUP_LASSO_N_1.L.3	761	4	12	-	7	6	6	0.00026	0.00119	0.01350	0.00156	0.00020	0.00007
GROUP_LASSO_N_1.L.4	761	4	11	-	7	6	6	0.00026	0.00110	0.01364	0.00151	0.00020	0.00007
GROUP_LASSO_N_1.L.5	761	4	11	-	7	6	6	0.00026	0.00112	0.01389	0.00151	0.00020	0.00006
GROUP_LASSO_N_1.L.6	761	4	11	-	6	6	6	0.00026	0.00107	0.01362	0.00139	0.00019	0.00006
GROUP_LASSO_N_1.L.7	761	5	10	-	7	6	6	0.00030	0.00097	0.01332	0.00150	0.00019	0.00006
GROUP_LASSO_N_1.L.8	761	4	12	-	7	6	6	0.00026	0.00114	0.01329	0.00159	0.00019	0.00006
GROUP_LASSO_N_1.L.9	761	5	10	-	6	6	6	0.00030	0.00109	0.01349	0.00148	0.00023	0.00007
GROUP_LASSO_N_1.L.10	761	4	9	-	7	6	6	0.00029	0.00101	0.01752	0.00167	0.00022	0.00007
GROUP_LASSO_N_1.L.11	761	4	11	-	5	6	6	0.00030	0.00123	0.01574	0.00139	0.00022	0.00007
GROUP_LASSO_N_1.L.12	761	5	9	-	6	6	6	0.00033	0.00100	0.01571	0.00153	0.00022	0.00007
GROUP_LASSO_N_1.L.13	761	4	11	-	7	6	6	0.00029	0.00122	0.01684	0.00166	0.00022	0.00007
GROUP_LASSO_N_1.L.14	761	4	12	-	6	6	6	0.00029	0.00129	0.01562	0.00154	0.00022	0.00007
GROUP_LASSO_N_1.L.15	761	4	12	-	7	6	6	0.00030	0.00121	0.01704	0.00169	0.00022	0.00007
GROUP_LASSO_N_1.L.16	761	4	11	-	7	6	6	0.00030	0.00121	0.01793	0.00166	0.00022	0.00007
GROUP_LASSO_N_1.L.17	761	4	12	-	7	6	6	0.00029	0.00120	0.01522	0.00164	0.00019	0.00006
GROUP_LASSO_N_1.L.18	761	5	11	-	7	6	6	0.00029	0.00110	0.01455	0.00152	0.00019	0.00007
GROUP_LASSO_N_1.L.19	761	5	10	-	5	6	6	0.00030	0.00098	0.01329	0.00126	0.00019	0.00007
GROUP_LASSO_N_2.L.0	2022	6	11	-	7	7	7	0.00080	0.00246	0.03452	0.00311	0.00054	0.00021
GROUP_LASSO_N_2.L.1	2022	7	11	-	7	7	7	0.00087	0.00248	0.04259	0.00309	0.00053	0.00022
GROUP_LASSO_N_2.L.2	2022	5	11	-	7	7	7	0.00073	0.00292	0.03949	0.00314	0.00057	0.00022
GROUP_LASSO_N_2.L.3	2022	7	12	-	8	8	8	0.00102	0.00316	0.04699	0.00371	0.00072	0.00024
GROUP_LASSO_N_2.L.4	2022	6	11	-	7	7	7	0.00092	0.00293	0.04221	0.00346	0.00062	0.00022
GROUP_LASSO_N_2.L.5	2022	7	11	-	9	8	8	0.00103	0.00243	0.04056	0.00357	0.00057	0.00021
GROUP_LASSO_N_2.L.6	2022	6	11	-	7	7	7	0.00079	0.00245	0.03730	0.00311	0.00052	0.00020
GROUP_LASSO_N_2.L.7	2022	6	10	-	7	7	7	0.00078	0.00230	0.03744	0.00310	0.00054	0.00020
GROUP_LASSO_N_2.L.8	2022	6	10	-	8	7	7	0.00079	0.00234	0.03938	0.00333	0.00053	0.00020
GROUP_LASSO_N_2.L.9	2022	6	11	-	8	7	7	0.00079	0.00235	0.03837	0.00336	0.00053	0.00022
GROUP_LASSO_N_2.L.10	2022	6	11	-	7	6	6	0.00080	0.00289	0.04676	0.00311	0.00056	0.00019
GROUP_LASSO_N_2.L.11	2022	7	11	-	7	7	7	0.00102	0.00287	0.04623	0.00345	0.00061	0.00019
GROUP_LASSO_N_2.L.12	2022	6	10	-	7	7	7	0.00091	0.00270	0.05127	0.00349	0.00061	0.00022
GROUP_LASSO_N_2.L.13	2022	7	13	-	7	7	7	0.00101	0.00347	0.04492	0.00356	0.00062	0.00022
GROUP_LASSO_N_2.L.14	2022	6	12	-	7	7	7	0.00094	0.00259	0.03859	0.00344	0.00055	0.00020
GROUP_LASSO_N_2.L.15	2022	6	11	-	8	7	7	0.00079	0.00243	0.03694	0.00336	0.00052	0.00022
GROUP_LASSO_N_2.L.16	2022	6	11	-	7	7	7	0.00081	0.00240	0.04211	0.00311	0.00052	0.00022
GROUP_LASSO_N_2.L.17	2022	6	11	-	7	7	7	0.00080	0.00251	0.03627	0.00307	0.00052	0.00020
GROUP_LASSO_N_2.L.18	2022	7	11	-	8	7	7	0.00087	0.00286	0.04580	0.00335	0.00062	0.00022
GROUP_LASSO_N_2.L.19	2022	6	10	-	8	7	7	0.00092	0.00235	0.03820	0.00342	0.00052	0.00022
GROUP_LASSO_N_3.L.0	3783	7	12	-	7	8	8	0.00179	0.00632	0.09303	0.00640	0.00136	0.00045
GROUP_LASSO_N_3.L.1	3783	7	12	-	8	8	8	0.00181	0.00663	0.10263	0.00699	0.00131	0.00045
GROUP_LASSO_N_3.L.2	3783	6	11	-	8	7	7	0.00167	0.00665	0.06413	0.00738	0.00134	0.00036
GROUP_LASSO_N_3.L.3	3783	6	11	-	8	7	7	0.00184	0.00611	0.09396	0.00745	0.00122	0.00040
GROUP_LASSO_N_3.L.4	3783	7	12	-	9	8	8	0.00179	0.00652	0.09673	0.00747	0.00129	0.00045
GROUP_LASSO_N_3.L.5	3783	6	12	-	8	7	7	0.00169	0.00627	0.09973	0.00699	0.00123	0.00040

Table 5: Iterations and solver runtimes for group lasso problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
GROUP_LASSO_N_3_I_6	3783	6	12	-	8	7	7	0.00167	0.00674	0.09808	0.00601	0.00123	0.00040
GROUP_LASSO_N_3_I_7	3783	7	12	-	9	8	8	0.00178	0.00665	0.10586	0.00756	0.00131	0.00045
GROUP_LASSO_N_3_I_8	3783	8	12	-	9	8	8	0.00195	0.00654	0.10374	0.00751	0.00131	0.00045
GROUP_LASSO_N_3_I_9	3783	7	12	-	9	7	7	0.00178	0.00671	0.09236	0.00744	0.00124	0.00040
GROUP_LASSO_N_3_I_10	3783	7	12	-	8	8	8	0.00179	0.00694	0.10241	0.00586	0.00145	0.00041
GROUP_LASSO_N_3_I_11	3783	8	12	-	10	8	8	0.00221	0.00650	0.09390	0.00712	0.00134	0.00046
GROUP_LASSO_N_3_I_12	3783	8	12	-	10	8	8	0.00194	0.00719	0.09329	0.00809	0.00146	0.00041
GROUP_LASSO_N_3_I_13	3783	7	12	-	8	7	7	0.00203	0.00664	0.09344	0.00617	0.00122	0.00041
GROUP_LASSO_N_3_I_14	3783	8	12	-	9	8	8	0.00198	0.00643	0.05164	0.00744	0.00136	0.00045
GROUP_LASSO_N_3_I_15	3783	8	12	-	9	8	8	0.00192	0.00626	0.09667	0.00624	0.00132	0.00045
GROUP_LASSO_N_3_I_16	3783	8	12	-	9	8	8	0.00198	0.00616	0.06490	0.00748	0.00137	0.00045
GROUP_LASSO_N_3_I_17	3783	7	11	-	9	7	7	0.00184	0.00646	0.09337	0.00752	0.00123	0.00036
GROUP_LASSO_N_3_I_18	3783	7	12	-	8	7	7	0.00200	0.00671	0.08989	0.00693	0.00124	0.00040
GROUP_LASSO_N_3_I_19	3783	7	12	-	8	8	8	0.00179	0.00707	0.09407	0.00697	0.00140	0.00039
GROUP_LASSO_N_4_I_0	6044	7	11	-	9	8	8	0.00372	0.00825	0.08675	0.00929	0.00254	0.00068
GROUP_LASSO_N_4_I_1	6044	7	11	-	9	7	7	0.00331	0.01034	0.07636	0.01163	0.00241	0.00060
GROUP_LASSO_N_4_I_2	6044	7	11	-	9	8	8	0.00332	0.00936	0.09203	0.01130	0.00233	0.00074
GROUP_LASSO_N_4_I_3	6044	6	11	-	7	7	7	0.00279	0.01065	0.07408	0.00972	0.00239	0.00060
GROUP_LASSO_N_4_I_4	6044	6	10	-	8	7	7	0.00309	0.00910	0.07713	0.01098	0.00240	0.00060
GROUP_LASSO_N_4_I_5	6044	7	11	-	9	8	8	0.00332	0.00932	0.07537	0.01131	0.00234	0.00075
GROUP_LASSO_N_4_I_6	6044	8	11	-	9	9	9	0.00319	0.01044	0.07465	0.01045	0.00272	0.00075
GROUP_LASSO_N_4_I_7	6044	7	11	-	9	8	8	0.00332	0.01038	0.08235	0.01195	0.00257	0.00067
GROUP_LASSO_N_4_I_8	6044	7	12	-	8	7	7	0.00332	0.01096	0.09033	0.01048	0.00239	0.00060
GROUP_LASSO_N_4_I_9	6044	8	12	-	9	8	8	0.00364	0.01153	0.09164	0.01165	0.00254	0.00067
GROUP_LASSO_N_4_I_10	6044	7	12	-	9	7	7	0.00338	0.01116	0.08548	0.00989	0.00240	0.00060
GROUP_LASSO_N_4_I_11	6044	7	11	-	8	7	7	0.00350	0.01017	0.08882	0.01088	0.00240	0.00061
GROUP_LASSO_N_4_I_12	6044	8	12	-	10	8	8	0.00373	0.01022	0.07332	0.01159	0.00231	0.00074
GROUP_LASSO_N_4_I_13	6044	8	12	-	10	8	8	0.00325	0.01025	0.08315	0.00932	0.00229	0.00074
GROUP_LASSO_N_4_I_14	6044	7	11	-	9	8	8	0.00303	0.01005	0.08752	0.01085	0.00255	0.00067
GROUP_LASSO_N_4_I_15	6044	7	11	-	8	7	7	0.00332	0.00827	0.08699	0.00823	0.00240	0.00060
GROUP_LASSO_N_4_I_16	6044	7	12	-	8	8	8	0.00328	0.01145	0.08421	0.01026	0.00248	0.00067
GROUP_LASSO_N_4_I_17	6044	7	12	-	8	7	7	0.00326	0.01063	0.07738	0.00827	0.00220	0.00066
GROUP_LASSO_N_4_I_18	6044	7	12	-	9	8	8	0.00307	0.01160	0.07562	0.01173	0.00255	0.00068
GROUP_LASSO_N_4_I_19	6044	7	11	-	9	8	8	0.00336	0.01011	0.08114	0.01054	0.00258	0.00067
GROUP_LASSO_N_5_I_0	8805	7	12	-	9	8	8	0.00539	0.01336	0.15490	0.01425	0.00376	0.00113
GROUP_LASSO_N_5_I_1	8805	7	12	-	8	8	8	0.00478	0.01432	0.12789	0.01328	0.00416	0.00099
GROUP_LASSO_N_5_I_2	8805	7	12	-	9	8	8	0.00527	0.01689	0.13607	0.01258	0.00412	0.00103
GROUP_LASSO_N_5_I_3	8805	7	12	-	9	8	8	0.00520	0.01569	0.12261	0.01514	0.00412	0.00101
GROUP_LASSO_N_5_I_4	8805	7	12	-	8	8	8	0.00522	0.01640	0.13366	0.01215	0.00409	0.00102
GROUP_LASSO_N_5_I_5	8805	8	12	-	9	8	8	0.00557	0.01594	0.13463	0.01448	0.00412	0.00103
GROUP_LASSO_N_5_I_6	8805	8	13	-	11	9	9	0.00557	0.01799	0.14506	0.01703	0.00430	0.00111
GROUP_LASSO_N_5_I_7	8805	8	13	-	11	9	9	0.00557	0.01593	0.13196	0.01626	0.00433	0.00114
GROUP_LASSO_N_5_I_8	8805	7	12	-	9	8	8	0.00578	0.01632	0.14206	0.01525	0.00414	0.00100
GROUP_LASSO_N_5_I_9	8805	8	12	-	10	9	9	0.00563	0.01265	0.10661	0.01594	0.00396	0.00125
GROUP_LASSO_N_5_I_10	8805	8	12	-	10	8	8	0.00507	0.01444	0.12481	0.01522	0.00386	0.00112
GROUP_LASSO_N_5_I_11	8805	7	12	-	9	8	8	0.00474	0.01315	0.13159	0.01476	0.00375	0.00113

Table 5: Iterations and solver runtimes for group lasso problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
GROUP_LASSO_N_5_I_12	8805	7	12	-	9	8	8	0.00471	0.01321	0.11506	0.01415	0.00376	0.00104
GROUP_LASSO_N_5_I_13	8805	7	12	-	9	8	8	0.00480	0.01528	0.12522	0.01420	0.00379	0.00113
GROUP_LASSO_N_5_I_14	8805	8	13	-	10	8	8	0.00504	0.01657	0.13945	0.01488	0.00412	0.00103
GROUP_LASSO_N_5_I_15	8805	6	11	-	9	7	7	0.00489	0.01440	0.15750	0.01530	0.00392	0.00091
GROUP_LASSO_N_5_I_16	8805	8	13	-	10	8	8	0.00556	0.01707	0.15121	0.01627	0.00411	0.00103
GROUP_LASSO_N_5_I_17	8805	7	12	-	8	8	8	0.00530	0.01363	0.12424	0.01428	0.00412	0.00098
GROUP_LASSO_N_5_I_18	8805	8	12	-	9	8	8	0.00552	0.01342	0.13743	0.01483	0.00376	0.00114
GROUP_LASSO_N_5_I_19	8805	7	12	-	8	8	8	0.00477	0.01616	0.15250	0.01364	0.00410	0.00103
GROUP_LASSO_N_8_I_0	20088	8	12	-	11	8	-	0.01663	0.03164	0.42595	0.03193	0.01264	-
GROUP_LASSO_N_8_I_1	20088	8	12	-	11	9	-	0.01533	0.03202	0.43969	0.03056	0.01308	-
GROUP_LASSO_N_8_I_2	20088	8	11	-	10	8	-	0.01536	0.03429	0.52114	0.03041	0.01263	-
GROUP_LASSO_N_8_I_3	20088	8	12	-	11	9	-	0.01680	0.03238	0.53397	0.03206	0.01312	-
GROUP_LASSO_N_8_I_4	20088	8	13	-	11	9	-	0.01536	0.03464	0.43105	0.03163	0.01310	-
GROUP_LASSO_N_8_I_5	20088	8	12	-	11	9	-	0.01529	0.03548	0.44647	0.03194	0.01441	-
GROUP_LASSO_N_8_I_6	20088	7	11	-	9	8	-	0.01607	0.03011	0.43955	0.03064	0.01260	-
GROUP_LASSO_N_8_I_7	20088	8	12	-	10	8	-	0.01534	0.03510	0.37856	0.03001	0.01385	-
GROUP_LASSO_N_8_I_8	20088	8	13	-	11	9	-	0.01667	0.03326	0.44272	0.03138	0.01322	-
GROUP_LASSO_N_8_I_9	20088	8	12	-	10	8	-	0.01531	0.03289	0.36489	0.02823	0.01272	-
GROUP_LASSO_N_8_I_10	20088	8	12	-	10	8	-	0.01546	0.02967	0.44515	0.03007	0.01382	-
GROUP_LASSO_N_8_I_11	20088	8	12	-	10	8	-	0.01682	0.03171	0.36262	0.02546	0.01268	-
GROUP_LASSO_N_8_I_12	20088	7	11	-	9	8	-	0.01448	0.03253	0.39282	0.02447	0.01382	-
GROUP_LASSO_N_8_I_13	20088	8	12	-	10	8	-	0.01684	0.03112	0.38244	0.03053	0.01271	-
GROUP_LASSO_N_8_I_14	20088	7	12	-	10	8	-	0.01587	0.03158	0.43929	0.02986	0.01263	-
GROUP_LASSO_N_8_I_15	20088	8	12	-	10	8	-	0.01534	0.03216	0.43672	0.02979	0.01272	-
GROUP_LASSO_N_8_I_16	20088	8	12	-	10	8	-	0.01541	0.03232	0.45250	0.03021	0.01268	-
GROUP_LASSO_N_8_I_17	20088	7	12	-	9	8	-	0.01487	0.03449	0.39412	0.02670	0.01387	-
GROUP_LASSO_N_8_I_18	20088	7	12	-	9	8	-	0.01594	0.03084	0.45118	0.02881	0.01258	-
GROUP_LASSO_N_8_I_19	20088	8	11	-	10	8	-	0.01543	0.03438	0.48799	0.03068	0.01379	-
GROUP_LASSO_N_10_I_0	30110	8	12	-	10	8	-	0.02990	0.04260	0.81705	0.03441	0.02297	-
GROUP_LASSO_N_10_I_1	30110	8	13	-	10	8	-	0.02640	0.04393	0.71839	0.03325	0.02295	-
GROUP_LASSO_N_10_I_2	30110	8	13	-	10	8	-	0.02634	0.04325	0.71089	0.03299	0.02309	-
GROUP_LASSO_N_10_I_3	30110	8	12	-	10	8	-	0.02629	0.04314	0.82863	0.03320	0.02288	-
GROUP_LASSO_N_10_I_4	30110	8	12	-	10	9	-	0.02683	0.04325	0.74742	0.03280	0.02406	-
GROUP_LASSO_N_10_I_5	30110	8	12	-	10	8	-	0.02697	0.04647	0.71730	0.03281	0.02506	-
GROUP_LASSO_N_10_I_6	30110	8	13	-	11	9	-	0.02953	0.04871	0.72293	0.03796	0.02569	-
GROUP_LASSO_N_10_I_7	30110	8	12	-	11	8	-	0.02942	0.04018	0.78965	0.03761	0.02297	-
GROUP_LASSO_N_10_I_8	30110	8	12	-	10	8	-	0.02650	0.04055	0.73732	0.03352	0.02290	-
GROUP_LASSO_N_10_I_9	30110	7	12	-	10	8	-	0.02551	0.04065	0.70643	0.03675	0.02300	-
GROUP_LASSO_N_10_I_10	30110	8	13	-	10	9	-	0.02617	0.04443	0.71020	0.03381	0.02367	-
GROUP_LASSO_N_10_I_11	30110	7	12	-	10	8	-	0.02521	0.04700	0.85205	0.03292	0.02512	-
GROUP_LASSO_N_10_I_12	30110	8	12	-	11	8	-	0.02932	0.03969	0.84706	0.03805	0.02333	-
GROUP_LASSO_N_10_I_13	30110	8	12	-	10	8	-	0.02654	0.04085	0.71497	0.03355	0.02305	-
GROUP_LASSO_N_10_I_14	30110	8	12	-	11	8	-	0.02646	0.04156	0.71650	0.03545	0.02291	-
GROUP_LASSO_N_10_I_15	30110	8	13	-	11	8	-	0.02719	0.04412	0.70731	0.03440	0.02288	-
GROUP_LASSO_N_10_I_16	30110	8	12	-	10	8	-	0.02660	0.04692	0.72722	0.03345	0.02508	-
GROUP_LASSO_N_10_I_17	30110	8	12	-	10	8	-	0.02956	0.04225	0.81926	0.03716	0.02288	-

Table 5: Iterations and solver runtimes for group lasso problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
GROUP_LASSO_N_10_L_18	30110	8	12	-	10	8	-	0.02757	0.04193	0.71638	0.03325	0.02331	-
GROUP_LASSO_N_10_L_19	30110	7	11	-	9	8	-	0.02621	0.03705	0.57778	0.03114	0.02293	-
GROUP_LASSO_N_12_L_0	42132	8	12	-	10	8	-	0.04176	0.05646	1.13886	0.04270	0.03342	-
GROUP_LASSO_N_12_L_1	42132	8	12	-	11	8	-	0.04718	0.06379	1.21747	0.04359	0.03531	-
GROUP_LASSO_N_12_L_2	42132	7	12	-	10	7	-	0.04523	0.06090	1.29220	0.04656	0.03292	-
GROUP_LASSO_N_12_L_3	42132	8	13	-	11	8	-	0.04103	0.06254	1.14120	0.04422	0.03333	-
GROUP_LASSO_N_12_L_4	42132	8	12	-	11	9	-	0.04334	0.05760	1.33111	0.04989	0.03406	-
GROUP_LASSO_N_12_L_5	42132	8	12	-	10	8	-	0.04179	0.05618	1.13627	0.04234	0.03340	-
GROUP_LASSO_N_12_L_6	42132	8	12	-	10	8	-	0.04743	0.06710	1.14156	0.04154	0.03561	-
GROUP_LASSO_N_12_L_7	42132	8	12	-	10	8	-	0.04925	0.06160	1.40540	0.04672	0.03352	-
GROUP_LASSO_N_12_L_8	42132	8	12	-	10	8	-	0.04718	0.05998	1.35485	0.04598	0.03311	-
GROUP_LASSO_N_12_L_9	42132	8	12	-	10	8	-	0.04905	0.05656	1.21177	0.04653	0.03330	-
GROUP_LASSO_N_12_L_10	42132	8	12	-	10	8	-	0.04734	0.06416	1.16632	0.04149	0.03473	-
GROUP_LASSO_N_12_L_11	42132	8	12	-	10	8	-	0.04929	0.05646	1.16314	0.05081	0.03312	-
GROUP_LASSO_N_12_L_12	42132	8	12	-	11	8	-	0.04759	0.06398	1.17661	0.05339	0.03473	-
GROUP_LASSO_N_12_L_13	42132	8	12	-	10	8	-	0.04926	0.05970	1.15317	0.05015	0.03347	-
GROUP_LASSO_N_12_L_14	42132	8	12	-	10	8	-	0.04236	0.05699	1.15607	0.04171	0.03296	-
GROUP_LASSO_N_12_L_15	42132	8	12	-	11	8	-	0.04935	0.06750	1.37147	0.04819	0.03532	-
GROUP_LASSO_N_12_L_16	42132	8	12	-	11	8	-	0.05005	0.06500	1.13286	0.04832	0.03555	-
GROUP_LASSO_N_12_L_17	42132	8	12	-	10	8	-	0.04763	0.06239	1.19049	0.04723	0.03542	-
GROUP_LASSO_N_12_L_18	42132	8	12	-	11	8	-	0.04341	0.06100	1.56013	0.04476	0.03346	-
GROUP_LASSO_N_12_L_19	42132	8	12	-	11	8	-	0.04741	0.06446	1.36957	0.05266	0.03500	-
GROUP_LASSO_N_14_L_0	56154	7	11	-	10	8	-	0.06732	0.08825	1.84934	0.06011	0.04971	-
GROUP_LASSO_N_14_L_1	56154	8	12	-	10	8	-	0.07072	0.09272	1.74199	0.06349	0.04997	-
GROUP_LASSO_N_14_L_2	56154	8	12	-	11	8	-	0.07052	0.08224	1.75298	0.06603	0.04582	-
GROUP_LASSO_N_14_L_3	56154	8	12	-	10	8	-	0.06288	0.08092	1.81541	0.05672	0.04587	-
GROUP_LASSO_N_14_L_4	56154	8	12	-	11	8	-	0.06408	0.08039	1.77375	0.06373	0.04576	-
GROUP_LASSO_N_14_L_5	56154	7	11	-	10	8	-	0.05957	0.08723	1.73803	0.05546	0.05051	-
GROUP_LASSO_N_14_L_6	56154	8	13	-	11	9	-	0.07036	0.08742	1.78430	0.06126	0.05179	-
GROUP_LASSO_N_14_L_7	56154	8	13	-	11	9	-	0.06455	0.08512	1.75818	0.05494	0.04697	-
GROUP_LASSO_N_14_L_8	56154	8	12	-	11	8	-	0.07086	0.08050	1.71973	0.06242	0.04588	-
GROUP_LASSO_N_14_L_9	56154	7	11	-	10	8	-	0.06089	0.07734	1.76336	0.05817	0.04569	-
GROUP_LASSO_N_14_L_10	56154	7	11	-	10	8	-	0.06136	0.07749	2.37906	0.06483	0.04582	-
GROUP_LASSO_N_14_L_11	56154	8	12	-	11	8	-	0.06271	0.08033	1.74188	0.05966	0.04585	-
GROUP_LASSO_N_14_L_12	56154	7	12	-	11	8	-	0.05962	0.09189	1.82984	0.05886	0.05023	-
GROUP_LASSO_N_14_L_13	56154	8	12	-	10	8	-	0.07108	0.08138	1.73734	0.05703	0.04582	-
GROUP_LASSO_N_14_L_14	56154	7	12	-	10	8	-	0.06805	0.09148	1.83138	0.05970	0.05013	-
GROUP_LASSO_N_14_L_15	56154	7	12	-	9	8	-	0.06833	0.08114	1.76118	0.06040	0.04593	-
GROUP_LASSO_N_14_L_16	56154	8	12	-	11	8	-	0.07149	0.08162	1.74090	0.05902	0.04518	-
GROUP_LASSO_N_14_L_17	56154	8	13	-	11	8	-	0.06544	0.08407	1.72602	0.05831	0.04578	-
GROUP_LASSO_N_14_L_18	56154	7	12	-	10	8	-	0.05882	0.08192	1.85183	0.06374	0.04648	-
GROUP_LASSO_N_14_L_19	56154	8	12	-	10	8	-	0.07138	0.08001	1.72838	0.05553	0.04590	-
GROUP_LASSO_N_16_L_0	72176	7	12	-	9	8	-	0.08672	0.10810	2.73494	0.06387	0.06441	-
GROUP_LASSO_N_16_L_1	72176	8	12	-	9	8	-	0.09175	0.10956	2.57650	0.06641	0.06458	-
GROUP_LASSO_N_16_L_2	72176	7	13	-	8	8	-	0.08420	0.11815	2.47541	0.06230	0.06381	-
GROUP_LASSO_N_16_L_3	72176	7	12	-	8	8	-	0.09670	0.11035	2.61814	0.06587	0.06468	-

Table 5: Iterations and solver runtimes for group lasso problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
GROUP_LASSO_N_16_L4	72176	7	12	-	9	8	-	0.08805	0.11655	2.53132	0.06833	0.06994	-
GROUP_LASSO_N_16_L5	72176	7	12	-	8	8	-	0.08591	0.11024	2.56714	0.06273	0.06479	-
GROUP_LASSO_N_16_L6	72176	8	12	-	9	8	-	0.09397	0.12189	2.57184	0.06543	0.07089	-
GROUP_LASSO_N_16_L7	72176	7	12	-	8	8	-	0.09662	0.11715	2.62896	0.07091	0.07108	-
GROUP_LASSO_N_16_L8	72176	7	12	-	8	8	-	0.09697	0.10666	2.52536	0.06546	0.06463	-
GROUP_LASSO_N_16_L9	72176	8	12	-	9	8	-	0.10105	0.11045	2.59268	0.07594	0.06850	-
GROUP_LASSO_N_16_L10	72176	8	12	-	8	8	-	0.08739	0.12383	2.55628	0.06493	0.07137	-
GROUP_LASSO_N_16_L11	72176	8	12	-	8	8	-	0.09985	0.11001	2.27946	0.07167	0.06446	-
GROUP_LASSO_N_16_L12	72176	8	12	-	9	8	-	0.09288	0.12516	2.65303	0.06511	0.07138	-
GROUP_LASSO_N_16_L13	72176	8	12	-	9	8	-	0.10170	0.12524	2.70107	0.06619	0.07118	-
GROUP_LASSO_N_16_L14	72176	8	12	-	8	8	-	0.10075	0.11424	2.69967	0.06278	0.06494	-
GROUP_LASSO_N_16_L15	72176	8	12	-	9	8	-	0.10100	0.12544	2.59960	0.06730	0.07098	-
GROUP_LASSO_N_16_L16	72176	8	12	-	9	8	-	0.10077	0.11113	2.91541	0.07759	0.06459	-
GROUP_LASSO_N_16_L17	72176	7	12	-	8	8	-	0.08758	0.12504	2.53327	0.07146	0.07131	-
GROUP_LASSO_N_16_L18	72176	8	12	-	8	8	-	0.10206	0.11034	2.64789	0.07121	0.06484	-
GROUP_LASSO_N_16_L19	72176	7	12	-	8	8	-	0.08468	0.11729	2.65421	0.07299	0.07011	-

Table 6: Iterations and solver runtimes for portfolio optimization problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
PORTFOLIO_N_2.1.0	804	9	12	14	12	9	9	0.00046	0.00099	0.00263	0.00485	0.00028	0.00013
PORTFOLIO_N_2.1.1	804	9	14	13	12	8	8	0.00045	0.00108	0.00255	0.00486	0.00027	0.00011
PORTFOLIO_N_2.1.2	804	8	12	11	11	8	8	0.00041	0.00102	0.00251	0.00461	0.00026	0.00010
PORTFOLIO_N_2.1.3	804	8	15	12	12	8	8	0.00041	0.00120	0.00252	0.00486	0.00025	0.00011
PORTFOLIO_N_2.1.4	804	9	14	13	11	8	8	0.00044	0.00113	0.00259	0.00461	0.00026	0.00010
PORTFOLIO_N_2.1.5	804	10	14	13	14	9	9	0.00049	0.00112	0.00256	0.00535	0.00027	0.00012
PORTFOLIO_N_2.1.6	804	10	18	12	12	10	10	0.00049	0.00151	0.00255	0.00485	0.00030	0.00014
PORTFOLIO_N_2.1.7	804	8	10	11	11	7	7	0.00041	0.00090	0.00248	0.00447	0.00024	0.00010
PORTFOLIO_N_2.1.8	804	9	17	11	10	9	9	0.00045	0.00150	0.00251	0.00423	0.00027	0.00012
PORTFOLIO_N_2.1.9	804	16	13	14	13	9	9	0.00070	0.00101	0.00258	0.00493	0.00028	0.00012
PORTFOLIO_N_2.1.10	804	10	13	15	13	9	9	0.00050	0.00100	0.00262	0.00490	0.00027	0.00011
PORTFOLIO_N_2.1.11	804	10	12	13	13	10	10	0.00044	0.00087	0.00229	0.00451	0.00028	0.00013
PORTFOLIO_N_2.1.12	804	10	11	13	12	10	10	0.00043	0.00079	0.00225	0.00426	0.00026	0.00012
PORTFOLIO_N_2.1.13	804	9	12	11	11	8	8	0.00040	0.00085	0.00220	0.00402	0.00023	0.00010
PORTFOLIO_N_2.1.14	804	9	14	13	13	9	9	0.00040	0.00095	0.00228	0.00444	0.00025	0.00011
PORTFOLIO_N_2.1.15	804	9	11	12	12	8	8	0.00040	0.00077	0.00224	0.00424	0.00022	0.00010
PORTFOLIO_N_2.1.16	804	10	14	11	12	8	8	0.00043	0.00095	0.00224	0.00425	0.00022	0.00010
PORTFOLIO_N_2.1.17	804	8	11	10	9	8	8	0.00037	0.00081	0.00219	0.00359	0.00023	0.00010
PORTFOLIO_N_2.1.18	804	9	13	11	11	9	9	0.00040	0.00092	0.00223	0.00400	0.00024	0.00011
PORTFOLIO_N_2.1.19	804	10	11	12	11	7	7	0.00044	0.00081	0.00224	0.00401	0.00021	0.00009
PORTFOLIO_N_4.1.0	2008	10	13	13	12	9	9	0.00107	0.00191	0.00293	0.00703	0.00054	0.00026
PORTFOLIO_N_4.1.1	2008	10	14	15	12	11	11	0.00107	0.00243	0.00310	0.00709	0.00062	0.00033
PORTFOLIO_N_4.1.2	2008	11	15	12	12	10	10	0.00131	0.00256	0.00287	0.00670	0.00058	0.00030
PORTFOLIO_N_4.1.3	2008	11	16	14	14	9	9	0.00113	0.00220	0.00299	0.00807	0.00056	0.00027
PORTFOLIO_N_4.1.4	2008	9	17	13	12	8	8	0.00098	0.00239	0.00284	0.00705	0.00049	0.00024
PORTFOLIO_N_4.1.5	2008	11	13	12	14	9	9	0.00113	0.00193	0.00279	0.00805	0.00053	0.00028
PORTFOLIO_N_4.1.6	2008	10	14	13	11	9	9	0.00107	0.00204	0.00290	0.00650	0.00054	0.00030
PORTFOLIO_N_4.1.7	2008	11	15	13	12	9	9	0.00110	0.00211	0.00293	0.00689	0.00053	0.00028
PORTFOLIO_N_4.1.8	2008	11	13	12	12	9	9	0.00109	0.00189	0.00292	0.00708	0.00053	0.00027
PORTFOLIO_N_4.1.9	2008	11	16	13	13	10	10	0.00111	0.00240	0.00295	0.00742	0.00057	0.00030
PORTFOLIO_N_4.1.10	2008	11	18	15	14	10	10	0.00112	0.00246	0.00311	0.00797	0.00057	0.00031
PORTFOLIO_N_4.1.11	2008	9	15	14	12	9	9	0.00097	0.00223	0.00291	0.00702	0.00056	0.00027
PORTFOLIO_N_4.1.12	2008	12	14	12	12	9	9	0.00120	0.00207	0.00278	0.00698	0.00054	0.00027
PORTFOLIO_N_4.1.13	2008	10	15	14	11	9	9	0.00102	0.00230	0.00303	0.00666	0.00053	0.00027
PORTFOLIO_N_4.1.14	2008	10	17	15	14	9	9	0.00106	0.00233	0.00305	0.00750	0.00054	0.00027
PORTFOLIO_N_4.1.15	2008	11	12	12	13	10	10	0.00115	0.00185	0.00289	0.00746	0.00058	0.00030
PORTFOLIO_N_4.1.16	2008	11	17	14	12	10	10	0.00114	0.00270	0.00292	0.00698	0.00058	0.00030
PORTFOLIO_N_4.1.17	2008	9	13	14	13	9	9	0.00098	0.00187	0.00292	0.00748	0.00052	0.00028
PORTFOLIO_N_4.1.18	2008	11	14	13	14	9	9	0.00112	0.00195	0.00292	0.00776	0.00052	0.00027
PORTFOLIO_N_4.1.19	2008	9	19	12	12	8	8	0.00096	0.00286	0.00285	0.00692	0.00049	0.00027
PORTFOLIO_N_6.1.0	3612	13	16	14	14	11	11	0.00215	0.00403	0.00395	0.01172	0.00112	0.00056
PORTFOLIO_N_6.1.1	3612	14	16	14	12	8	8	0.00224	0.00401	0.00387	0.01036	0.00090	0.00041
PORTFOLIO_N_6.1.2	3612	12	14	14	13	10	10	0.00210	0.00408	0.00452	0.01138	0.00124	0.00057
PORTFOLIO_N_6.1.3	3612	15	18	13	14	10	10	0.00279	0.00505	0.00437	0.01193	0.00121	0.00057
PORTFOLIO_N_6.1.4	3612	12	14	13	15	11	11	0.00235	0.00424	0.00442	0.01279	0.00129	0.00063
PORTFOLIO_N_6.1.5	3612	11	13	12	12	9	9	0.00221	0.00385	0.00422	0.01174	0.00112	0.00052

Table 6: Iterations and solver runtimes for portfolio optimization problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
PORTFOLIO_N_6_I_6	3612	10	15	14	14	9	9	0.00208	0.00457	0.00453	0.01331	0.00118	0.00052
PORTFOLIO_N_6_I_7	3612	12	14	14	13	9	9	0.00237	0.00393	0.00451	0.01255	0.00114	0.00051
PORTFOLIO_N_6_I_8	3612	12	16	14	14	10	10	0.00235	0.00405	0.00454	0.01340	0.00124	0.00053
PORTFOLIO_N_6_I_9	3612	13	15	15	14	10	10	0.00217	0.00389	0.00398	0.01178	0.00102	0.00057
PORTFOLIO_N_6_I_10	3612	11	15	14	13	10	10	0.00198	0.00365	0.00387	0.01253	0.00104	0.00050
PORTFOLIO_N_6_I_11	3612	11	14	13	11	9	9	0.00194	0.00415	0.00434	0.00978	0.00114	0.00051
PORTFOLIO_N_6_I_12	3612	12	15	16	13	11	11	0.00236	0.00388	0.00410	0.01128	0.00117	0.00056
PORTFOLIO_N_6_I_13	3612	11	15	13	13	10	10	0.00197	0.00428	0.00435	0.01116	0.00122	0.00053
PORTFOLIO_N_6_I_14	3612	13	16	14	14	10	10	0.00252	0.00461	0.00452	0.01329	0.00122	0.00058
PORTFOLIO_N_6_I_15	3612	12	17	13	14	10	10	0.00238	0.00484	0.00438	0.01335	0.00120	0.00058
PORTFOLIO_N_6_I_16	3612	11	17	14	12	9	9	0.00225	0.00431	0.00386	0.01178	0.00103	0.00047
PORTFOLIO_N_6_I_17	3612	13	15	15	13	10	10	0.00250	0.00414	0.00485	0.01259	0.00121	0.00057
PORTFOLIO_N_6_I_18	3612	12	20	15	14	10	10	0.00244	0.00614	0.00461	0.01338	0.00123	0.00057
PORTFOLIO_N_6_I_19	3612	13	13	12	14	9	9	0.00248	0.00333	0.00375	0.01202	0.00096	0.00046
PORTFOLIO_N_8_I_0	5616	11	15	14	14	11	11	0.00327	0.00665	0.00574	0.01784	0.00183	0.00101
PORTFOLIO_N_8_I_1	5616	12	15	13	13	11	11	0.00396	0.00685	0.00584	0.01858	0.00189	0.00097
PORTFOLIO_N_8_I_2	5616	12	16	13	13	11	11	0.00392	0.00691	0.00569	0.01836	0.00181	0.00090
PORTFOLIO_N_8_I_3	5616	13	15	16	15	10	10	0.00418	0.00551	0.00537	0.02087	0.00136	0.00087
PORTFOLIO_N_8_I_4	5616	13	16	13	12	11	11	0.00357	0.00635	0.00478	0.01551	0.00159	0.00100
PORTFOLIO_N_8_I_5	5616	12	15	13	15	10	10	0.00340	0.00580	0.00476	0.01890	0.00138	0.00083
PORTFOLIO_N_8_I_6	5616	14	17	14	14	10	10	0.00377	0.00628	0.00500	0.01778	0.00137	0.00085
PORTFOLIO_N_8_I_7	5616	12	14	15	15	9	9	0.00340	0.00544	0.00526	0.01896	0.00133	0.00084
PORTFOLIO_N_8_I_8	5616	13	17	14	15	11	11	0.00363	0.00637	0.00525	0.01838	0.00150	0.00088
PORTFOLIO_N_8_I_9	5616	14	18	16	15	10	10	0.00379	0.00621	0.00547	0.01805	0.00144	0.00084
PORTFOLIO_N_8_I_10	5616	13	18	15	16	10	10	0.00360	0.00625	0.00516	0.01937	0.00135	0.00087
PORTFOLIO_N_8_I_11	5616	12	18	15	14	10	10	0.00338	0.00660	0.00522	0.01747	0.00138	0.00084
PORTFOLIO_N_8_I_12	5616	11	15	13	13	9	9	0.00322	0.00561	0.00483	0.01652	0.00138	0.00074
PORTFOLIO_N_8_I_13	5616	14	17	15	14	10	10	0.00376	0.00652	0.00520	0.01770	0.00143	0.00083
PORTFOLIO_N_8_I_14	5616	12	17	15	15	10	10	0.00342	0.00622	0.00521	0.01825	0.00136	0.00086
PORTFOLIO_N_8_I_15	5616	14	18	15	15	10	10	0.00379	0.00653	0.00515	0.01854	0.00143	0.00084
PORTFOLIO_N_8_I_16	5616	14	17	15	15	12	12	0.00379	0.00609	0.00514	0.01866	0.00158	0.00098
PORTFOLIO_N_8_I_17	5616	12	15	15	13	9	9	0.00337	0.00571	0.00525	0.01621	0.00133	0.00075
PORTFOLIO_N_8_I_18	5616	12	14	14	15	10	10	0.00345	0.00547	0.00507	0.01890	0.00136	0.00083
PORTFOLIO_N_8_I_19	5616	12	16	15	15	10	10	0.00342	0.00583	0.00514	0.01860	0.00137	0.00083
PORTFOLIO_N_10_I_0	8020	13	18	13	14	9	9	0.00567	0.01012	0.00598	0.02258	0.00187	0.00144
PORTFOLIO_N_10_I_1	8020	12	19	14	13	10	10	0.00543	0.01160	0.00641	0.02101	0.00202	0.00157
PORTFOLIO_N_10_I_2	8020	14	18	15	14	10	10	0.00595	0.00946	0.00654	0.02224	0.00202	0.00158
PORTFOLIO_N_10_I_3	8020	14	16	15	15	10	10	0.00590	0.00945	0.00747	0.02371	0.00237	0.00178
PORTFOLIO_N_10_I_4	8020	12	16	15	14	9	9	0.00619	0.00860	0.00655	0.02196	0.00188	0.00143
PORTFOLIO_N_10_I_5	8020	11	15	15	14	11	11	0.00517	0.00979	0.00648	0.02214	0.00222	0.00195
PORTFOLIO_N_10_I_6	8020	14	17	17	15	11	11	0.00587	0.01039	0.00793	0.02337	0.00256	0.00198
PORTFOLIO_N_10_I_7	8020	14	15	14	14	11	11	0.00677	0.01008	0.00753	0.02500	0.00258	0.00195
PORTFOLIO_N_10_I_8	8020	13	16	16	13	11	11	0.00647	0.00867	0.00691	0.02054	0.00224	0.00172
PORTFOLIO_N_10_I_9	8020	12	15	15	13	11	11	0.00541	0.00941	0.00759	0.02041	0.00249	0.00195
PORTFOLIO_N_10_I_10	8020	13	19	15	14	11	11	0.00653	0.01125	0.00770	0.02481	0.00257	0.00196
PORTFOLIO_N_10_I_11	8020	14	18	17	15	10	10	0.00681	0.01118	0.00832	0.02592	0.00240	0.00161

Table 6: Iterations and solver runtimes for portfolio optimization problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
PORTFOLIO_N_10_I_12	8020	13	18	14	13	12	12	0.00645	0.00939	0.00615	0.02070	0.00242	0.00185
PORTFOLIO_N_10_I_13	8020	13	13	14	14	9	9	0.00562	0.00730	0.00627	0.02257	0.00187	0.00143
PORTFOLIO_N_10_I_14	8020	13	18	16	15	10	10	0.00577	0.00979	0.00696	0.02354	0.00206	0.00161
PORTFOLIO_N_10_I_15	8020	12	16	15	13	11	11	0.00540	0.00855	0.00646	0.02049	0.00221	0.00174
PORTFOLIO_N_10_I_16	8020	13	18	15	14	12	12	0.00570	0.00988	0.00650	0.02208	0.00235	0.00188
PORTFOLIO_N_10_I_17	8020	13	17	16	15	10	10	0.00572	0.00930	0.00695	0.02366	0.00207	0.00157
PORTFOLIO_N_10_I_18	8020	13	18	15	13	10	10	0.00565	0.01098	0.00751	0.02122	0.00236	0.00179
PORTFOLIO_N_10_I_19	8020	13	15	14	14	12	12	0.00643	0.00842	0.00650	0.02185	0.00245	0.00186
PORTFOLIO_N_15_I_0	15780	13	18	14	15	11	-	0.01573	0.01625	0.01104	0.03435	0.00476	-
PORTFOLIO_N_15_I_1	15780	13	18	15	16	11	-	0.01555	0.01698	0.01102	0.03537	0.00480	-
PORTFOLIO_N_15_I_2	15780	13	16	17	16	11	-	0.01575	0.01525	0.01227	0.03390	0.00488	-
PORTFOLIO_N_15_I_3	15780	12	18	15	15	12	-	0.01520	0.02108	0.01112	0.03209	0.00529	-
PORTFOLIO_N_15_I_4	15780	13	18	15	14	10	-	0.01562	0.01648	0.01098	0.03003	0.00447	-
PORTFOLIO_N_15_I_5	15780	13	15	15	15	10	-	0.01560	0.01402	0.01105	0.03340	0.00443	-
PORTFOLIO_N_15_I_6	15780	14	15	17	15	11	-	0.01616	0.01385	0.01165	0.03171	0.00479	-
PORTFOLIO_N_15_I_7	15780	13	16	16	15	10	-	0.01560	0.01443	0.01140	0.03200	0.00441	-
PORTFOLIO_N_15_I_8	15780	14	19	17	15	11	-	0.01606	0.01592	0.01228	0.03191	0.00484	-
PORTFOLIO_N_15_I_9	15780	13	14	16	15	10	-	0.01554	0.01470	0.01153	0.03193	0.00446	-
PORTFOLIO_N_15_I_10	15780	14	17	15	16	10	-	0.01614	0.01479	0.01128	0.03533	0.00447	-
PORTFOLIO_N_15_I_11	15780	13	17	14	15	11	-	0.01566	0.01509	0.01094	0.03390	0.00480	-
PORTFOLIO_N_15_I_12	15780	14	21	17	16	11	-	0.01614	0.01952	0.01213	0.03332	0.00481	-
PORTFOLIO_N_15_I_13	15780	13	16	15	14	12	-	0.01543	0.01474	0.01108	0.02999	0.00519	-
PORTFOLIO_N_15_I_14	15780	12	17	16	16	12	-	0.01511	0.01508	0.01151	0.03332	0.00521	-
PORTFOLIO_N_15_I_15	15780	15	17	18	16	13	-	0.01665	0.01487	0.01277	0.03564	0.00569	-
PORTFOLIO_N_15_I_16	15780	12	16	15	15	11	-	0.01519	0.01448	0.01143	0.03405	0.00487	-
PORTFOLIO_N_15_I_17	15780	13	15	15	14	11	-	0.01557	0.01546	0.01118	0.03211	0.00489	-
PORTFOLIO_N_15_I_18	15780	13	17	15	15	10	-	0.01562	0.01611	0.01145	0.03200	0.00438	-
PORTFOLIO_N_15_I_19	15780	12	17	16	14	10	-	0.01538	0.01628	0.01147	0.03246	0.00449	-
PORTFOLIO_N_20_I_0	26040	13	15	16	14	11	-	0.01579	0.02319	0.01691	0.04306	0.00867	-
PORTFOLIO_N_20_I_1	26040	13	19	16	15	10	-	0.01596	0.02887	0.01702	0.04652	0.00800	-
PORTFOLIO_N_20_I_2	26040	13	17	16	15	13	-	0.01584	0.02901	0.01678	0.04624	0.00982	-
PORTFOLIO_N_20_I_3	26040	13	16	16	14	11	-	0.01462	0.02260	0.01738	0.04299	0.00871	-
PORTFOLIO_N_20_I_4	26040	15	19	16	16	13	-	0.01763	0.03265	0.01731	0.04858	0.01017	-
PORTFOLIO_N_20_I_5	26040	13	16	16	15	10	-	0.01851	0.02140	0.01698	0.04604	0.00802	-
PORTFOLIO_N_20_I_6	26040	14	20	16	15	11	-	0.01711	0.02815	0.01691	0.04646	0.00873	-
PORTFOLIO_N_20_I_7	26040	15	17	17	15	11	-	0.01803	0.03054	0.02014	0.04598	0.00995	-
PORTFOLIO_N_20_I_8	26040	13	18	15	15	11	-	0.01829	0.02487	0.01659	0.04978	0.00834	-
PORTFOLIO_N_20_I_9	26040	14	17	15	15	11	-	0.01674	0.02765	0.01654	0.04574	0.00868	-
PORTFOLIO_N_20_I_10	26040	14	18	17	14	11	-	0.01678	0.02426	0.01734	0.04360	0.00856	-
PORTFOLIO_N_20_I_11	26040	13	16	17	15	13	-	0.01566	0.02243	0.01722	0.04618	0.01007	-
PORTFOLIO_N_20_I_12	26040	15	17	16	16	11	-	0.01770	0.02716	0.01693	0.04894	0.00866	-
PORTFOLIO_N_20_I_13	26040	13	15	14	13	10	-	0.01550	0.02817	0.01549	0.04090	0.00795	-
PORTFOLIO_N_20_I_14	26040	13	18	17	14	10	-	0.01559	0.02457	0.01755	0.04368	0.00788	-
PORTFOLIO_N_20_I_15	26040	13	16	16	14	11	-	0.01849	0.03036	0.01962	0.05036	0.01012	-
PORTFOLIO_N_20_I_16	26040	13	17	15	15	12	-	0.01848	0.02328	0.01900	0.05309	0.01077	-
PORTFOLIO_N_20_I_17	26040	13	18	17	16	12	-	0.01567	0.02475	0.01766	0.04937	0.00933	-

Table 6: Iterations and solver runtimes for portfolio optimization problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
PORTFOLIO_N_20_I_18	26040	13	18	17	15	10	-	0.01576	0.02845	0.01770	0.04576	0.00794	-
PORTFOLIO_N_20_I_19	26040	14	16	16	16	11	-	0.01681	0.02650	0.01691	0.04792	0.00869	-
PORTFOLIO_N_25_I_0	38800	15	18	18	15	12	-	0.02811	0.03697	0.02796	0.06077	0.01479	-
PORTFOLIO_N_25_I_1	38800	13	19	17	15	12	-	0.02489	0.03911	0.02666	0.06077	0.01491	-
PORTFOLIO_N_25_I_2	38800	14	15	17	16	11	-	0.02627	0.03189	0.02655	0.06440	0.01396	-
PORTFOLIO_N_25_I_3	38800	15	20	16	15	12	-	0.02832	0.03838	0.02565	0.06078	0.01489	-
PORTFOLIO_N_25_I_4	38800	13	16	16	15	11	-	0.02451	0.03297	0.02543	0.06389	0.01387	-
PORTFOLIO_N_25_I_5	38800	14	15	16	15	12	-	0.02645	0.03208	0.02487	0.06108	0.01474	-
PORTFOLIO_N_25_I_6	38800	14	18	16	15	11	-	0.02653	0.03622	0.02487	0.06103	0.01371	-
PORTFOLIO_N_25_I_7	38800	14	19	17	14	13	-	0.02668	0.04121	0.03103	0.05905	0.01871	-
PORTFOLIO_N_25_I_8	38800	14	17	16	15	11	-	0.03083	0.03273	0.02523	0.06138	0.01385	-
PORTFOLIO_N_25_I_9	38800	14	19	17	16	12	-	0.02653	0.03570	0.02635	0.06449	0.01502	-
PORTFOLIO_N_25_I_10	38800	14	18	15	15	11	-	0.03111	0.03593	0.02820	0.07083	0.01615	-
PORTFOLIO_N_25_I_11	38800	15	21	17	16	11	-	0.02816	0.03953	0.02618	0.06469	0.01383	-
PORTFOLIO_N_25_I_12	38800	15	18	16	15	12	-	0.02831	0.03316	0.02426	0.06071	0.01493	-
PORTFOLIO_N_25_I_13	38800	16	19	17	16	11	-	0.02977	0.03784	0.02601	0.06431	0.01380	-
PORTFOLIO_N_25_I_14	38800	14	16	17	15	10	-	0.02643	0.03204	0.02684	0.06072	0.01276	-
PORTFOLIO_N_25_I_15	38800	13	16	17	14	11	-	0.02484	0.03209	0.02575	0.05748	0.01383	-
PORTFOLIO_N_25_I_16	38800	14	19	16	14	12	-	0.02663	0.06186	0.02918	0.05802	0.01725	-
PORTFOLIO_N_25_I_17	38800	14	15	17	14	11	-	0.03080	0.03863	0.03059	0.06712	0.01616	-
PORTFOLIO_N_25_I_18	38800	15	18	16	16	12	-	0.03304	0.03557	0.02505	0.06457	0.01495	-
PORTFOLIO_N_25_I_19	38800	14	17	17	14	12	-	0.02603	0.03617	0.02648	0.05771	0.01506	-
PORTFOLIO_N_30_I_0	54060	15	17	17	15	11	-	0.04078	0.04232	0.03562	0.07782	0.01979	-
PORTFOLIO_N_30_I_1	54060	15	18	19	14	11	-	0.04038	0.04706	0.03848	0.07431	0.01976	-
PORTFOLIO_N_30_I_2	54060	14	20	16	14	14	-	0.03791	0.04927	0.03356	0.07436	0.02453	-
PORTFOLIO_N_30_I_3	54060	14	20	17	16	12	-	0.03790	0.05631	0.03541	0.08273	0.02143	-
PORTFOLIO_N_30_I_4	54060	14	17	17	15	13	-	0.03847	0.04522	0.03540	0.07840	0.02313	-
PORTFOLIO_N_30_I_5	54060	14	18	18	15	11	-	0.03849	0.04513	0.03699	0.08154	0.01995	-
PORTFOLIO_N_30_I_6	54060	12	19	16	14	11	-	0.03361	0.04782	0.03416	0.07452	0.01964	-
PORTFOLIO_N_30_I_7	54060	14	17	17	16	11	-	0.03834	0.04254	0.03533	0.08245	0.01989	-
PORTFOLIO_N_30_I_8	54060	15	17	16	15	12	-	0.04104	0.04122	0.03423	0.07857	0.02149	-
PORTFOLIO_N_30_I_9	54060	14	18	17	14	11	-	0.03878	0.05691	0.03565	0.07451	0.02303	-
PORTFOLIO_N_30_I_10	54060	13	18	16	14	11	-	0.04241	0.04240	0.03351	0.07457	0.02012	-
PORTFOLIO_N_30_I_11	54060	14	18	17	15	12	-	0.03828	0.05314	0.03513	0.07874	0.02485	-
PORTFOLIO_N_30_I_12	54060	14	18	16	16	11	-	0.04465	0.04759	0.03386	0.08244	0.01961	-
PORTFOLIO_N_30_I_13	54060	15	21	16	15	12	-	0.04048	0.05437	0.03384	0.07851	0.02141	-
PORTFOLIO_N_30_I_14	54060	16	19	17	15	11	-	0.04310	0.05265	0.03532	0.07844	0.01982	-
PORTFOLIO_N_30_I_15	54060	14	19	17	15	12	-	0.03834	0.04615	0.03562	0.07836	0.02172	-
PORTFOLIO_N_30_I_16	54060	15	17	17	16	14	-	0.04096	0.05073	0.04097	0.08245	0.02876	-
PORTFOLIO_N_30_I_17	54060	16	16	15	14	11	-	0.05060	0.04024	0.03210	0.07422	0.02012	-
PORTFOLIO_N_30_I_18	54060	15	19	16	15	12	-	0.04045	0.04823	0.03355	0.07831	0.02184	-
PORTFOLIO_N_30_I_19	54060	16	18	16	15	11	-	0.04294	0.04589	0.03384	0.07850	0.02013	-
PORTFOLIO_N_35_I_0	71820	15	19	18	16	12	-	0.05584	0.05739	0.04426	0.10356	0.02967	-
PORTFOLIO_N_35_I_1	71820	15	20	16	15	12	-	0.05662	0.06042	0.04109	0.09836	0.02957	-
PORTFOLIO_N_35_I_2	71820	14	18	16	14	11	-	0.05282	0.07135	0.04395	0.10664	0.03179	-
PORTFOLIO_N_35_I_3	71820	15	20	17	16	13	-	0.06657	0.07905	0.04376	0.10612	0.03325	-

Table 6: Iterations and solver runtimes for portfolio optimization problems

Problem	Size	Iterations						Solver Runtime (s)					
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM
PORTFOLIO_N_35_I_4	71820	16	19	16	15	13	-	0.07068	0.07492	0.04398	0.11254	0.03697	-
PORTFOLIO_N_35_I_5	71820	14	19	17	16	13	-	0.06208	0.06215	0.04377	0.11941	0.03192	-
PORTFOLIO_N_35_I_6	71820	15	18	16	15	12	-	0.05657	0.05702	0.03915	0.09702	0.02968	-
PORTFOLIO_N_35_I_7	71820	17	17	17	16	12	-	0.06331	0.05526	0.04299	0.10258	0.02950	-
PORTFOLIO_N_35_I_8	71820	15	20	16	16	13	-	0.05627	0.06575	0.04163	0.10279	0.03188	-
PORTFOLIO_N_35_I_9	71820	14	17	18	16	12	-	0.05318	0.06848	0.04473	0.10844	0.03426	-
PORTFOLIO_N_35_I_10	71820	15	18	17	17	11	-	0.06658	0.06619	0.04656	0.12444	0.03171	-
PORTFOLIO_N_35_I_11	71820	15	20	15	17	12	-	0.06651	0.06644	0.04067	0.10984	0.02958	-
PORTFOLIO_N_35_I_12	71820	15	18	16	15	11	-	0.05663	0.06125	0.04135	0.09925	0.02731	-
PORTFOLIO_N_35_I_13	71820	15	20	17	15	12	-	0.05644	0.06690	0.04293	0.09958	0.02954	-
PORTFOLIO_N_35_I_14	71820	16	20	19	16	13	-	0.05986	0.07504	0.04919	0.10278	0.03688	-
PORTFOLIO_N_35_I_15	71820	15	19	16	15	11	-	0.06674	0.06148	0.04025	0.11378	0.02719	-
PORTFOLIO_N_35_I_16	71820	14	18	18	15	13	-	0.05262	0.06579	0.04489	0.09922	0.03192	-
PORTFOLIO_N_35_I_17	71820	13	17	16	16	13	-	0.04921	0.05822	0.04275	0.10370	0.03170	-
PORTFOLIO_N_35_I_18	71820	16	19	18	16	13	-	0.05986	0.06595	0.04472	0.10778	0.03192	-
PORTFOLIO_N_35_I_19	71820	16	17	17	15	12	-	0.05994	0.05646	0.04273	0.09732	0.02933	-

Table 7: Iterations and solver runtimes for oscillating masses problems

Problem	Size	Iterations							Solver Runtime (s)						
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN
OSCILLATING_MASSES_N_8_I_0	1344	5	13	9	10	5	5	7	0.00055	0.00128	0.00327	0.00375	0.00029	0.00012	0.00016
OSCILLATING_MASSES_N_8_I_1	1344	5	14	9	10	5	5	6	0.00052	0.00132	0.00326	0.00377	0.00029	0.00011	0.00013
OSCILLATING_MASSES_N_8_I_2	1344	5	17	9	11	5	5	6	0.00052	0.00156	0.00329	0.00427	0.00030	0.00012	0.00014
OSCILLATING_MASSES_N_8_I_3	1344	5	14	9	11	5	5	6	0.00052	0.00130	0.00327	0.00422	0.00030	0.00011	0.00013
OSCILLATING_MASSES_N_8_I_4	1344	5	12	10	11	5	5	7	0.00053	0.00114	0.00338	0.00434	0.00030	0.00011	0.00015
OSCILLATING_MASSES_N_8_I_5	1344	5	12	9	10	5	5	6	0.00052	0.00113	0.00326	0.00381	0.00030	0.00012	0.00014
OSCILLATING_MASSES_N_8_I_6	1344	10	13	10	12	7	7	10	0.00093	0.00105	0.00304	0.00426	0.00034	0.00016	0.00022
OSCILLATING_MASSES_N_8_I_7	1344	5	13	12	11	5	5	6	0.00047	0.00119	0.00357	0.00400	0.00030	0.00012	0.00014
OSCILLATING_MASSES_N_8_I_8	1344	5	14	8	11	4	4	6	0.00052	0.00125	0.00316	0.00404	0.00026	0.00010	0.00013
OSCILLATING_MASSES_N_8_I_9	1344	5	15	9	11	5	5	6	0.00052	0.00133	0.00326	0.00407	0.00030	0.00011	0.00013
OSCILLATING_MASSES_N_8_I_10	1344	5	14	8	13	5	5	6	0.00052	0.00125	0.00317	0.00472	0.00030	0.00012	0.00014
OSCILLATING_MASSES_N_8_I_11	1344	5	13	10	12	5	5	6	0.00052	0.00119	0.00337	0.00426	0.00030	0.00011	0.00013
OSCILLATING_MASSES_N_8_I_12	1344	5	14	9	12	5	5	7	0.00052	0.00130	0.00332	0.00453	0.00030	0.00012	0.00016
OSCILLATING_MASSES_N_8_I_13	1344	5	12	9	10	5	5	6	0.00052	0.00113	0.00330	0.00375	0.00030	0.00012	0.00013
OSCILLATING_MASSES_N_8_I_14	1344	5	12	9	10	5	5	6	0.00052	0.00095	0.00284	0.00375	0.00026	0.00011	0.00012
OSCILLATING_MASSES_N_8_I_15	1344	5	13	9	10	5	5	7	0.00047	0.00105	0.00284	0.00353	0.00025	0.00011	0.00015
OSCILLATING_MASSES_N_8_I_16	1344	6	14	10	11	6	6	8	0.00055	0.00115	0.00293	0.00379	0.00029	0.00013	0.00017
OSCILLATING_MASSES_N_8_I_17	1344	5	13	8	11	5	5	6	0.00047	0.00102	0.00276	0.00365	0.00026	0.00012	0.00013
OSCILLATING_MASSES_N_8_I_18	1344	5	12	9	10	4	4	6	0.00047	0.00096	0.00286	0.00334	0.00023	0.00009	0.00012
OSCILLATING_MASSES_N_8_I_19	1344	5	12	9	11	5	5	6	0.00048	0.00097	0.00288	0.00379	0.00026	0.00011	0.00012
OSCILLATING_MASSES_N_20_I_0	3312	5	15	9	12	5	5	7	0.00111	0.00287	0.00466	0.01414	0.00060	0.00028	0.00040
OSCILLATING_MASSES_N_20_I_1	3312	5	14	10	13	5	5	7	0.00113	0.00263	0.00488	0.01447	0.00060	0.00028	0.00038
OSCILLATING_MASSES_N_20_I_2	3312	5	15	10	13	5	5	7	0.00112	0.00280	0.00497	0.01484	0.00061	0.00029	0.00040
OSCILLATING_MASSES_N_20_I_3	3312	7	14	12	13	7	7	8	0.00141	0.00265	0.00529	0.01475	0.00077	0.00038	0.00045
OSCILLATING_MASSES_N_20_I_4	3312	7	14	9	14	7	7	9	0.00146	0.00266	0.00462	0.01572	0.00077	0.00039	0.00051
OSCILLATING_MASSES_N_20_I_5	3312	5	14	9	12	5	5	6	0.00109	0.00262	0.00462	0.01343	0.00061	0.00028	0.00034
OSCILLATING_MASSES_N_20_I_6	3312	5	16	10	12	5	5	7	0.00110	0.00296	0.00492	0.01387	0.00061	0.00028	0.00040
OSCILLATING_MASSES_N_20_I_7	3312	5	14	9	13	5	5	6	0.00109	0.00270	0.00463	0.01458	0.00060	0.00029	0.00034
OSCILLATING_MASSES_N_20_I_8	3312	5	13	9	13	5	5	7	0.00111	0.00255	0.00467	0.01429	0.00062	0.00031	0.00039
OSCILLATING_MASSES_N_20_I_9	3312	5	16	10	13	5	5	6	0.00109	0.00297	0.00492	0.01376	0.00064	0.00027	0.00034
OSCILLATING_MASSES_N_20_I_10	3312	5	14	9	13	5	5	6	0.00110	0.00269	0.00463	0.01413	0.00062	0.00028	0.00035
OSCILLATING_MASSES_N_20_I_11	3312	5	14	9	13	5	5	7	0.00111	0.00268	0.00467	0.01373	0.00061	0.00031	0.00040
OSCILLATING_MASSES_N_20_I_12	3312	5	15	9	13	5	5	7	0.00112	0.00276	0.00465	0.01485	0.00062	0.00031	0.00040
OSCILLATING_MASSES_N_20_I_13	3312	5	14	9	12	5	5	6	0.00108	0.00262	0.00464	0.01416	0.00062	0.00027	0.00035
OSCILLATING_MASSES_N_20_I_14	3312	5	15	9	13	5	5	6	0.00109	0.00280	0.00465	0.01435	0.00061	0.00029	0.00038
OSCILLATING_MASSES_N_20_I_15	3312	5	13	9	12	5	5	6	0.00110	0.00253	0.00469	0.01373	0.00062	0.00029	0.00034
OSCILLATING_MASSES_N_20_I_16	3312	5	14	9	14	5	5	6	0.00109	0.00270	0.00460	0.01496	0.00061	0.00029	0.00035
OSCILLATING_MASSES_N_20_I_17	3312	5	15	10	13	5	5	7	0.00116	0.00286	0.00488	0.01485	0.00061	0.00029	0.00040
OSCILLATING_MASSES_N_20_I_18	3312	5	15	9	14	5	5	6	0.00111	0.00279	0.00467	0.01549	0.00061	0.00029	0.00035
OSCILLATING_MASSES_N_20_I_19	3312	5	15	9	13	5	5	7	0.00110	0.00288	0.00464	0.01489	0.00063	0.00028	0.00040
OSCILLATING_MASSES_N_32_I_0	5280	5	14	10	14	5	5	-	0.00172	0.00422	0.00671	0.02045	0.00097	0.00053	-
OSCILLATING_MASSES_N_32_I_1	5280	5	14	9	13	5	5	-	0.00181	0.00420	0.00629	0.01951	0.00096	0.00048	-
OSCILLATING_MASSES_N_32_I_2	5280	5	15	9	12	5	5	-	0.00173	0.00442	0.00628	0.01808	0.00101	0.00047	-
OSCILLATING_MASSES_N_32_I_3	5280	7	13	11	12	6	6	-	0.00219	0.00385	0.00695	0.01738	0.00109	0.00056	-
OSCILLATING_MASSES_N_32_I_4	5280	9	15	10	14	8	8	-	0.00281	0.00461	0.00681	0.02077	0.00137	0.00073	-
OSCILLATING_MASSES_N_32_I_5	5280	5	15	9	11	5	5	-	0.00172	0.00446	0.00631	0.01756	0.00098	0.00048	-

Table 7: Iterations and solver runtimes for oscillating masses problems

Problem	Size	Iterations							Solver Runtime (s)						
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN
OSCILLATING_MASSES_N_32_I_6	5280	6	14	10	11	6	6	-	0.00194	0.00415	0.00677	0.01658	0.00110	0.00056	-
OSCILLATING_MASSES_N_32_I_7	5280	6	15	11	12	6	6	-	0.00196	0.00443	0.00702	0.01703	0.00110	0.00058	-
OSCILLATING_MASSES_N_32_I_8	5280	5	16	9	13	5	5	-	0.00176	0.00477	0.00631	0.01762	0.00099	0.00048	-
OSCILLATING_MASSES_N_32_I_9	5280	5	14	9	12	5	5	-	0.00171	0.00414	0.00630	0.01706	0.00099	0.00048	-
OSCILLATING_MASSES_N_32_I_10	5280	6	15	10	12	5	5	-	0.00193	0.00448	0.00678	0.01870	0.00097	0.00048	-
OSCILLATING_MASSES_N_32_I_11	5280	5	13	9	12	5	5	-	0.00172	0.00391	0.00622	0.01769	0.00102	0.00048	-
OSCILLATING_MASSES_N_32_I_12	5280	5	15	9	11	5	5	-	0.00170	0.00438	0.00631	0.01586	0.00099	0.00049	-
OSCILLATING_MASSES_N_32_I_13	5280	5	15	9	14	5	5	-	0.00173	0.00454	0.00632	0.01974	0.00097	0.00048	-
OSCILLATING_MASSES_N_32_I_14	5280	6	16	10	11	6	6	-	0.00196	0.00470	0.00678	0.01605	0.00111	0.00057	-
OSCILLATING_MASSES_N_32_I_15	5280	5	16	9	13	5	5	-	0.00173	0.00480	0.00629	0.01822	0.00098	0.00053	-
OSCILLATING_MASSES_N_32_I_16	5280	5	15	9	13	5	5	-	0.00174	0.00444	0.00622	0.01941	0.00096	0.00048	-
OSCILLATING_MASSES_N_32_I_17	5280	5	15	10	12	5	5	-	0.00171	0.00446	0.00665	0.01877	0.00097	0.00048	-
OSCILLATING_MASSES_N_32_I_18	5280	5	15	9	13	5	5	-	0.00170	0.00459	0.00731	0.01839	0.00115	0.00048	-
OSCILLATING_MASSES_N_32_I_19	5280	5	15	10	13	5	5	-	0.00173	0.00454	0.00670	0.01873	0.00100	0.00048	-
OSCILLATING_MASSES_N_44_I_0	7248	5	15	9	12	5	5	-	0.00230	0.00611	0.00789	0.02372	0.00134	0.00065	-
OSCILLATING_MASSES_N_44_I_1	7248	6	16	10	12	6	6	-	0.00266	0.00640	0.00840	0.02427	0.00153	0.00080	-
OSCILLATING_MASSES_N_44_I_2	7248	6	15	11	13	6	6	-	0.00272	0.00604	0.00884	0.02677	0.00156	0.00080	-
OSCILLATING_MASSES_N_44_I_3	7248	5	14	9	13	5	5	-	0.00240	0.00599	0.00783	0.02612	0.00134	0.00068	-
OSCILLATING_MASSES_N_44_I_4	7248	5	16	9	13	5	5	-	0.00235	0.00668	0.00783	0.02650	0.00137	0.00069	-
OSCILLATING_MASSES_N_44_I_5	7248	5	14	9	12	5	5	-	0.00235	0.00573	0.00780	0.02321	0.00136	0.00068	-
OSCILLATING_MASSES_N_44_I_6	7248	6	15	11	13	6	6	-	0.00278	0.00623	0.00880	0.02700	0.00154	0.00080	-
OSCILLATING_MASSES_N_44_I_7	7248	5	15	9	12	5	5	-	0.00235	0.00624	0.00781	0.02492	0.00132	0.00066	-
OSCILLATING_MASSES_N_44_I_8	7248	5	15	9	13	5	5	-	0.00238	0.00622	0.00785	0.02579	0.00133	0.00069	-
OSCILLATING_MASSES_N_44_I_9	7248	5	15	9	12	5	5	-	0.00239	0.00604	0.00778	0.02401	0.00134	0.00069	-
OSCILLATING_MASSES_N_44_I_10	7248	5	15	9	13	5	5	-	0.00246	0.00608	0.00775	0.02662	0.00134	0.00069	-
OSCILLATING_MASSES_N_44_I_11	7248	5	14	9	12	5	5	-	0.00236	0.00586	0.00777	0.02545	0.00133	0.00068	-
OSCILLATING_MASSES_N_44_I_12	7248	5	16	9	13	5	5	-	0.00234	0.00642	0.00786	0.02667	0.00134	0.00068	-
OSCILLATING_MASSES_N_44_I_13	7248	5	14	9	12	5	5	-	0.00235	0.00589	0.00779	0.02546	0.00133	0.00076	-
OSCILLATING_MASSES_N_44_I_14	7248	5	16	9	13	5	5	-	0.00240	0.00668	0.00790	0.02632	0.00134	0.00068	-
OSCILLATING_MASSES_N_44_I_15	7248	5	14	9	14	5	5	-	0.00237	0.00578	0.00781	0.02633	0.00135	0.00069	-
OSCILLATING_MASSES_N_44_I_16	7248	5	16	9	13	5	5	-	0.00237	0.00647	0.00781	0.02624	0.00135	0.00068	-
OSCILLATING_MASSES_N_44_I_17	7248	5	15	9	13	5	5	-	0.00237	0.00618	0.00781	0.02594	0.00135	0.00070	-
OSCILLATING_MASSES_N_44_I_18	7248	5	14	9	13	5	5	-	0.00238	0.00576	0.00776	0.02651	0.00133	0.00069	-
OSCILLATING_MASSES_N_44_I_19	7248	5	14	9	13	5	5	-	0.00238	0.00584	0.00783	0.02656	0.00134	0.00068	-
OSCILLATING_MASSES_N_56_I_0	9216	5	15	9	14	5	5	-	0.00304	0.00893	0.00976	0.03563	0.00197	0.00101	-
OSCILLATING_MASSES_N_56_I_1	9216	5	16	9	12	5	5	-	0.00340	0.00968	0.00980	0.02983	0.00201	0.00101	-
OSCILLATING_MASSES_N_56_I_2	9216	5	16	9	12	5	5	-	0.00341	0.00834	0.00839	0.03087	0.00171	0.00092	-
OSCILLATING_MASSES_N_56_I_3	9216	5	15	9	13	5	5	-	0.00306	0.00785	0.00836	0.03244	0.00178	0.00092	-
OSCILLATING_MASSES_N_56_I_4	9216	5	15	9	13	5	5	-	0.00351	0.00900	0.00832	0.03240	0.00198	0.00101	-
OSCILLATING_MASSES_N_56_I_5	9216	7	16	11	13	6	6	-	0.00401	0.00826	0.00938	0.03206	0.00194	0.00107	-
OSCILLATING_MASSES_N_56_I_6	9216	5	14	9	12	5	5	-	0.00304	0.00734	0.00846	0.03137	0.00173	0.00090	-
OSCILLATING_MASSES_N_56_I_7	9216	5	19	10	14	5	5	-	0.00298	0.00972	0.00894	0.03386	0.00170	0.00091	-
OSCILLATING_MASSES_N_56_I_8	9216	5	14	9	12	5	5	-	0.00300	0.00740	0.00838	0.02979	0.00175	0.00091	-
OSCILLATING_MASSES_N_56_I_9	9216	6	16	9	13	6	6	-	0.00340	0.00822	0.00859	0.03321	0.00194	0.00105	-
OSCILLATING_MASSES_N_56_I_10	9216	5	19	9	12	5	5	-	0.00301	0.00978	0.00830	0.03279	0.00171	0.00091	-
OSCILLATING_MASSES_N_56_I_11	9216	5	14	9	12	5	5	-	0.00301	0.00738	0.00836	0.03012	0.00172	0.00089	-

Table 7: Iterations and solver runtimes for oscillating masses problems

Problem	Size	Iterations							Solver Runtime (s)						
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN
OSCILLATING_MASSES_N_56.l.12	9216	5	15	9	12	5	5	-	0.00299	0.00775	0.00846	0.03024	0.00172	0.00092	-
OSCILLATING_MASSES_N_56.l.13	9216	5	14	9	12	5	5	-	0.00299	0.00730	0.00832	0.03158	0.00170	0.00090	-
OSCILLATING_MASSES_N_56.l.14	9216	5	15	9	13	5	5	-	0.00298	0.00762	0.00838	0.03245	0.00172	0.00090	-
OSCILLATING_MASSES_N_56.l.15	9216	6	14	11	13	6	6	-	0.00351	0.00844	0.01098	0.03534	0.00228	0.00119	-
OSCILLATING_MASSES_N_56.l.16	9216	5	16	9	13	5	5	-	0.00353	0.00958	0.00973	0.03549	0.00200	0.00101	-
OSCILLATING_MASSES_N_56.l.17	9216	6	15	10	14	6	6	-	0.00393	0.00903	0.01064	0.03770	0.00225	0.00120	-
OSCILLATING_MASSES_N_56.l.18	9216	5	16	9	13	5	5	-	0.00351	0.00846	0.00840	0.03598	0.00170	0.00090	-
OSCILLATING_MASSES_N_56.l.19	9216	7	16	10	16	7	7	-	0.00404	0.00836	0.00929	0.03900	0.00224	0.00121	-
OSCILLATING_MASSES_N_76.l.0	12496	5	16	9	12	5	-	-	0.00404	0.01138	0.01050	0.04013	0.00236	-	-
OSCILLATING_MASSES_N_76.l.1	12496	6	15	10	13	5	-	-	0.00479	0.01055	0.01141	0.04462	0.00233	-	-
OSCILLATING_MASSES_N_76.l.2	12496	5	15	9	13	5	-	-	0.00409	0.01073	0.01062	0.04632	0.00236	-	-
OSCILLATING_MASSES_N_76.l.3	12496	5	14	9	13	6	-	-	0.00409	0.00973	0.01075	0.04351	0.00268	-	-
OSCILLATING_MASSES_N_76.l.4	12496	5	14	9	12	5	-	-	0.00406	0.01169	0.01070	0.04310	0.00235	-	-
OSCILLATING_MASSES_N_76.l.5	12496	5	15	9	13	5	-	-	0.00476	0.01237	0.01246	0.04689	0.00272	-	-
OSCILLATING_MASSES_N_76.l.6	12496	5	15	9	13	5	-	-	0.00472	0.01268	0.01276	0.04681	0.00270	-	-
OSCILLATING_MASSES_N_76.l.7	12496	5	15	9	14	5	-	-	0.00471	0.01057	0.01069	0.04750	0.00236	-	-
OSCILLATING_MASSES_N_76.l.8	12496	6	14	10	12	6	-	-	0.00469	0.00984	0.01144	0.04088	0.00266	-	-
OSCILLATING_MASSES_N_76.l.9	12496	5	14	10	14	5	-	-	0.00415	0.00987	0.01140	0.04779	0.00235	-	-
OSCILLATING_MASSES_N_76.l.10	12496	6	15	10	12	6	-	-	0.00454	0.01066	0.01137	0.04373	0.00266	-	-
OSCILLATING_MASSES_N_76.l.11	12496	5	14	9	12	5	-	-	0.00412	0.00997	0.01064	0.04083	0.00234	-	-
OSCILLATING_MASSES_N_76.l.12	12496	7	15	10	14	7	-	-	0.00533	0.01259	0.01348	0.04971	0.00346	-	-
OSCILLATING_MASSES_N_76.l.13	12496	7	17	12	14	6	-	-	0.00618	0.01371	0.01473	0.04861	0.00310	-	-
OSCILLATING_MASSES_N_76.l.14	12496	5	15	9	14	5	-	-	0.00477	0.01248	0.01232	0.05148	0.00271	-	-
OSCILLATING_MASSES_N_76.l.15	12496	5	16	9	13	5	-	-	0.00476	0.01134	0.01066	0.04689	0.00232	-	-
OSCILLATING_MASSES_N_76.l.16	12496	5	15	9	11	5	-	-	0.00433	0.01051	0.01066	0.03930	0.00235	-	-
OSCILLATING_MASSES_N_76.l.17	12496	5	15	9	13	5	-	-	0.00411	0.01072	0.01072	0.04533	0.00236	-	-
OSCILLATING_MASSES_N_76.l.18	12496	5	16	9	13	5	-	-	0.00406	0.01292	0.01258	0.04323	0.00273	-	-
OSCILLATING_MASSES_N_76.l.19	12496	5	15	9	12	5	-	-	0.00471	0.01225	0.01249	0.04638	0.00272	-	-
OSCILLATING_MASSES_N_96.l.0	15776	5	15	9	13	5	-	-	0.00602	0.01550	0.01450	0.05848	0.00348	-	-
OSCILLATING_MASSES_N_96.l.1	15776	6	16	11	13	6	-	-	0.00698	0.01420	0.01402	0.05978	0.00339	-	-
OSCILLATING_MASSES_N_96.l.2	15776	5	16	9	13	5	-	-	0.00511	0.01426	0.01238	0.05680	0.00300	-	-
OSCILLATING_MASSES_N_96.l.3	15776	5	17	9	13	5	-	-	0.00512	0.01756	0.01263	0.05612	0.00347	-	-
OSCILLATING_MASSES_N_96.l.4	15776	5	13	13	12	5	-	-	0.00581	0.01338	0.01845	0.05332	0.00346	-	-
OSCILLATING_MASSES_N_96.l.5	15776	5	15	9	15	5	-	-	0.00603	0.01337	0.01244	0.07083	0.00305	-	-
OSCILLATING_MASSES_N_96.l.6	15776	5	14	9	13	5	-	-	0.00512	0.01271	0.01272	0.05773	0.00301	-	-
OSCILLATING_MASSES_N_96.l.7	15776	5	15	9	13	5	-	-	0.00520	0.01367	0.01246	0.05746	0.00308	-	-
OSCILLATING_MASSES_N_96.l.8	15776	5	16	9	13	5	-	-	0.00523	0.01456	0.01244	0.05736	0.00300	-	-
OSCILLATING_MASSES_N_96.l.9	15776	5	15	9	11	5	-	-	0.00513	0.01334	0.01273	0.05200	0.00300	-	-
OSCILLATING_MASSES_N_96.l.10	15776	6	14	10	14	6	-	-	0.00571	0.01237	0.01338	0.05674	0.00340	-	-
OSCILLATING_MASSES_N_96.l.11	15776	6	15	10	12	6	-	-	0.00590	0.01362	0.01320	0.05358	0.00341	-	-
OSCILLATING_MASSES_N_96.l.12	15776	5	17	9	13	5	-	-	0.00512	0.01509	0.01248	0.05466	0.00298	-	-
OSCILLATING_MASSES_N_96.l.13	15776	5	15	9	14	5	-	-	0.00520	0.01339	0.01243	0.05965	0.00300	-	-
OSCILLATING_MASSES_N_96.l.14	15776	5	17	9	13	5	-	-	0.00518	0.01539	0.01246	0.05609	0.00302	-	-
OSCILLATING_MASSES_N_96.l.15	15776	6	15	11	12	6	-	-	0.00584	0.01363	0.01427	0.05236	0.00339	-	-
OSCILLATING_MASSES_N_96.l.16	15776	5	14	10	14	5	-	-	0.00528	0.01269	0.01350	0.05957	0.00299	-	-
OSCILLATING_MASSES_N_96.l.17	15776	5	16	9	15	5	-	-	0.00510	0.01405	0.01259	0.06245	0.00307	-	-

Table 7: Iterations and solver runtimes for oscillating masses problems

Problem	Size	Iterations							Solver Runtime (s)						
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN
OSCILLATING_MASSES_N_96_I_18	15776	5	14	9	12	5	-	-	0.00504	0.01257	0.01469	0.05114	0.00347	-	-
OSCILLATING_MASSES_N_96_I_19	15776	5	16	9	13	5	-	-	0.00514	0.01434	0.01248	0.05771	0.00298	-	-
OSCILLATING_MASSES_N_116_I_0	19056	5	15	9	13	5	-	-	0.00627	0.01601	0.01484	0.06574	0.00368	-	-
OSCILLATING_MASSES_N_116_I_1	19056	6	14	10	12	6	-	-	0.00695	0.01539	0.01599	0.06194	0.00419	-	-
OSCILLATING_MASSES_N_116_I_2	19056	5	15	9	12	5	-	-	0.00637	0.01643	0.01469	0.06273	0.00367	-	-
OSCILLATING_MASSES_N_116_I_3	19056	5	16	9	13	5	-	-	0.00622	0.01964	0.01696	0.06735	0.00422	-	-
OSCILLATING_MASSES_N_116_I_4	19056	5	15	10	12	5	-	-	0.00715	0.01912	0.01840	0.06859	0.00423	-	-
OSCILLATING_MASSES_N_116_I_5	19056	6	16	10	12	6	-	-	0.00811	0.01714	0.01554	0.07024	0.00412	-	-
OSCILLATING_MASSES_N_116_I_6	19056	6	14	10	10	6	-	-	0.00646	0.01515	0.01561	0.05609	0.00416	-	-
OSCILLATING_MASSES_N_116_I_7	19056	5	15	9	13	5	-	-	0.00638	0.01621	0.01461	0.06767	0.00369	-	-
OSCILLATING_MASSES_N_116_I_8	19056	7	17	11	14	7	-	-	0.00841	0.01864	0.01665	0.07346	0.00465	-	-
OSCILLATING_MASSES_N_116_I_9	19056	5	15	9	14	5	-	-	0.00626	0.01642	0.01470	0.07328	0.00373	-	-
OSCILLATING_MASSES_N_116_I_10	19056	5	14	10	13	5	-	-	0.00623	0.01727	0.01818	0.06714	0.00425	-	-
OSCILLATING_MASSES_N_116_I_11	19056	5	16	9	13	5	-	-	0.00700	0.01690	0.01750	0.06795	0.00426	-	-
OSCILLATING_MASSES_N_116_I_12	19056	5	16	9	12	5	-	-	0.00608	0.01737	0.01494	0.06339	0.00366	-	-
OSCILLATING_MASSES_N_116_I_13	19056	5	15	9	13	6	-	-	0.00619	0.01623	0.01471	0.07116	0.00419	-	-
OSCILLATING_MASSES_N_116_I_14	19056	6	15	10	13	6	-	-	0.00718	0.01647	0.01562	0.07111	0.00415	-	-
OSCILLATING_MASSES_N_116_I_15	19056	5	16	9	13	5	-	-	0.00619	0.01733	0.01465	0.07095	0.00366	-	-
OSCILLATING_MASSES_N_116_I_16	19056	5	14	9	14	5	-	-	0.00620	0.01531	0.01466	0.07134	0.00363	-	-
OSCILLATING_MASSES_N_116_I_17	19056	5	16	9	14	5	-	-	0.00627	0.02028	0.01707	0.07243	0.00424	-	-
OSCILLATING_MASSES_N_116_I_18	19056	5	15	10	12	6	-	-	0.00719	0.01678	0.01573	0.07083	0.00417	-	-
OSCILLATING_MASSES_N_116_I_19	19056	5	16	9	14	5	-	-	0.00635	0.01737	0.01461	0.06908	0.00362	-	-
OSCILLATING_MASSES_N_136_I_0	22336	5	16	9	13	5	-	-	0.00721	0.02032	0.01688	0.08071	0.00432	-	-
OSCILLATING_MASSES_N_136_I_1	22336	5	15	10	12	5	-	-	0.00733	0.01903	0.01791	0.07587	0.00430	-	-
OSCILLATING_MASSES_N_136_I_2	22336	5	14	9	12	5	-	-	0.00743	0.01827	0.01725	0.07667	0.00436	-	-
OSCILLATING_MASSES_N_136_I_3	22336	5	16	10	13	5	-	-	0.00734	0.02020	0.01796	0.08116	0.00437	-	-
OSCILLATING_MASSES_N_136_I_4	22336	5	15	9	13	5	-	-	0.00729	0.01963	0.01699	0.07803	0.00438	-	-
OSCILLATING_MASSES_N_136_I_5	22336	5	16	10	12	5	-	-	0.00737	0.02324	0.02100	0.07585	0.00503	-	-
OSCILLATING_MASSES_N_136_I_6	22336	5	15	10	14	5	-	-	0.00860	0.02233	0.02066	0.09022	0.00504	-	-
OSCILLATING_MASSES_N_136_I_7	22336	5	15	9	14	5	-	-	0.00839	0.01910	0.01967	0.09066	0.00509	-	-
OSCILLATING_MASSES_N_136_I_8	22336	5	15	9	13	5	-	-	0.00723	0.01913	0.01691	0.07890	0.00433	-	-
OSCILLATING_MASSES_N_136_I_9	22336	5	14	9	12	5	-	-	0.00725	0.01794	0.01715	0.07905	0.00436	-	-
OSCILLATING_MASSES_N_136_I_10	22336	6	14	10	13	6	-	-	0.00821	0.01806	0.01861	0.07904	0.00500	-	-
OSCILLATING_MASSES_N_136_I_11	22336	6	15	10	13	6	-	-	0.00817	0.01932	0.01792	0.07561	0.00504	-	-
OSCILLATING_MASSES_N_136_I_12	22336	5	15	9	13	5	-	-	0.00727	0.01911	0.01712	0.07888	0.00435	-	-
OSCILLATING_MASSES_N_136_I_13	22336	5	15	9	14	5	-	-	0.00751	0.01968	0.01682	0.08226	0.00433	-	-
OSCILLATING_MASSES_N_136_I_14	22336	5	16	9	14	5	-	-	0.00726	0.02032	0.01678	0.08191	0.00440	-	-
OSCILLATING_MASSES_N_136_I_15	22336	6	15	9	12	6	-	-	0.00809	0.01898	0.01684	0.07451	0.00493	-	-
OSCILLATING_MASSES_N_136_I_16	22336	6	16	10	12	6	-	-	0.00810	0.01970	0.01790	0.07366	0.00493	-	-
OSCILLATING_MASSES_N_136_I_17	22336	6	16	10	12	5	-	-	0.00806	0.01983	0.01801	0.07859	0.00436	-	-
OSCILLATING_MASSES_N_136_I_18	22336	5	17	9	13	5	-	-	0.00729	0.02159	0.01689	0.08000	0.00432	-	-
OSCILLATING_MASSES_N_136_I_19	22336	6	16	9	12	6	-	-	0.00810	0.01987	0.01685	0.07215	0.00495	-	-
OSCILLATING_MASSES_N_156_I_0	25616	6	15	10	14	7	-	-	0.00975	0.02223	0.02161	0.09490	0.00636	-	-
OSCILLATING_MASSES_N_156_I_1	25616	6	14	10	13	6	-	-	0.00928	0.02040	0.02191	0.09083	0.00581	-	-
OSCILLATING_MASSES_N_156_I_2	25616	7	20	12	14	5	-	-	0.01080	0.02879	0.02462	0.09866	0.00515	-	-
OSCILLATING_MASSES_N_156_I_3	25616	5	16	9	14	5	-	-	0.00833	0.02689	0.02182	0.09493	0.00589	-	-

Table 7: Iterations and solver runtimes for oscillating masses problems

Problem	Size	Iterations							Solver Runtime (s)						
		CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN	CLARABEL	ECOS	GUROBI	MOSEK	QOCO	QOCO_CUSTOM	CVXGEN
OSCILLATING_MASSES_N_156_I_4	25616	6	16	10	13	6	-	-	0.01073	0.02285	0.02305	0.09158	0.00571	-	-
OSCILLATING_MASSES_N_156_I_5	25616	5	16	9	12	5	-	-	0.00830	0.02309	0.02070	0.08317	0.00504	-	-
OSCILLATING_MASSES_N_156_I_6	25616	5	15	9	14	5	-	-	0.00851	0.02211	0.02133	0.09482	0.00500	-	-
OSCILLATING_MASSES_N_156_I_7	25616	5	16	9	13	5	-	-	0.00815	0.02310	0.02083	0.08544	0.00506	-	-
OSCILLATING_MASSES_N_156_I_8	25616	5	15	9	13	5	-	-	0.00841	0.02230	0.02131	0.09227	0.00514	-	-
OSCILLATING_MASSES_N_156_I_9	25616	5	15	9	14	5	-	-	0.00839	0.02164	0.02064	0.09737	0.00506	-	-
OSCILLATING_MASSES_N_156_I_10	25616	6	15	10	13	6	-	-	0.00960	0.02565	0.02176	0.09410	0.00671	-	-
OSCILLATING_MASSES_N_156_I_11	25616	5	17	9	13	5	-	-	0.00818	0.02474	0.02085	0.08873	0.00504	-	-
OSCILLATING_MASSES_N_156_I_12	25616	5	15	9	13	5	-	-	0.00840	0.02249	0.02040	0.09105	0.00503	-	-
OSCILLATING_MASSES_N_156_I_13	25616	5	14	9	12	5	-	-	0.00841	0.02069	0.02099	0.08307	0.00514	-	-
OSCILLATING_MASSES_N_156_I_14	25616	6	16	9	12	6	-	-	0.00932	0.02232	0.02112	0.08811	0.00664	-	-
OSCILLATING_MASSES_N_156_I_15	25616	5	14	9	14	5	-	-	0.00832	0.02064	0.02225	0.09329	0.00589	-	-
OSCILLATING_MASSES_N_156_I_16	25616	6	13	11	11	6	-	-	0.00927	0.01946	0.02327	0.07696	0.00576	-	-
OSCILLATING_MASSES_N_156_I_17	25616	5	17	9	13	5	-	-	0.00838	0.02463	0.02070	0.09187	0.00505	-	-
OSCILLATING_MASSES_N_156_I_18	25616	7	16	13	13	7	-	-	0.01111	0.02653	0.02615	0.08826	0.00740	-	-
OSCILLATING_MASSES_N_156_I_19	25616	5	15	9	13	5	-	-	0.00985	0.02505	0.02235	0.09991	0.00586	-	-